Árboles, búsquedas y optimización

Matemática Discreta II
Curso 2025/26

Dpto. de Matemática Aplicada a las TIC
E.T.S. Ingenieros Informáticos
Universidad Politécnica de Madrid



UNIVERSIDAD
POLITÉCNICA
DE MADRID



ESCUELA TÉCNICA
SUPERIOR DE INGENIEROS
INFORMÁTICOS

Contenidos

1 Árboles

2 Árboles etiquetados

3 Árboles generadores

4 Búsquedas en grafos

5 Grafos ponderados

6 Árboles generadores de peso mínimo

Árboles

- Árbol: definición, ejemplos, propiedades y caracterización.
- Árbol dirigido: definición y ejemplos.
- Árbol con raíz: definición, elementos y propiedades.

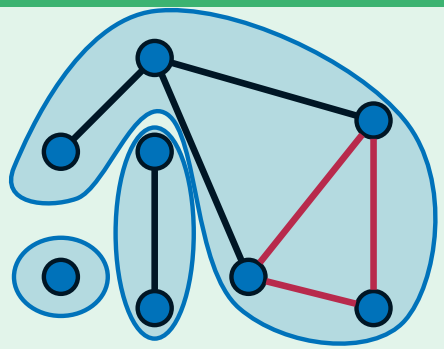
Árboles

Definición y propiedades

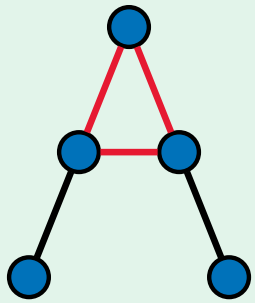
Definición

- Un grafo es **acíclico** si no contiene ciclos.
- Un grafo es un **árbol** si es conexo y acíclico.

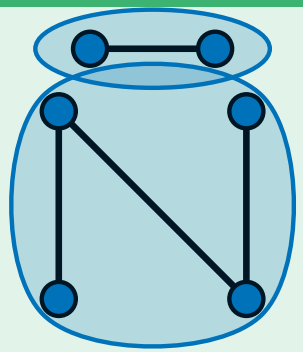
Ejemplo



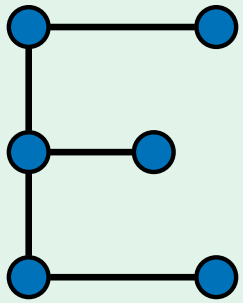
No es acíclico
No es conexo



No es acíclico
Es conexo



Es acíclico
No es conexo



Es árbol

Propiedades de los árboles

- Un árbol de n vértices tiene $n - 1$ aristas.
- Todo árbol no trivial tiene al menos dos vértices de grado 1.

Proposición de caracterización

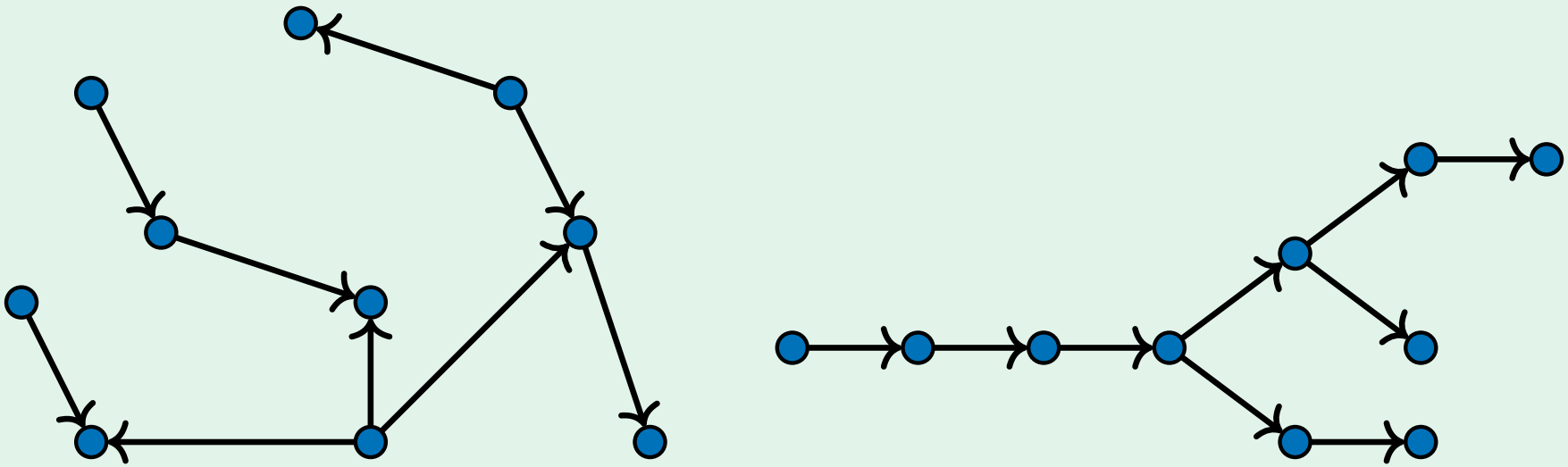
Sea $G = (V, E)$ un grafo tal que $|V| = n$. Las siguientes condiciones son equivalentes:

- G es un árbol.
- Entre cada par de vértices existe un camino único.
- G es conexo y toda arista es un puente.
- G es conexo y $|E| = n - 1$.
- G es acíclico y $|E| = n - 1$.

Definición

Un **árbol dirigido** es un digrafo cuyo grafo subyacente es un árbol.

Ejemplo (Árbol dirigido)



Árbol con raíz

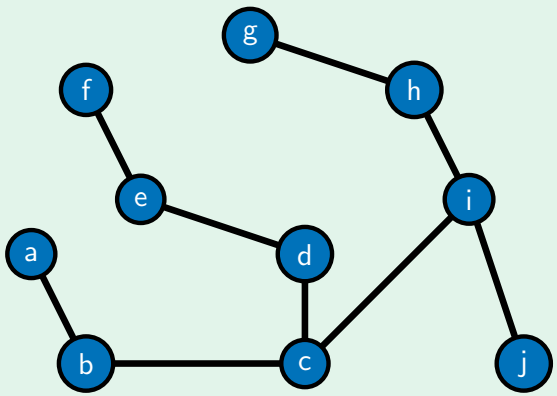
Definición y ejemplo

Definición

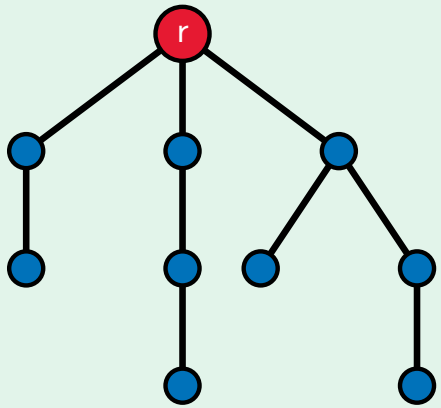
Un **árbol con raíz** es un árbol con un vértice distinguido r, llamado **raíz**. El único camino que une r con cada vértice determina una **orientación** en el árbol.

Ejemplo (Árboles con raíz)

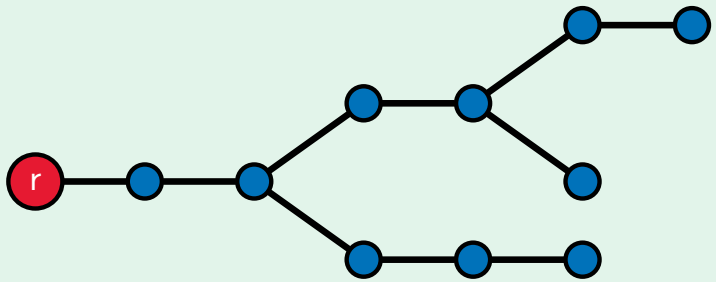
Árbol sin raíz



Árbol con raíz
$r = c$



Árbol con raíz
$r = f$



Elementos de un árbol con raíz

Altura y niveles

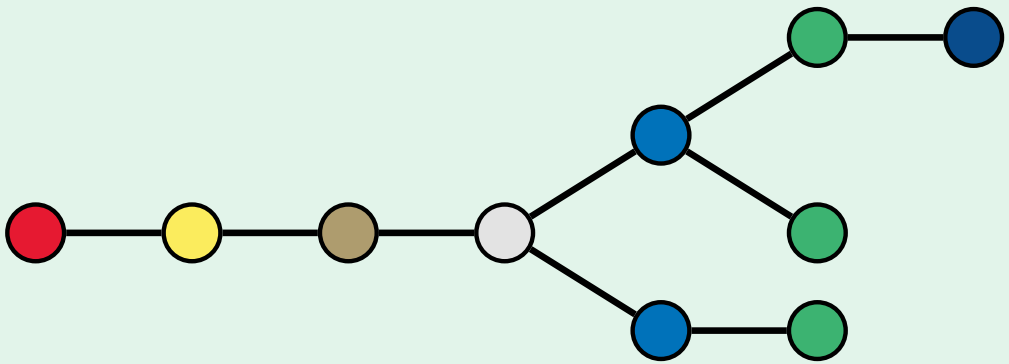
Definición

Sea T un árbol con raíz y m un entero positivo.

- La **altura** de T es la longitud del camino maximal de T. Se denota por $\text{alt}(T)$.
- El **nivel** m del árbol está formado por los vértices de T que están conectados con la raíz por un camino de longitud m.

Ejemplo (Elementos del árbol)

Árbol con raíz T



Nivel 1 Nivel 4
Nivel 2 Nivel 5
Nivel 3 Nivel 6

Elementos de un árbol con raíz

Altura, hojas y vértices interiores

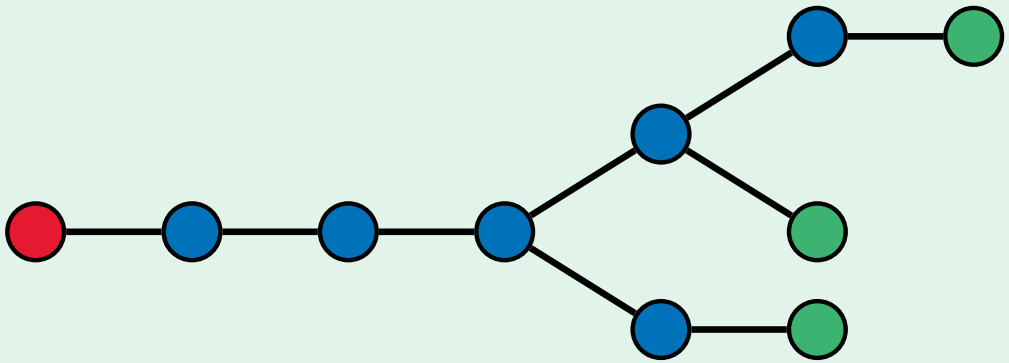
Definición

Sea T un árbol con raíz.

- Los vértices de grado 1 diferentes de la raíz son **hojas** de T.
- Los vértices que no son raíz ni hojas son **vértices interiores**.

Ejemplo (Elementos del árbol)

Árbol con raíz T



Raíz

Vértices interiores

Hojas

Elementos de un árbol con raíz

Ancestro, padre, descendiente e hijo

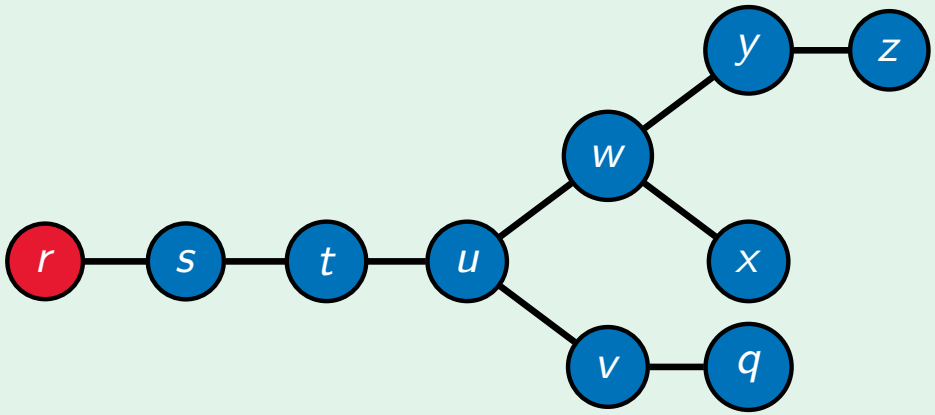
Definición

Sea T un árbol con raíz y v un vértice interior de T.

- Un vértice w es **ancestro** de v si el camino que une la raíz con v pasa por w. Si además es adyacente a v, entonces w es un **padre**.
- Un vértice w es **descendiente** de v si el camino que une la raíz con w pasa por v. Si además es adyacente a v, entonces w es un **hijo**.

Ejemplo (Elementos del árbol)

Árbol con raíz T



- s es ancestro de u
- t es padre de u
- q, x, y, z son descendientes de u
- v y w son hijos de u

Características de un árbol con raíz

Árbol m-ario. Árbol equilibrado

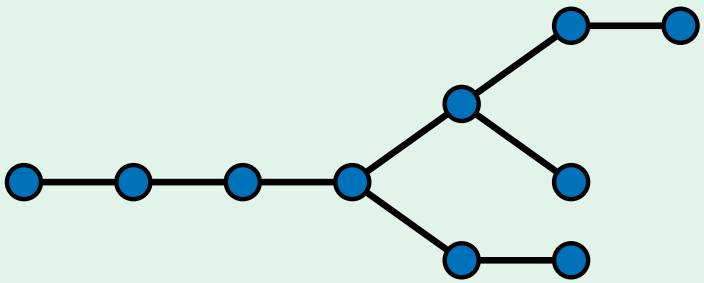
Definición

Sea T un árbol con raíz de altura $h = \text{alt}(T)$. Sea m un entero positivo.

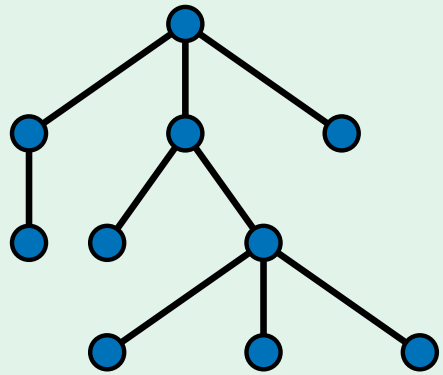
- T es un árbol **m-ario** si cada padre tiene como mucho m hijos. Es decir, si $\delta(v) \leq m + 1$ para todo vértice v del árbol.
- T es **equilibrado** si las hojas están en el nivel h ó $h - 1$.

Ejemplo

Árbol binario equilibrado



Árbol ternario no equilibrado



Árboles etiquetados

- Árboles etiquetados: definición e isomorfismos.
- Problemas enumerativos: código de Prüfer, fórmula de Cayley y descripción.

Árboles etiquetados

Árboles etiquetados

Definición

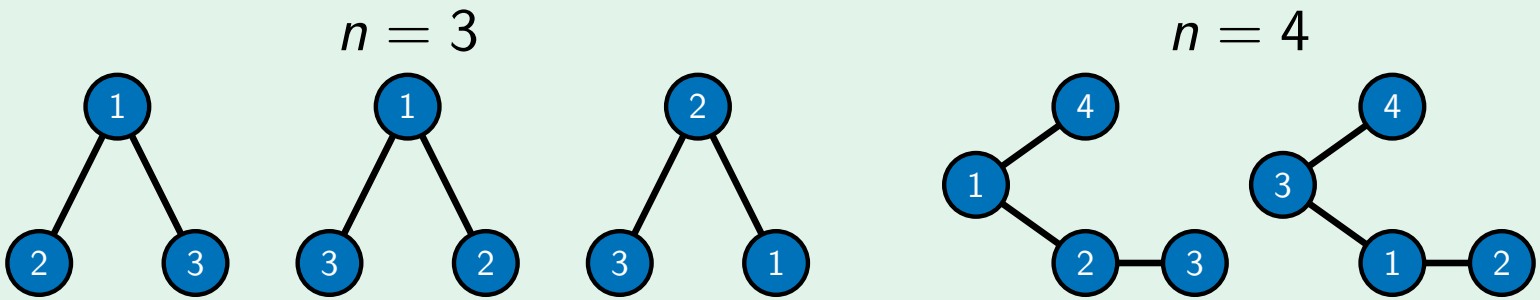
Definición

Sea $T = (V, E)$ un árbol con $|V| = n$. Decimos que T es **etiquetado por** $\{1, 2, \ldots, n\}$ si existe una aplicación biyectiva $L\colon V \to \{1, 2, \ldots, n\}$.
En ese caso, lo denotamos por (T, L)

Observación

Un árbol está etiquetado por un conjunto dado si a cada vértice le corresponde una etiqueta única de dicho conjunto.

Ejemplo (Árboles etiquetados)



Definición

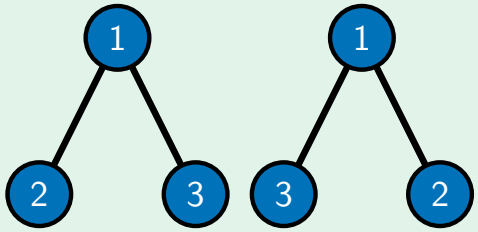
Sean (T, L) y (T', L') árboles etiquetados. Un isomorfismo de grafos $f : T \to T'$ es un **isomorfismo de árboles etiquetados** si se cumple $L(v) = L'(f(v))$ para todo vértice v de T.

Observación

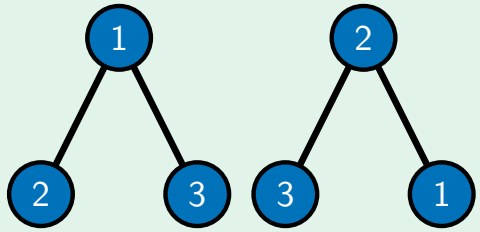
Un isomorfismo de árboles etiquetados es un isomorfismo de grafos que respeta las etiquetas.

Ejemplo (Isomorfismos de árboles etiquetados)

Son isomorfos



No son isomorfos

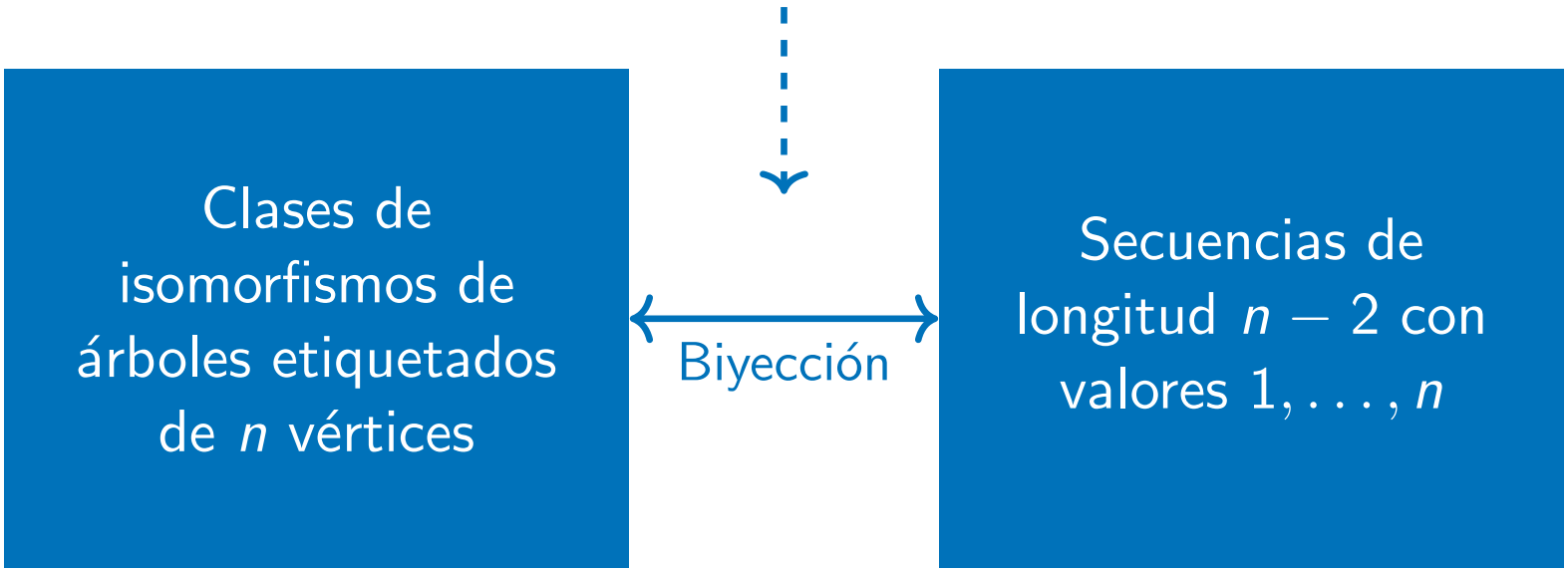


Problemas enumerativos

Problema I

¿Cuántas clases de isomorfismo de árboles etiquetados de n vértices existen?

El código de Prüfer



Código de Prüfer de un árbol etiquetado

Definición y reconstrucción del árbol etiquetado

Definición ($\rightarrow$)

Sea (T, L) un árbol etiquetado con $L = \{1, \ldots, n\}$. El **código de Prüfer** de T es la secuencia de etiquetas, de longitud n–2, obtenida de la siguiente manera:

1. Se anota el vértice adyacente a la hoja de T con menor etiqueta.
2. Se elimina dicha hoja y su arista adyacente, obteniendo un nuevo árbol con $n - 1$ vértices.
3. Se repite el proceso hasta que solo quedan dos vértices.

Reconstrucción del árbol etiquetado a partir del código de Prüfer ($\leftarrow$)

Sea $C = (c_1, c_2, \ldots, c_{n-2})$ un código de Prüfer.

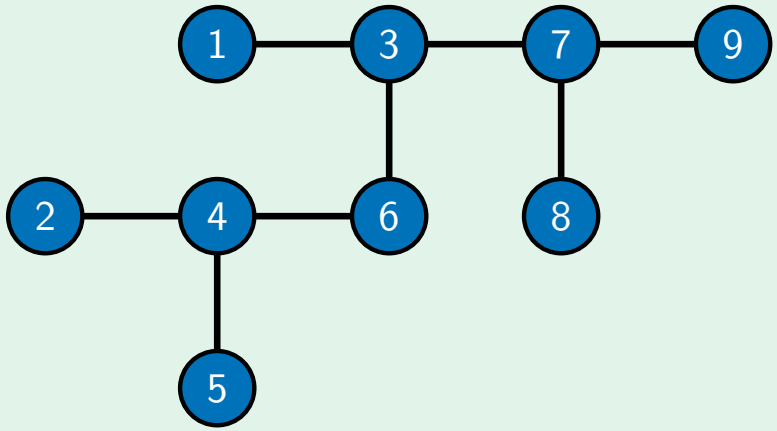
1. Se define $L = \{1, 2, \ldots, n\}$ y se dibujan n vértices etiquetados.
2. Se une el primer vértice en C con el menor vértice de L que no está en C.
3. Se eliminan de C y L los términos utilizados.
4. Se repite el proceso hasta que no quedan términos en C.
5. Se unen los dos vértices restantes de L

Ejemplo (Paso 0: Grafo inicial)

Obtención del código de Prüfer

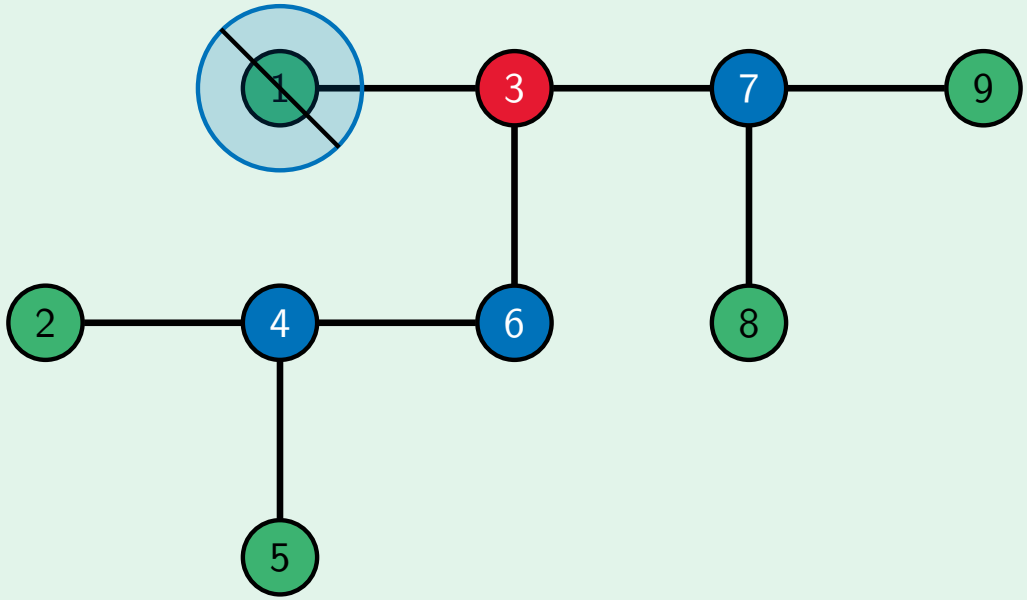
En cada paso:

- Coloreamos las **hojas**.
- Señalamos el **vértice adyacente** a la hoja con menor etiqueta.
- Eliminamos la ~~hoja~~ con menor etiqueta.



Ejemplo (Iteración 1)

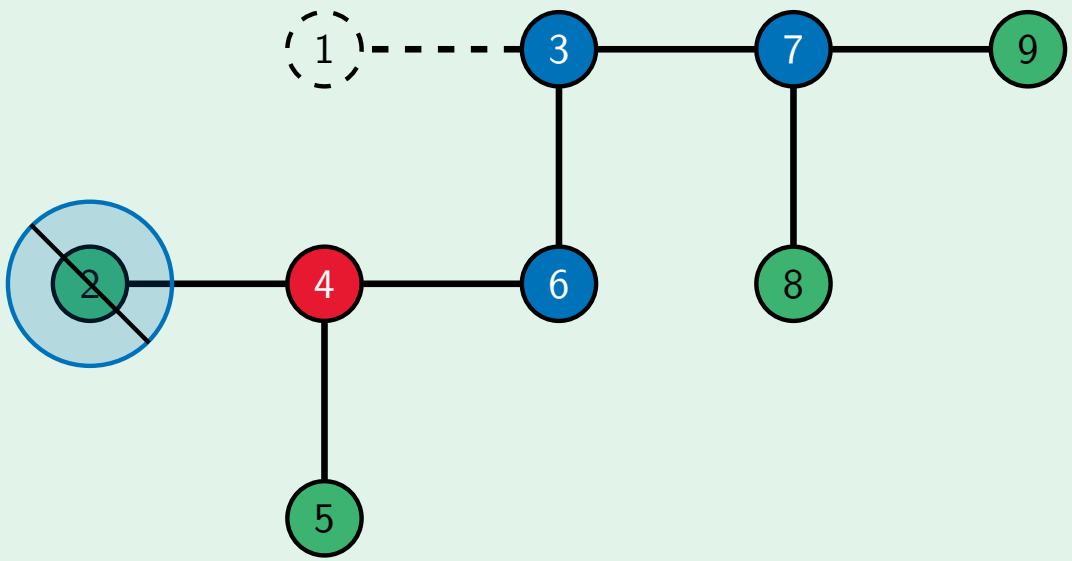
Código de Prüfer: 3



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 2)

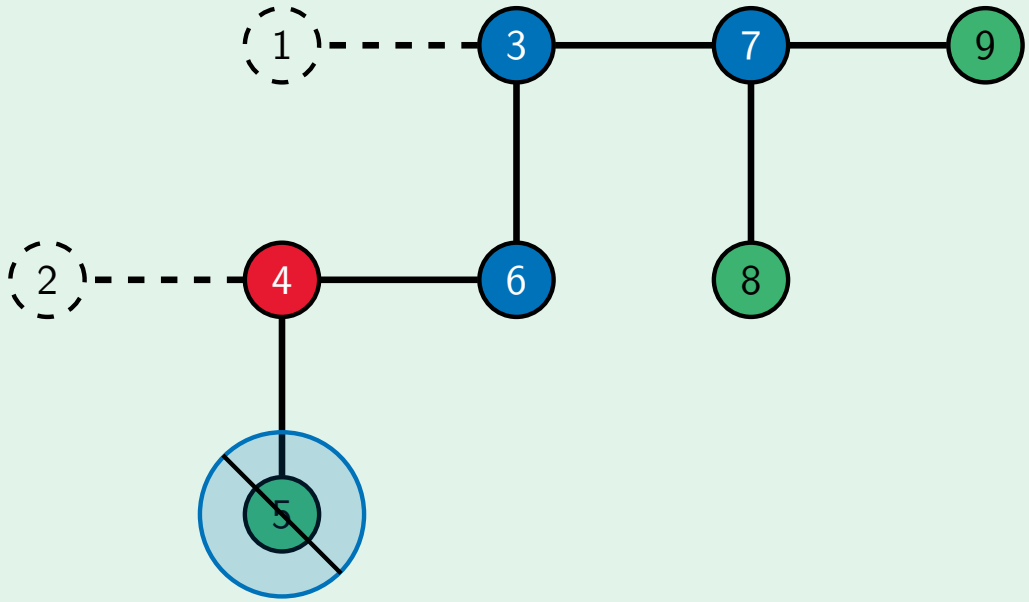
Código de Prüfer: 3, 4



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 3)

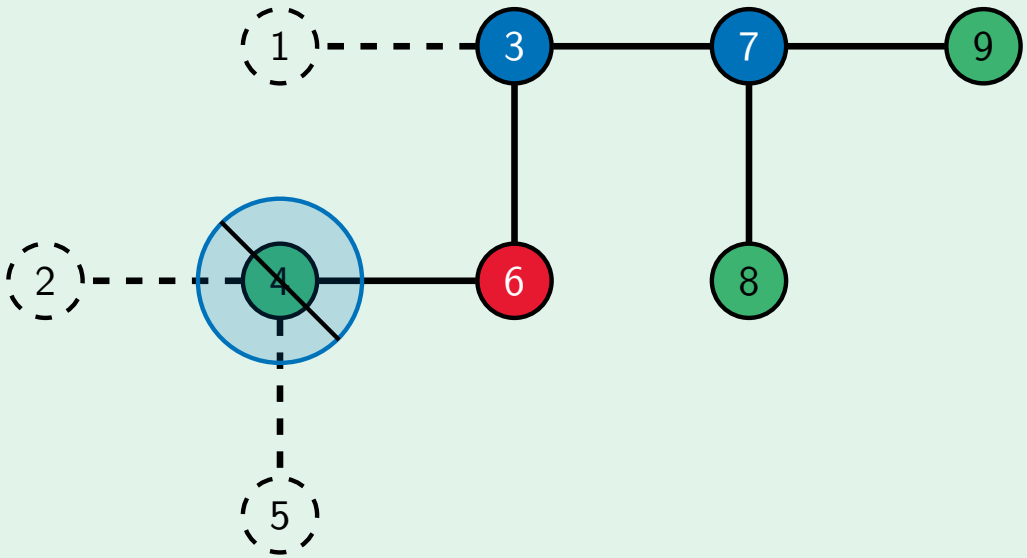
Código de Prüfer: $3, 4, 4$



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 4)

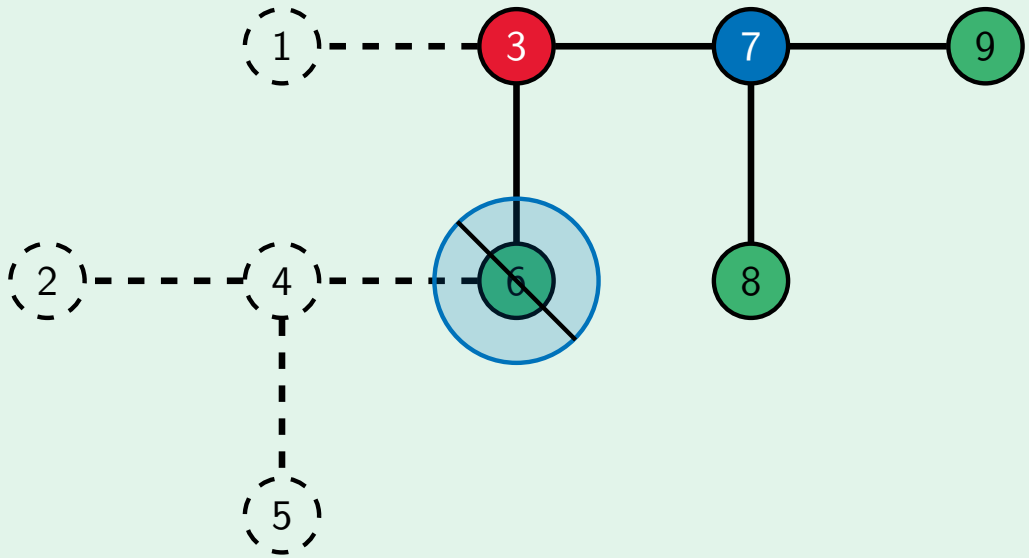
Código de Prüfer: $3, 4, 4, 6$



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 5)

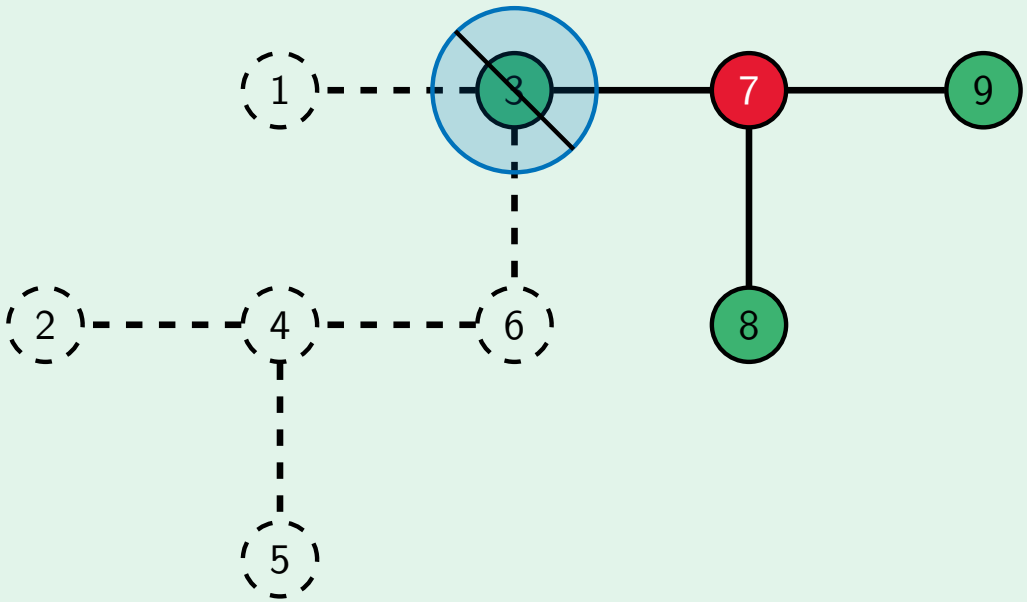
Código de Prüfer: $3, 4, 4, 6, 3$



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 6)

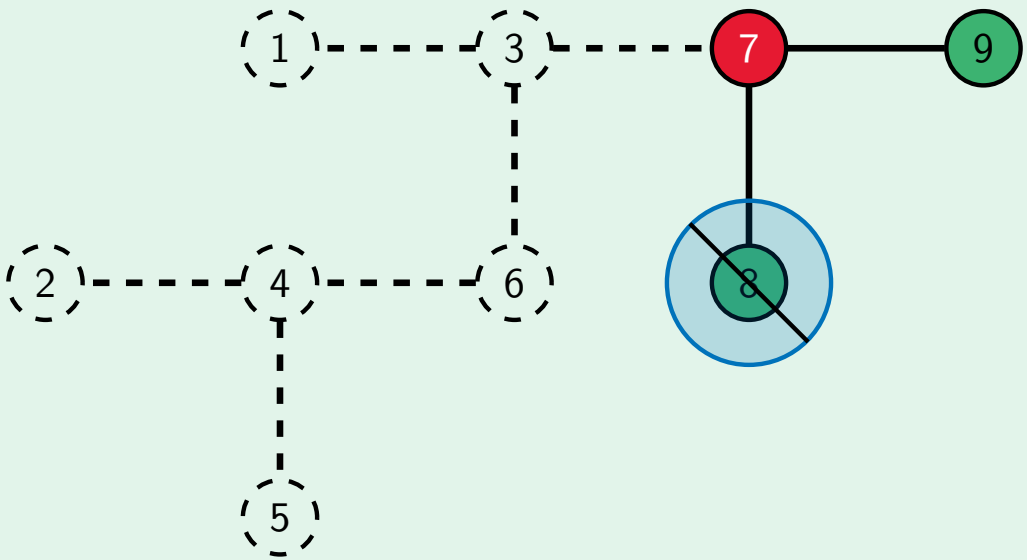
Código de Prüfer: $3, 4, 4, 6, 3, 7$



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 7)

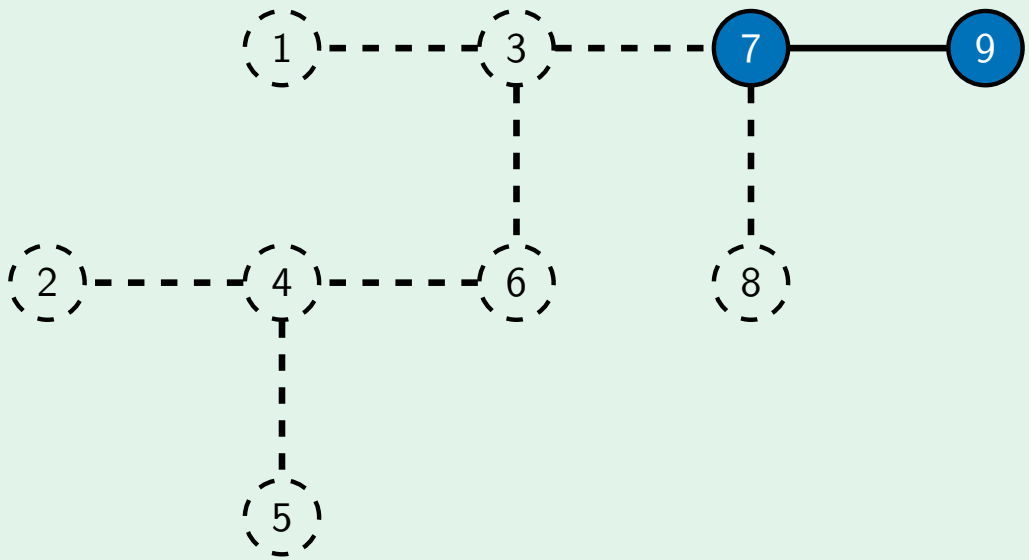
Código de Prüfer: $3, 4, 4, 6, 3, 7, 7$



Clave: *Anotar el vecino de la hoja con menor etiqueta hasta que queden solo dos vértices*

Ejemplo (Iteración 8: Código resultante)

Código de Prüfer: $3, 4, 4, 6, 3, 7, 7$



Final: *El código está completo, solo quedan dos vértices*

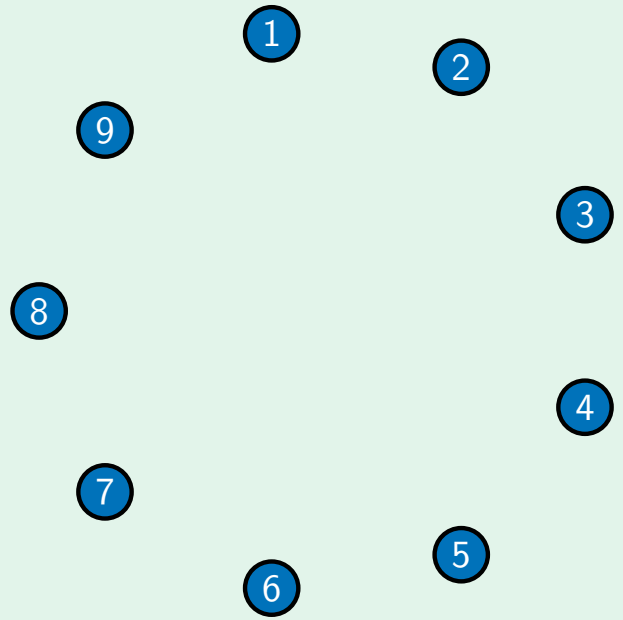
Ejemplo (Paso 0: Código inicial)

Reconstrucción del grafo

En cada paso:

- Coloreamos el **vértice** de C usado.
- Dibujamos la **arista** que une ese vértice con el menor **elemento** de L que no está en C.
- Eliminamos los ~~términos~~ de C y L que han sido usados.

Código de Prüfer:
$C = (3, 4, 4, 6, 3, 7, 7)$

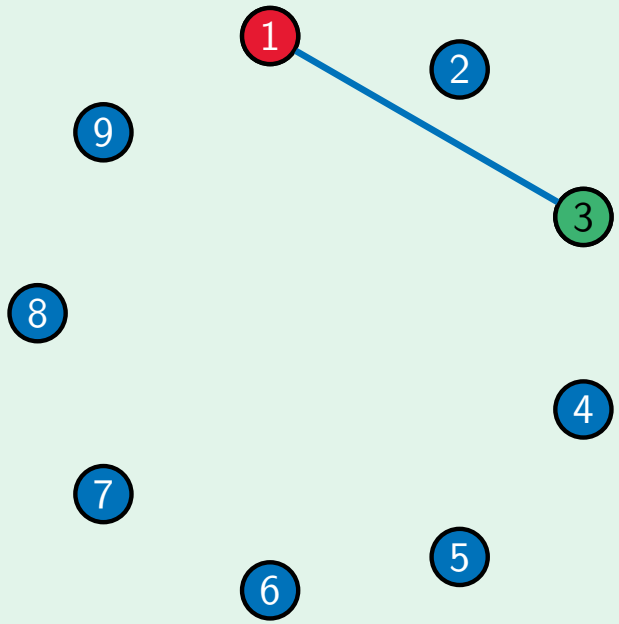


$L = \{1, 2, \ldots, 9\}$

Ejemplo (Iteración 1)

$$C = (3, 4, 4, 6, 3, 7, 7)$$

$$L = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

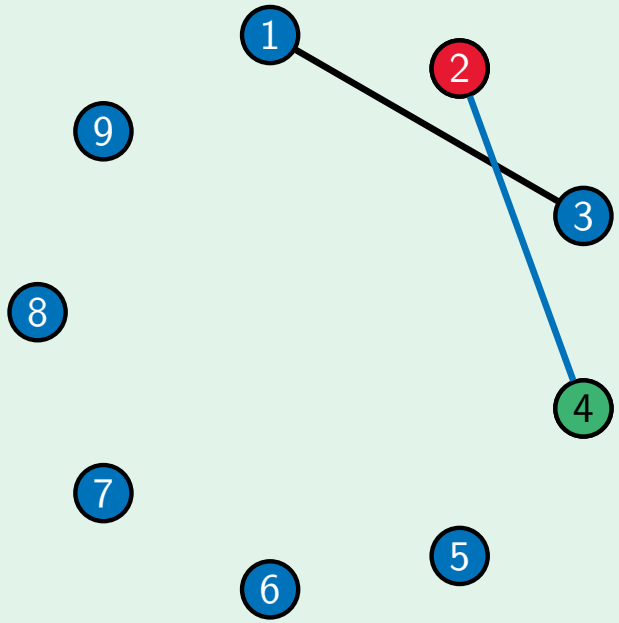


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 2)

$$C = (\cancel{3}, 4, 4, 6, 3, 7, 7)$$

$$L = \{\cancel{1}, 2, 3, 4, 5, 6, 7, 8, 9\}$$

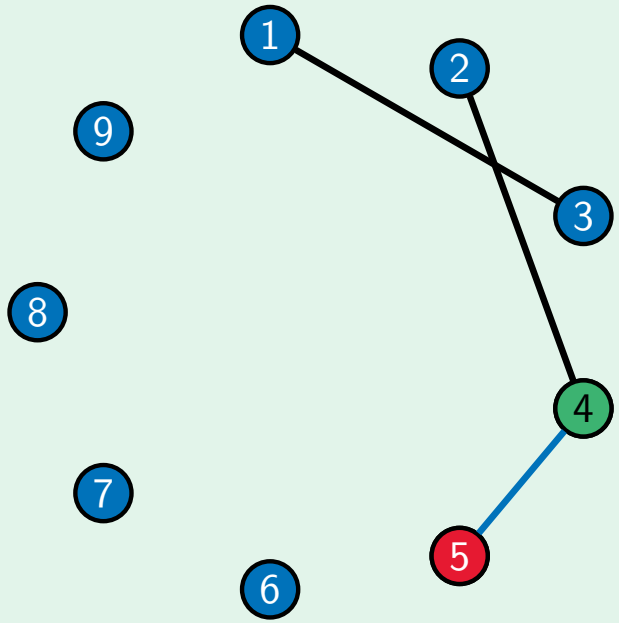


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 3)

$$C = (\cancel{3}, \cancel{4}, 4, 6, 3, 7, 7)$$

$$L = \{\cancel{1}, \cancel{2}, 3, 4, 5, 6, 7, 8, 9\}$$

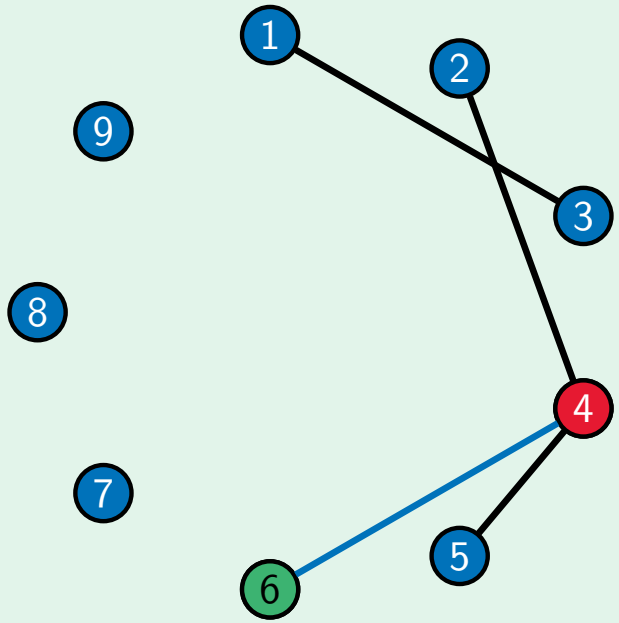


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 4)

$$C = (\cancel{3}, \cancel{4}, \cancel{4}, 6, 3, 7, 7)$$

$$L = \{\cancel{1}, \cancel{2}, 3, 4, \cancel{5}, 6, 7, 8, 9\}$$

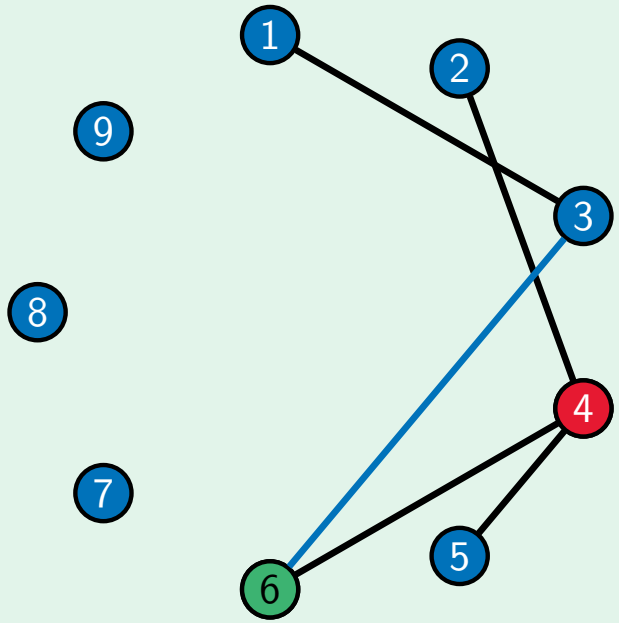


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 5)

$$C = (\cancel{3}, \cancel{4}, \cancel{4}, \cancel{6}, 3, 7, 7)$$

$$L = \{\cancel{1}, \cancel{2}, 3, \cancel{4}, \cancel{5}, 6, 7, 8, 9\}$$

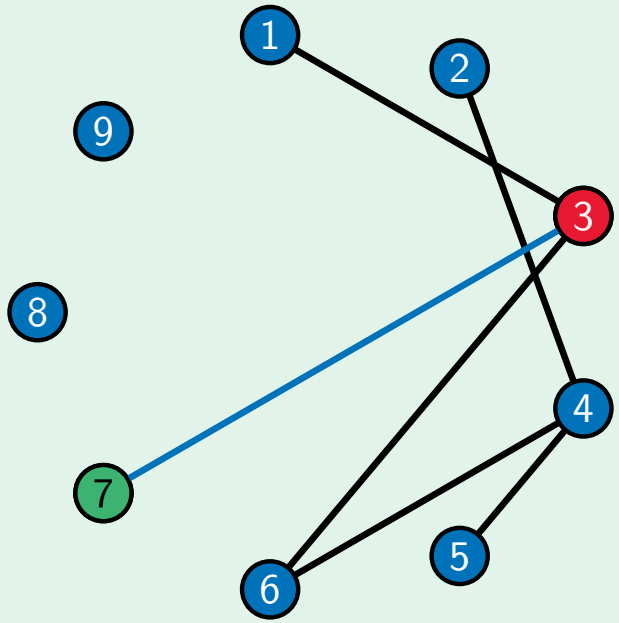


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 6)

$$C = (\cancel{3}, \cancel{4}, \cancel{4}, \cancel{6}, \cancel{3}, 7, 7)$$

$$L = \{\cancel{1}, \cancel{2}, 3, \cancel{4}, \cancel{5}, \cancel{6}, 7, 8, 9\}$$

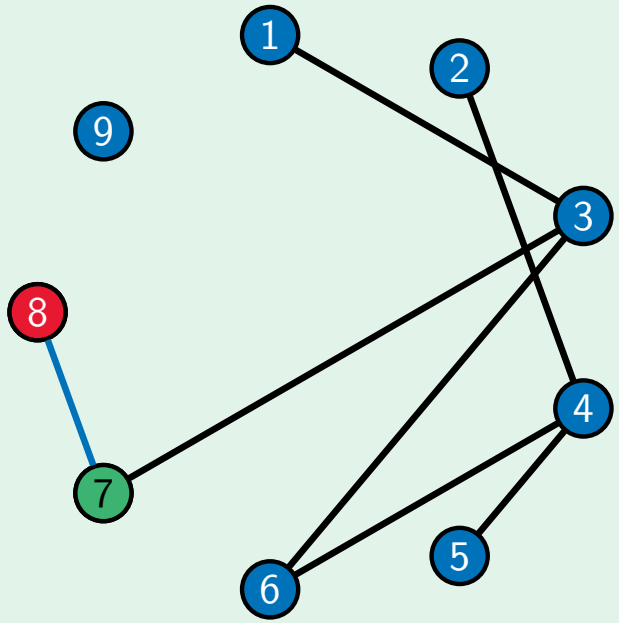


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 7)

$$C = (\cancel{3}, \cancel{4}, \cancel{4}, \cancel{6}, \cancel{3}, \cancel{7}, 7)$$

$$L = \{\cancel{1}, \cancel{2}, \cancel{3}, \cancel{4}, \cancel{5}, \cancel{6}, 7, 8, 9\}$$

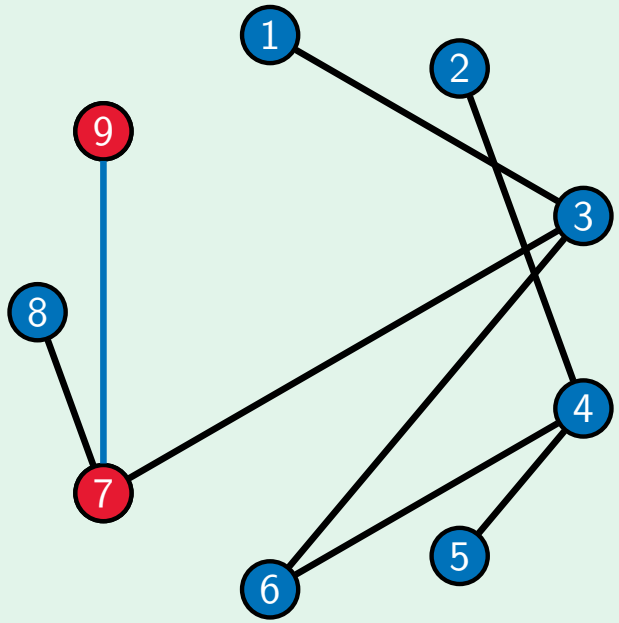


Clave: *Unir el primer vértice en C*
con el menor etiquetado de L que no está en C

Ejemplo (Iteración 8)

$$C = (\cancel{3}, \cancel{4}, \cancel{4}, \cancel{6}, \cancel{3}, \cancel{7}, \cancel{7})$$

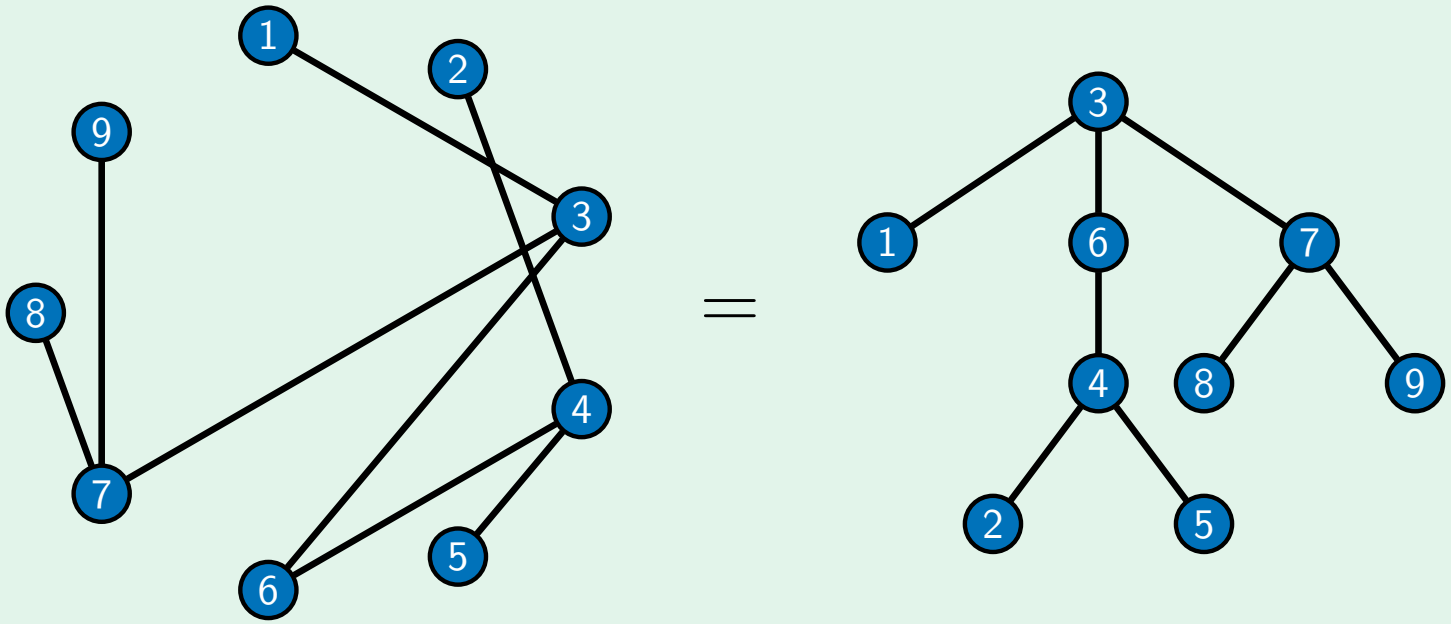
$$L = \{\cancel{1}, \cancel{2}, \cancel{3}, \cancel{4}, \cancel{5}, \cancel{6}, 7, \cancel{8}, 9\}$$



Clave: *Unir los dos vértices restantes de L*

Ejemplo (Final: Grafo resultante)

Código de Prüfer: $C = (3, 4, 4, 6, 3, 7, 7)$



Final: *No quedan elementos restantes en C ni en L*

Problema I

¿Cuántas clases de isomorfismo de árboles etiquetados de n vértices existen?

Proposición. Fórmula de Cayley

Sea n un entero positivo. El número de clases de isomorfismo de árboles etiquetados de n vértices es n^{n-2}.

Idea de la demostración.

Cada clase de isomorfismo se corresponde de manera única con un código de Prüfer. Si tenemos n vértices, hay n^{n-2} códigos posibles. □

Problema II

¿Cuántas clases de isomorfismo de grafos simples etiquetados de n vértices existen?

Proposición

Sea n un entero positivo. El número de clases de isomorfismo de grafos simples etiquetados de n vértices es $2^{\binom{n}{2}}$.

Demostración.

El número de aristas posibles es $\binom{n}{2}$, una por cada par de vértices. El conjunto de todas las aristas puede identificarse con

$$E = \{\{1,2\},\{1,3\},\ldots,\{1,n\},\{2,3\},\{2,4\},\ldots,\{2,n\},\ldots,\{n-1,n\}\}$$

Cada uno de los subconjuntos del conjunto de aristas da lugar a uno de estos grafos. Por tanto, el número de grafos simples es el cardinal del conjunto potencia de E, es decir, $2^{\binom{n}{2}}$.

Problema II

¿Cuántas clases de isomorfismo de grafos simples no etiquetados de n vértices existen?

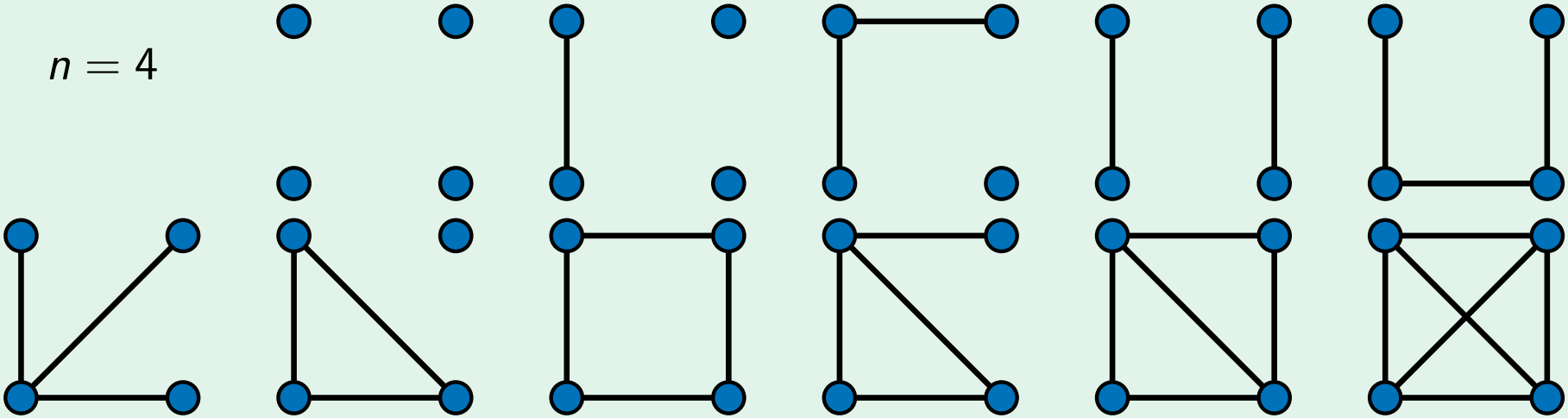
Aproximación

No existe una respuesta o fórmula sencilla.

Ejemplo

Hay 11 clases de isomorfismo de grafos simples no etiquetados de 4 vértices.

$n = 4$



Árboles generadores

Árboles generadores

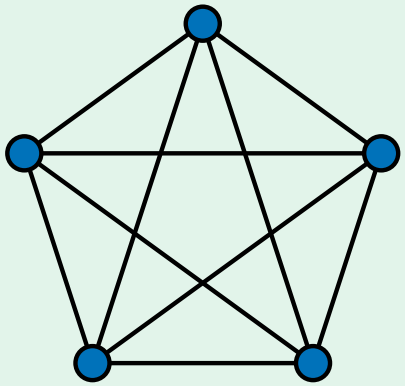
Definición y ejemplo

Definición

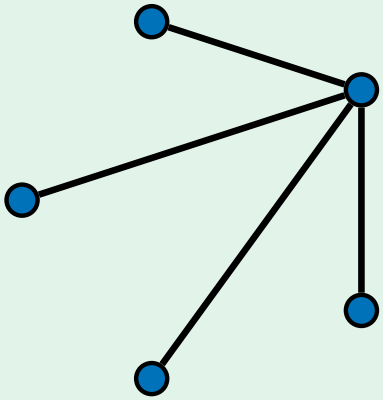
Sea G un grafo. Un **árbol generador** de G es un subgrafo generador de G que es un árbol.

Ejemplo (Árboles generadores)

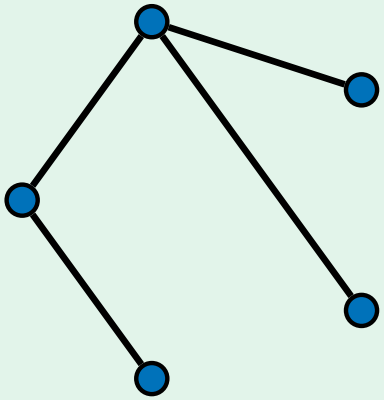
Grafo G



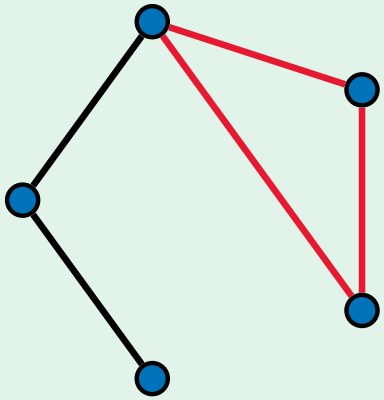
Árbol generador



Árbol generador



Generador, no árbol



Observación

Algunos árboles generadores se pueden construir utilizando algoritmos de búsqueda en profundidad y búsqueda en anchura.

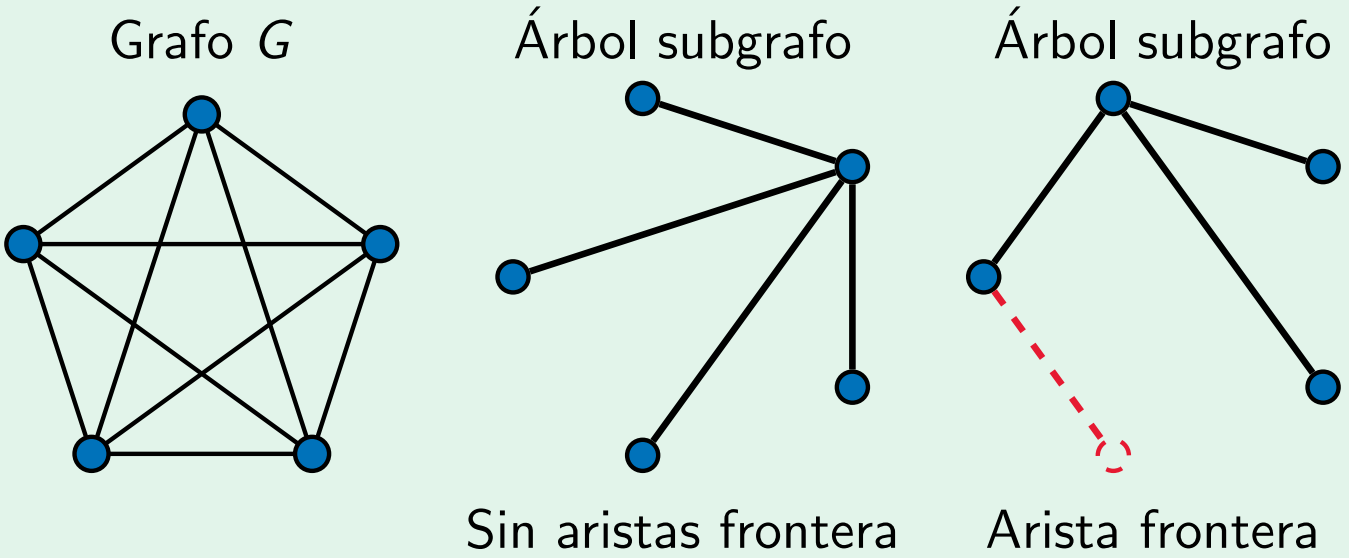
Aristas frontera

Definición y ejemplos

Definición

Sea T un árbol subgrafo de un grafo $G = (V, E)$. Decimos que $e \in E$ es una **arista frontera** de T si sólo uno de sus extremos está en T.

Ejemplo (Aristas frontera)



Conexión y árboles generadores

Proposición

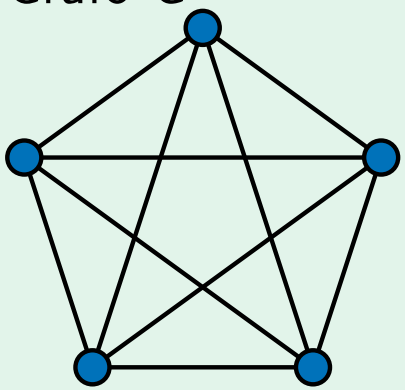
Sea T un árbol subgrafo de un grafo G. Entonces T es un árbol generador de una componente conexa de G si y solo si no tiene aristas frontera.

Teorema

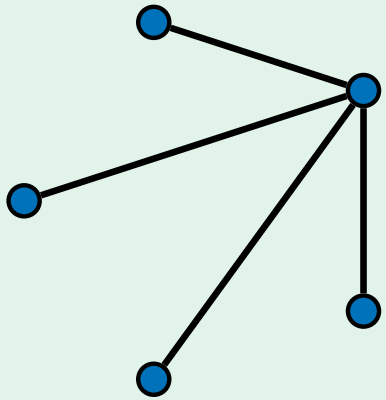
Un grafo es conexo si y solo si posee un árbol generador.

Ejemplo

Grafo G



Árbol subgrafo T



- T es un árbol generador de G.
- T no tiene aristas frontera.
- G es un grafo conexo.

Búsquedas en grafos

- Búsqueda en anchura.
- Búsqueda en profundidad.
- Aplicaciones y comparación de los algoritmos de búsqueda.

Motivación

Encontrar árboles subgrafos con ciertas propiedades dentro de un grafo.

Estructura común a algunos algoritmos

Input: Grafo G y vértice inicial v_0.

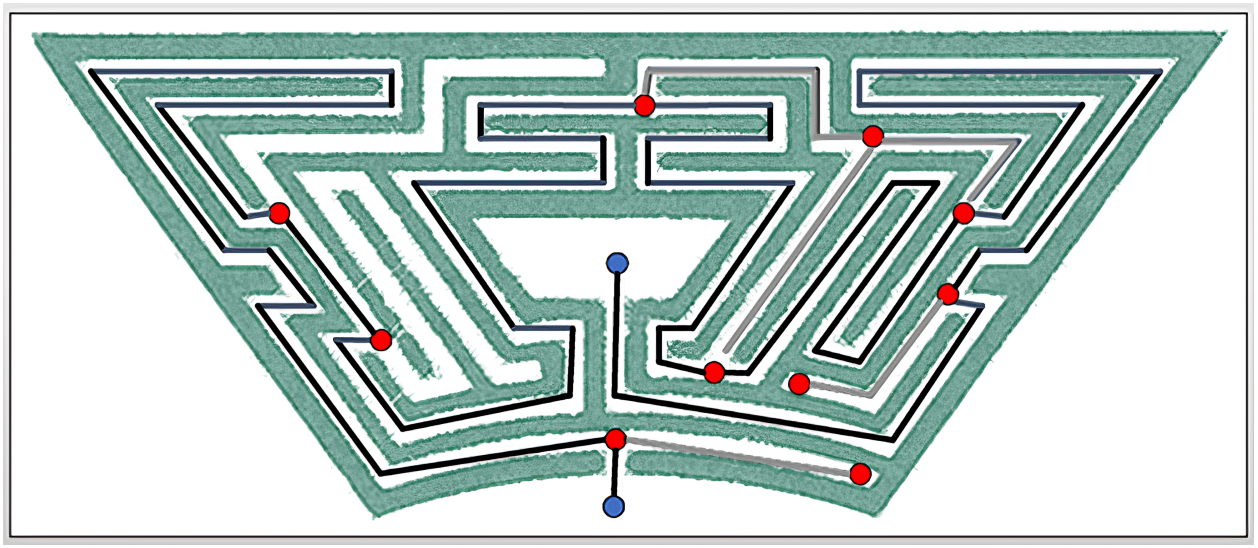
1. Iniciamos $T = (\{v_0\}, \emptyset)$
2. Mientras haya aristas frontera:
   1. Escogemos una arista frontera e de T.
      Según el criterio de elección de la arista, cambia el algoritmo.
   2. Añadimos a T la arista e y su extremo w que no está en T.
3. Devuelve el árbol generador.

Output: Árbol generador T, con raíz v_0, de la componente conexa $C_G(v_0)$ de G que contiene a v_0.

Origen

Estudio de los laberintos:

- Búsqueda en profundidad (final s. XIX)
- Búsqueda en anchura (1959).



Laberinto de Hampton-Court

Búsqueda en anchura

Estructura del algoritmo

Input: Grafo G y vértice inicial v_0.

1. Iniciamos $T = (\{v_0\}, \emptyset)$, $df(v_0) = 0$, $i = 0$;
2. Mientras haya aristas frontera:
   1. Escogemos una arista frontera e de T cuyo extremo tenga menor etiquetado df.
   2. Añadimos a T la arista e y su extremo w que no está en T.
   3. $df(w) = i + 1$, $i = i + 1$.
3. Devuelve el árbol generador T con etiquetado df.

Output: Árbol generador T, con raíz v_0 y etiquetado df, de la componente conexa $C_G(v_0)$ de G que contiene a v_0.

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$ /* Inicialización de datos /*
while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P /* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo, hacer:

NodoVisitado $\leftarrow v$; $V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$ /* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$ /* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$ /* Se añade el vértice activo al árbol resultante /*

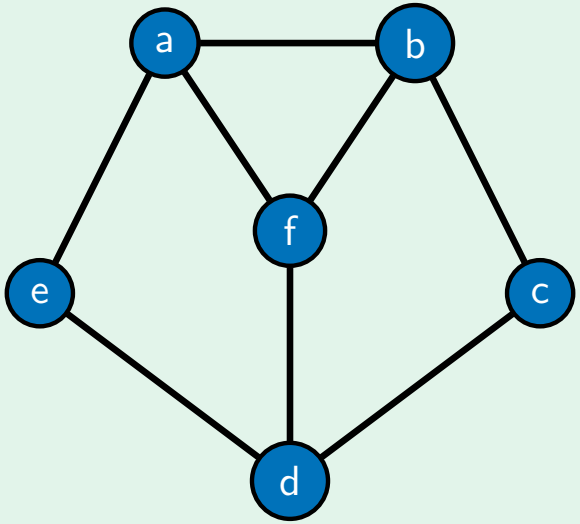
End if

End while
Return (T, A') árbol de búsqueda

Búsqueda en anchura

Ejemplo

Ejemplo (Inicialización)



$P = [a]$

$V' = [b, c, d, e, f]$

$T = []$

$A' = []$

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$

/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

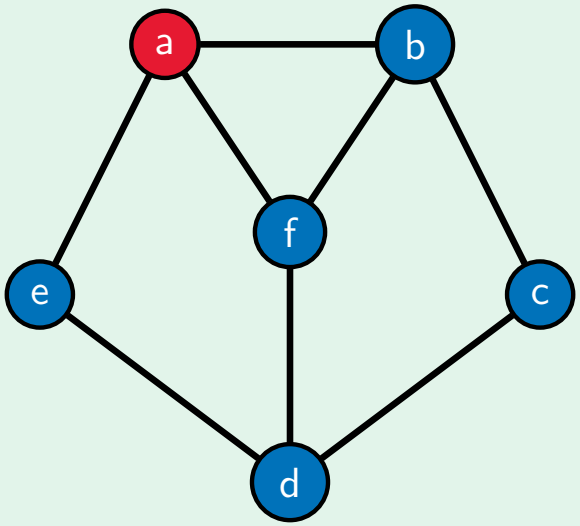
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Comprobación de la pila y selección de nodo activo



$P = [a] \quad V' = [b, c, d, e, f]$

$T = []$

$A' = []$

P no es vacía

Se elige como nodo activo al primer nodo de P: a

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

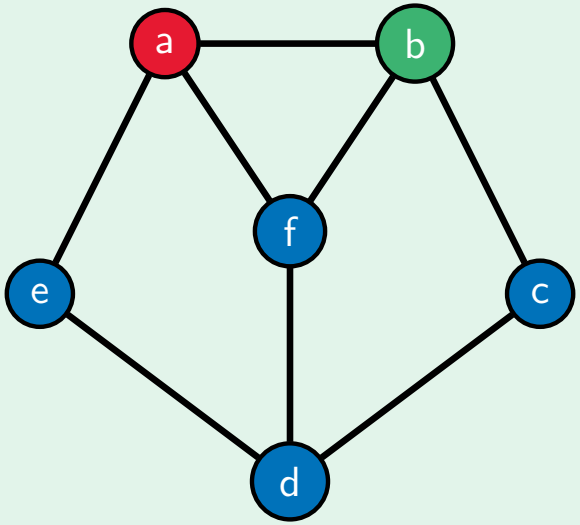
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a] \quad V' = [\not{b}, c, d, e, f]$

$T = []$

$A' = []$

a tiene adyacentes en V'

Se elige b como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

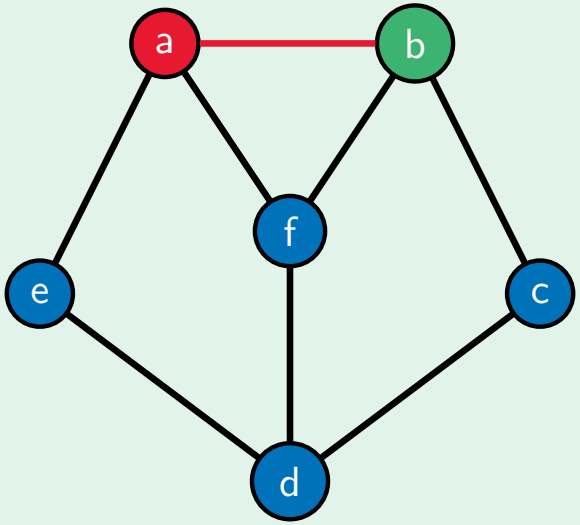
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Añadir arista nodo activo - nodo visitado



$P = [a] \quad V' = [c, d, e, f]$

$T = []$

$A' = [ab]$

Añadimos a A' la arista nodo activo - nodo visitado: ab

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

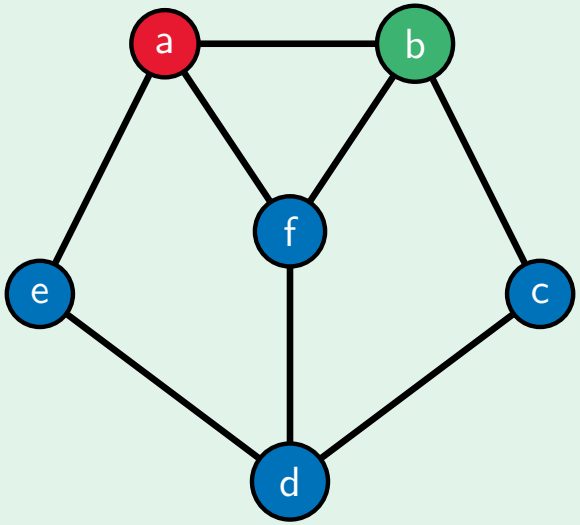
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Añadir nodo visitado al final de la pila



$P = [a, b] \quad V' = [c, d, e, f]$
$T = []$
$A' = [ab]$

Añadimos b al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

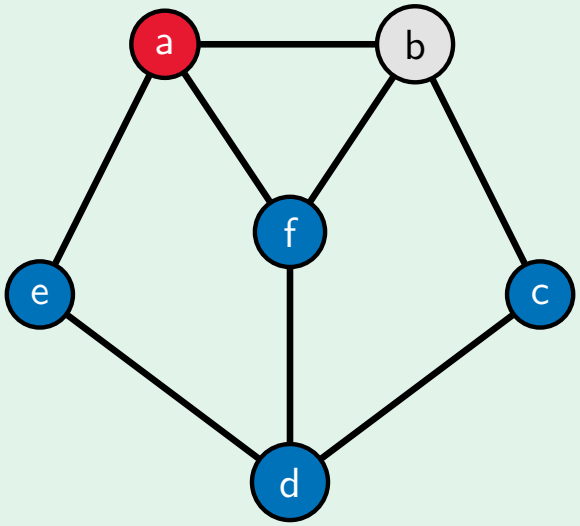
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b] \quad V' = [c, d, e, f]$

$T = []$

$A' = [ab]$

P no es vacía

Se elige como nodo activo al primer nodo de P: a

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

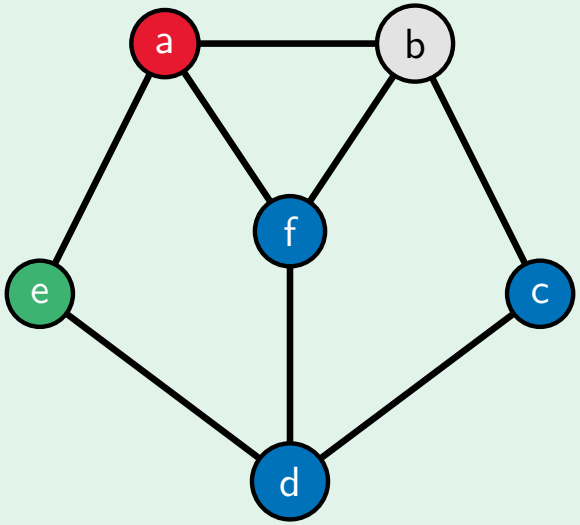
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a, b], \quad V' = [c, d, \not{e}, f]$

$T = []$

$A' = [ab]$

a tiene adyacentes en V'

Se elige e como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

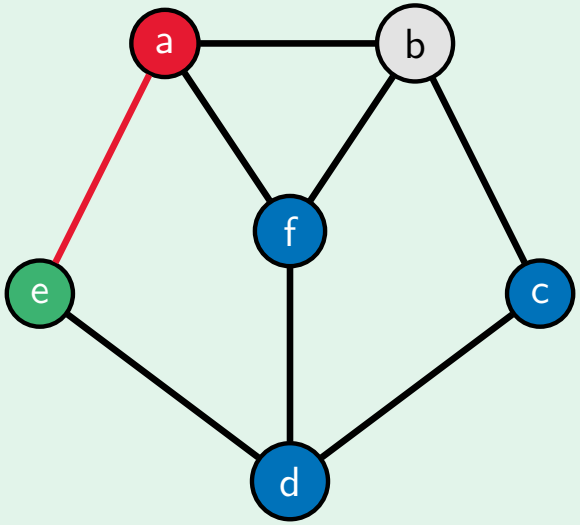
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Añadir arista nodo activo - nodo visitado



$P = [a, b] \quad V' = [c, d, f]$

$T = []$

$A' = [ab, ae]$

Añadimos a A' la arista nodo activo - nodo visitado: ae

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

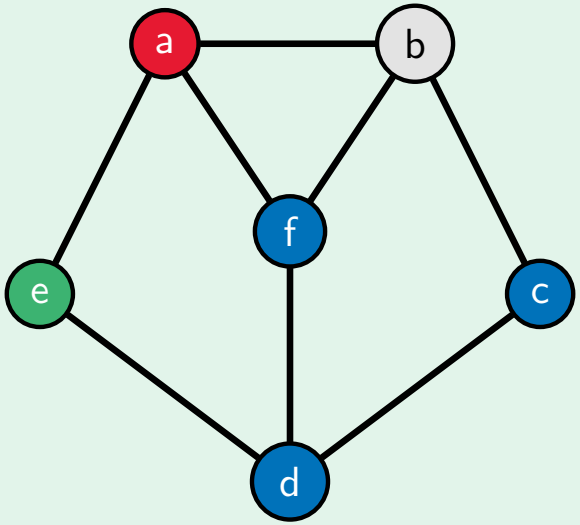
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Añadir nodo visitado al final de la pila



$P = [a, b, e] \quad V' = [c, d, f]$
$T = []$
$A' = [ab, ae]$

Añadimos e al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

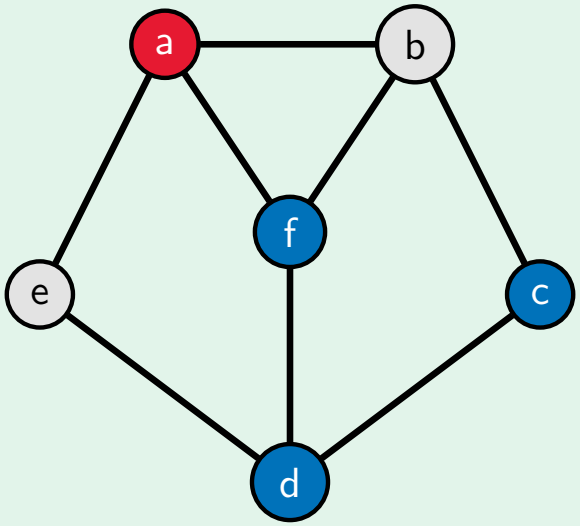
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b, e] \quad V' = [c, d, f]$
$T = []$
$A' = [ab, ae]$

P no es vacía

Se elige como nodo activo al primer nodo de P: a

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

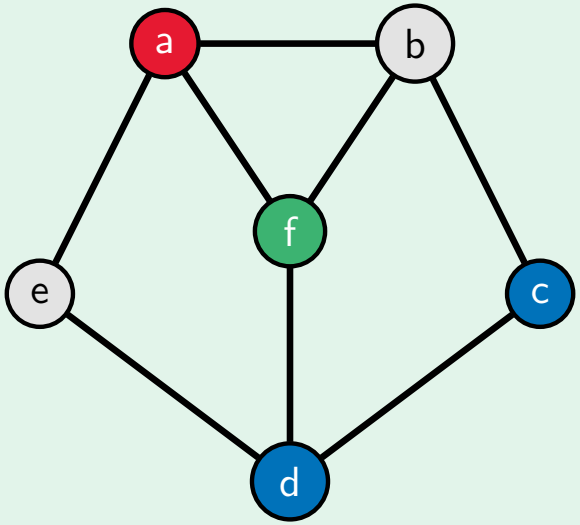
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a, b, e] \quad V' = [c, d, f]$
$T = []$
$A' = [ab, ae]$

a tiene adyacentes en V'

Se elige f como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

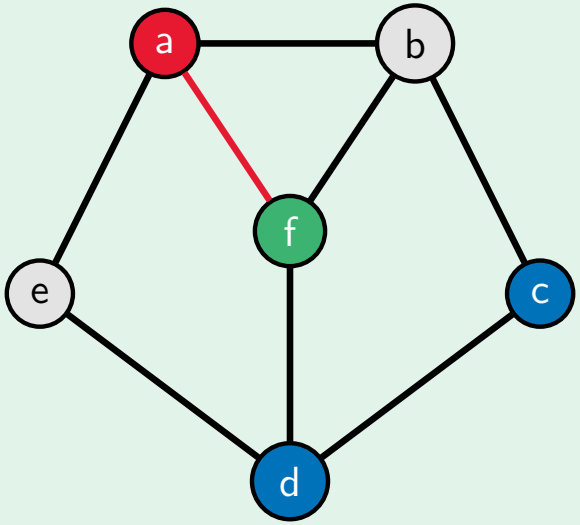
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Añadir arista nodo activo - nodo visitado



$P = [a, b, e] \quad V' = [c, d]$

$T = []$

$A' = [ab, ae, af]$

Añadimos a A' la arista nodo activo - nodo visitado: af

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

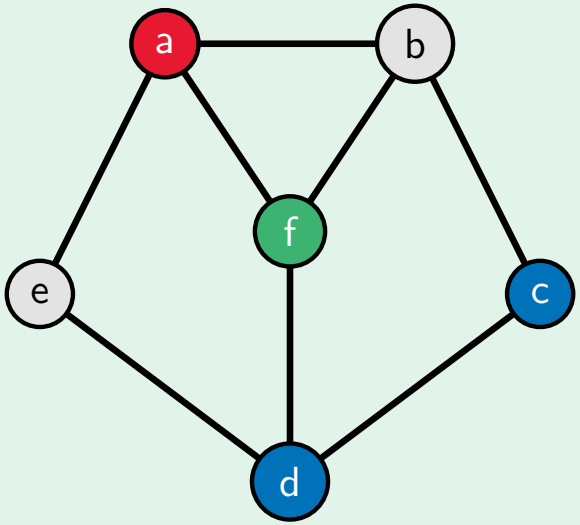
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Añadir nodo visitado al final de la pila



$P = [a, b, e, f] \quad V' = [c, d]$
$T = []$
$A' = [ab, ae, af]$

Añadimos f al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

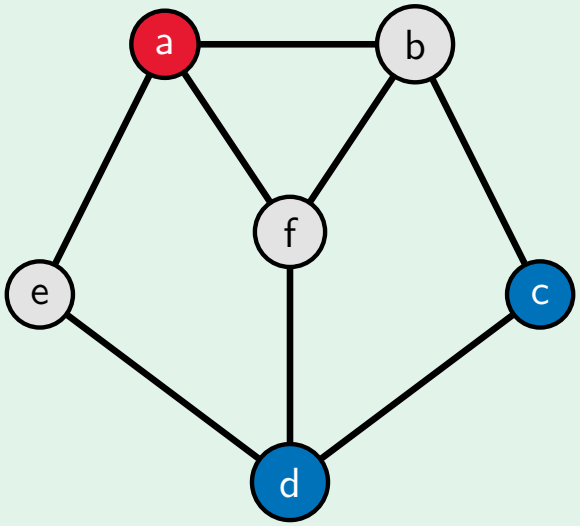
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Cuarta iteración)

Comprobación de la pila y selección del nodo activo



$P = [a, b, e, f] \quad V' = [c, d]$

$T = []$

$A' = [ab, ae, af]$

P no es vacía

Se elige como nodo activo al primer nodo de P: a

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

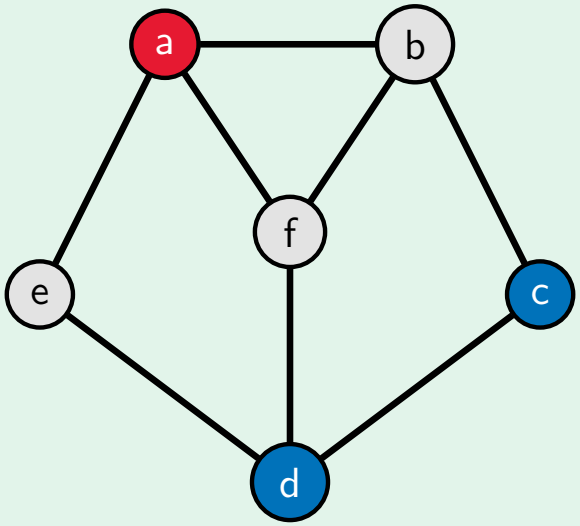
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Cuarta iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [\not{a}, b, e, f] \quad V' = [c, d]$

$T = [a]$

$A' = [ab, ae, af]$

a no tiene adyacentes en V'

Se elimina a de la pila P y se incorpora a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

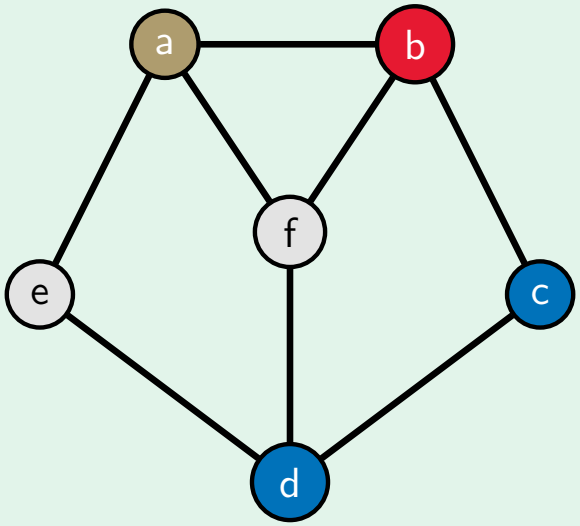
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Quinta iteración)

Comprobación de la pila y selección del nodo activo



$P = [b, e, f] \quad V' = [c, d]$

$T = [a]$

$A' = [ab, ae, af]$

P no es vacía

Se elige como nodo activo al primer nodo de P: b

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

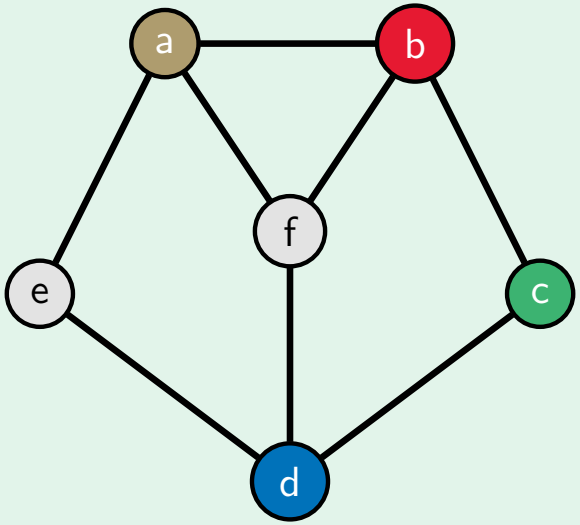
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Quinta iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [b, e, f] \quad V' = [\not{c}, d]$

$T = [a]$

$A' = [ab, ae, af]$

b tiene adyacentes en V'

Se elige c como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

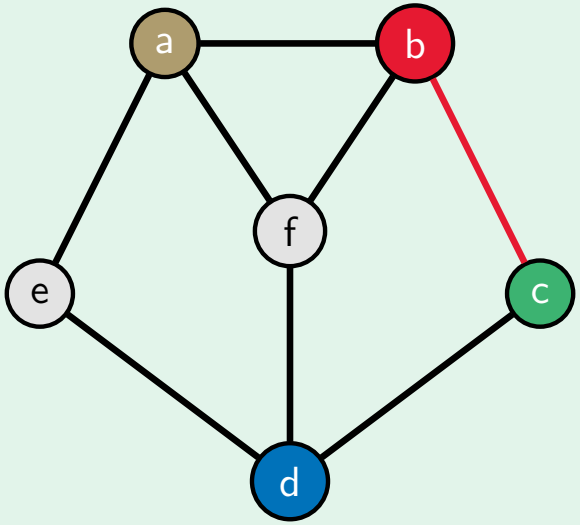
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Quinta iteración)

Añadir arista nodo activo - nodo visitado



$P = [b, e, f] \quad V' = [\not{c}, d]$

$T = [a]$

$A' = [ab, ae, af, bc]$

Añadimos a A' la arista nodo activo - nodo visitado: bc

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

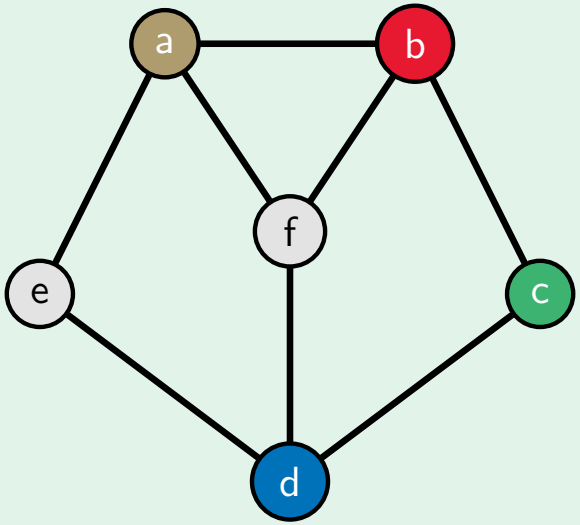
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Quinta iteración)

Añadir nodo visitado al final de la pila



$P = [b, e, f, c] \quad V' = [d]$
$T = [a]$
$A' = [ab, ae, af, bc]$

Añadimos c al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

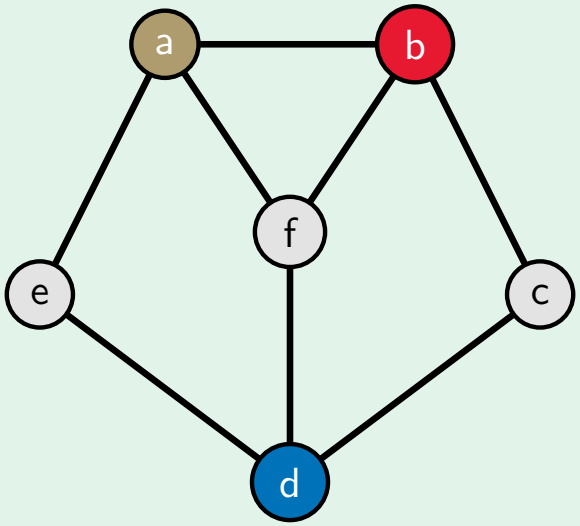
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Sexta iteración)

Comprobación de la pila y selección del nodo activo



$P = [b, e, f, c] \quad V' = [d]$

$T = [a]$

$A' = [ab, ae, af, bc]$

P no es vacía

Se elige como nodo activo al primer nodo de P: b

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

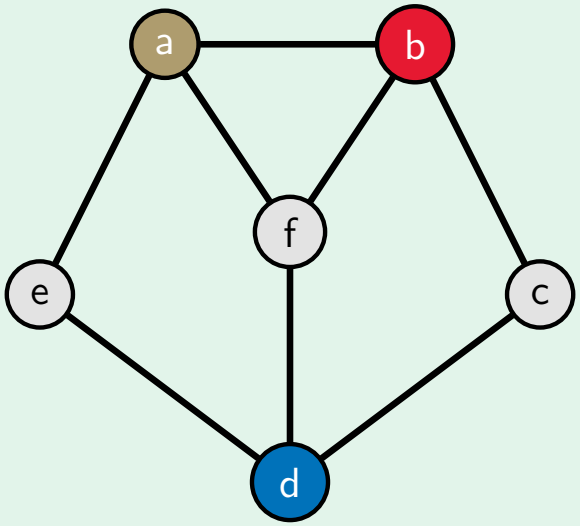
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Sexta iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [\not{b}, e, f, c] \quad V' = [d]$

$T = [a, b]$

$A' = [ab, ae, af, bc]$

b no tiene adyacentes en V'

Se elimina b de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

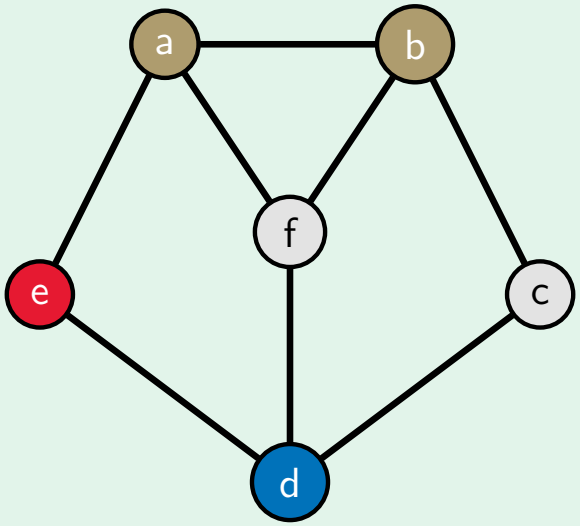
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Séptima iteración)

Comprobación de la pila y selección de nodo activo



$P = [e, f, c] \quad V' = [d]$
$T = [a, b]$
$A' = [ab, ae, af, bc]$

P no es vacía

Se elige como nodo activo al primer nodo de P: e

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

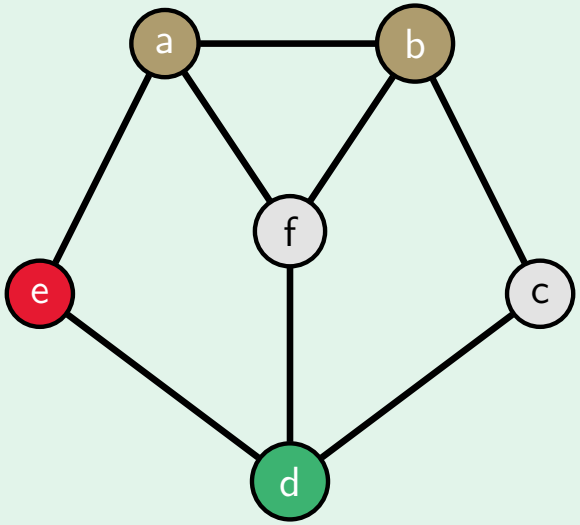
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Séptima iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [e, f, c] \quad V' = [\not{d}]$

$T = [a, b]$

$A' = [ab, ae, af, bc]$

e tiene adyacentes en V'

Se elige d como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

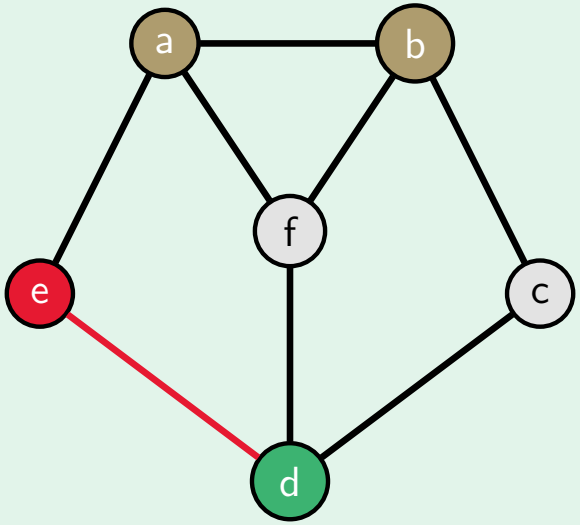
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Séptima iteración)

Añadir arista nodo activo - nodo visitado



$P = [e, f, c] \quad V' = []$

$T = [a, b]$

$A' = [ab, ae, af, bc, ed]$

Añadimos a A' la arista nodo activo - nodo visitado: ed

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

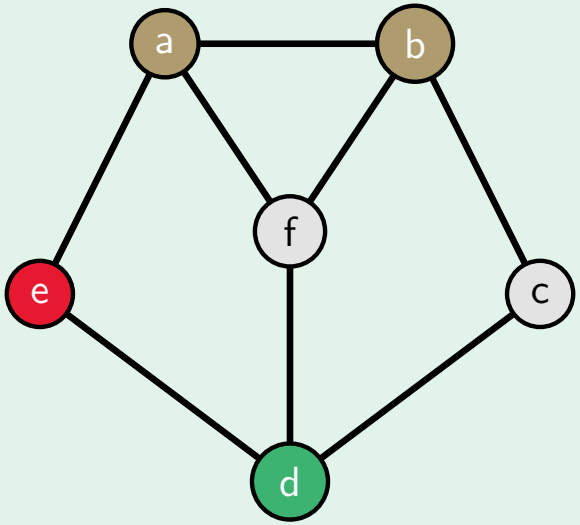
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Séptima iteración)

Añadir nodo visitado al final de la pila



$P = [e, f, c, d] \quad V' = []$

$T = [a, b]$

$A' = [ab, ae, af, bc, ed]$

Añadimos d al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

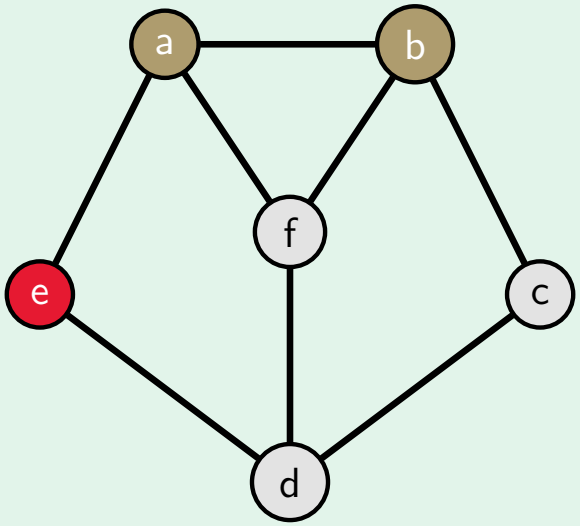
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Octava iteración)

Comprobación de la pila y selección de nodo activo



$P = [e, f, c, d] \quad V' = []$
$T = [a, b]$
$A' = [ab, ae, af, bc, ed]$

P no es vacía

Se elige como nodo activo al primer nodo de P: e

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

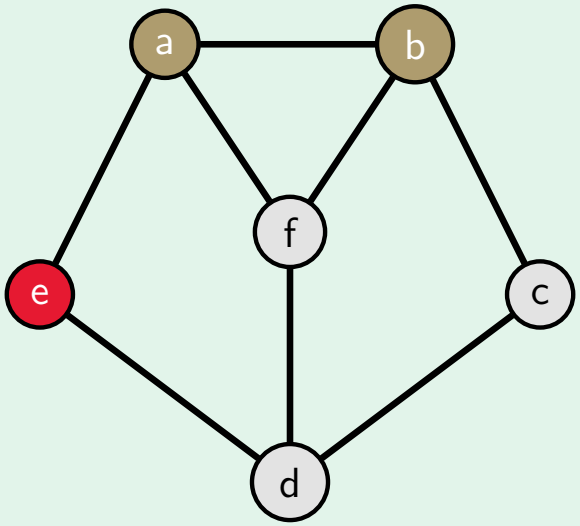
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Octava iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [\not{e}, f, c, d] \quad V' = []$

$T = [a, b, e]$

$A' = [ab, ae, af, bc, ed]$

e no tiene adyacentes en V'

Se elimina e de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

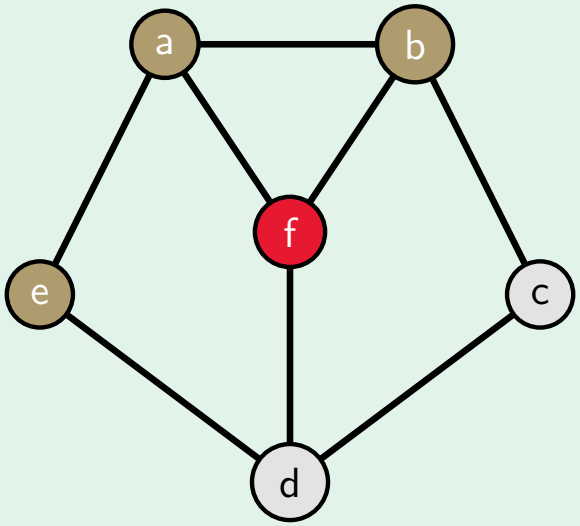
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Novena iteración)

Comprobación de la pila y selección de nodo activo



$$P = [f, c, d] \quad V' = []$$
$$T = [a, b, e]$$
$$A' = [ab, ae, af, bc, ed]$$

P no es vacía

Se elige como nodo activo al primer nodo de P: f

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

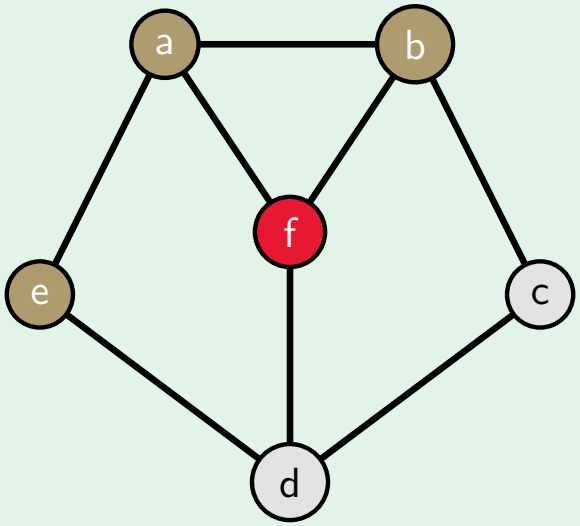
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Novena iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [f, c, d] \quad V' = []$

$T = [a, b, e, f]$

$A' = [ab, ae, af, bc, ed]$

f no tiene adyacentes en V'

Se elimina f de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

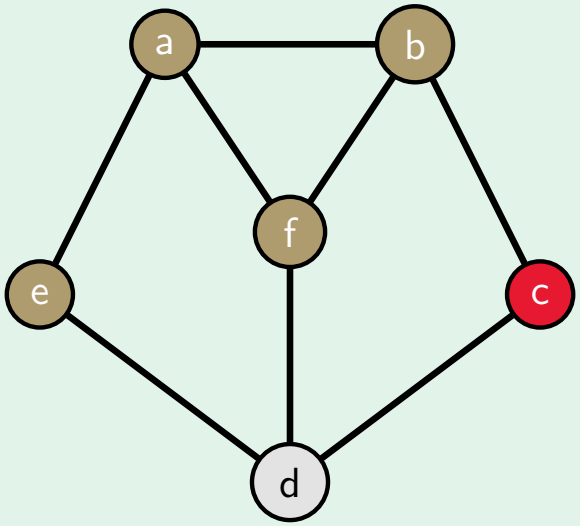
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Décima iteración)

Comprobación de la pila y selección de nodo activo



$$P = [c, d] \quad V' = []$$
$$T = [a, b, e, f]$$
$$A' = [ab, ae, af, bc, ed]$$

P no es vacía

Se elige como nodo activo al primer nodo de P: c

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

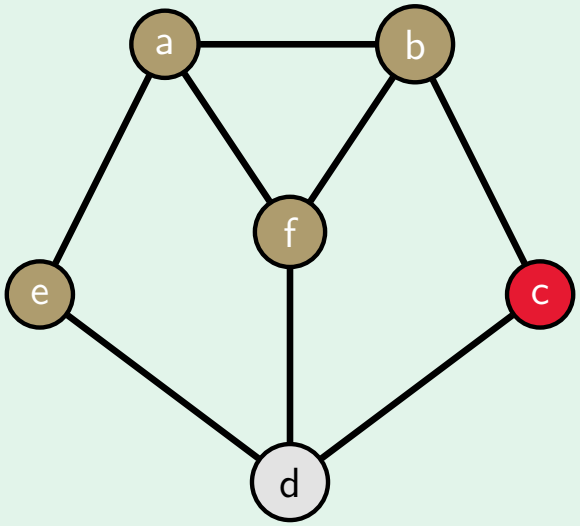
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Décima iteración)

Comprobación de adyacentes y eliminación de la pila



$$P = [\not{c}, d] \quad V' = []$$
$$T = [a, b, c, e, f]$$
$$A' = [ab, ae, af, bc, ed]$$

c no tiene adyacentes en V'

Se elimina c de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

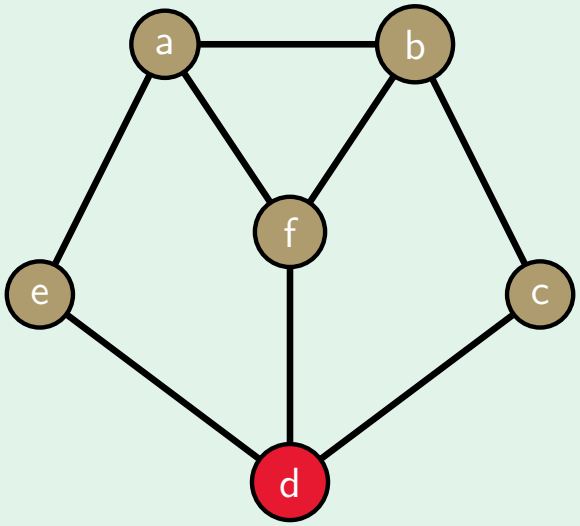
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Undécima iteración)

Comprobación de la pila y selección de nodo activo



$$P = [d] \quad V' = []$$
$$T = [a, b, c, e, f]$$
$$A' = [ab, ae, af, bc, ed]$$

P no es vacía

Se elige como nodo activo al primer nodo de P: d

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P

/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

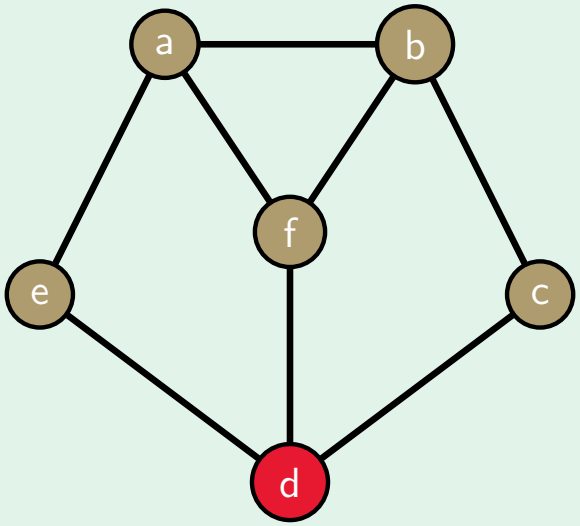
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Undécima iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [\not{d}] \quad V' = []$

$T = [a, b, c, d, e, f]$

$A' = [ab, ae, af, bc, ed]$

d no tiene adyacentes en V'

Se elimina d de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

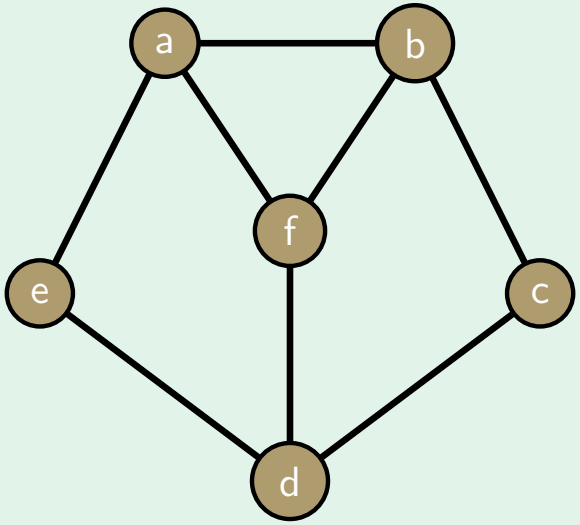
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Decimotercera iteración)

Comprobación de la pila y finalización del algoritmo



$$P = [] \quad V' = []$$
$$T = [a, b, c, d, e, f]$$
$$A' = [ab, ae, af, bc, ed]$$

P es vacía

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

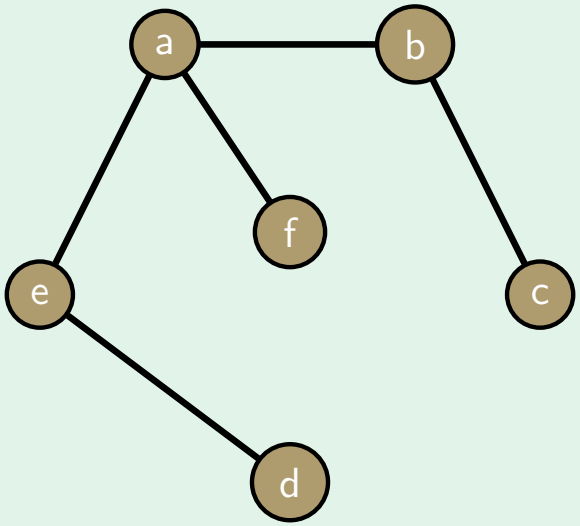
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Resultado)

Árbol resultante



$P = []$ $V' = []$

$T = [a, b, c, d, e, f]$

$A' = [ab, ae, af, bc, ed]$

(T, A') es el árbol resultante

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **primer** nodo de P
/* Activación en cola (FIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

End if

End while

Return (T, A') árbol de búsqueda

Búsqueda en profundidad

Estructura del algoritmo

Input: Grafo G y vértice inicial v_0.

1. Iniciamos $T = (\{v_0\}, \emptyset)$, $df(v_0) = 0$, $i = 0$;
2. Mientras haya aristas frontera:
   1. Escogemos una arista frontera e de T cuyo extremo tenga mayor etiquetado df.
   2. Añadimos a T la arista e y su extremo w que no está en T.
   3. $df(w) = i + 1$, $i = i + 1$.
3. Devuelve el árbol generador T con etiquetado df.

Output: Árbol generador T, con raíz v_0 y etiquetado df, de la componente conexa $C_G(v_0)$ de G que contiene a v_0.

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$ /* Inicialización de datos /*
while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P /* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo, hacer:

NodoVisitado $\leftarrow v$; $V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$ /* Se añade v al final de la pila P /*

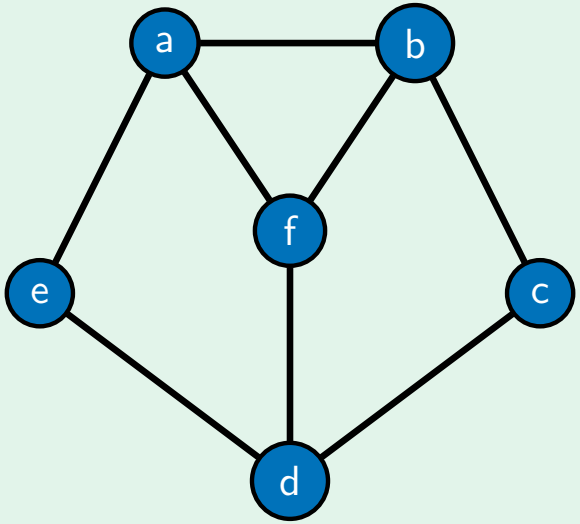
Else

$P \leftarrow P - [\text{NodoActivo}]$ /* Se elimina el vértice activo de la pila /*
$T \leftarrow T \cup [\text{NodoActivo}]$ /* Se añade el vértice activo al árbol resultante /*

End if

End while
Return (T, A') árbol de búsqueda

Ejemplo (Inicialización)



$P = [a]$

$V' = [b, c, d, e, f]$

$T = []$

$A' = []$

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$

/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

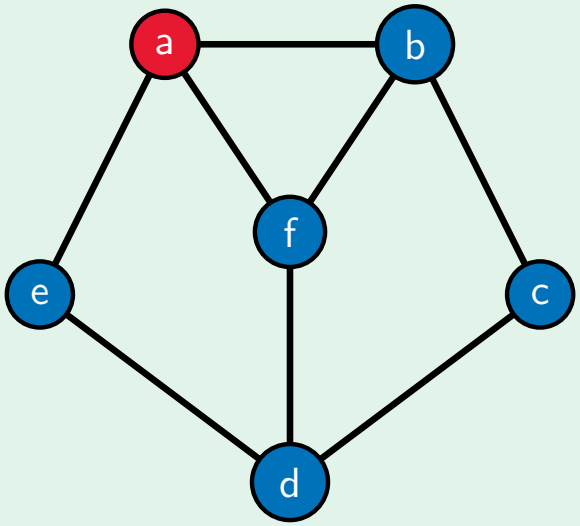
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Comprobación de la pila y selección de nodo activo



$P = [a] \quad V' = [b, c, d, e, f]$
$T = []$
$A' = []$

P no es vacía

Se elige como nodo activo al último nodo de P: a

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

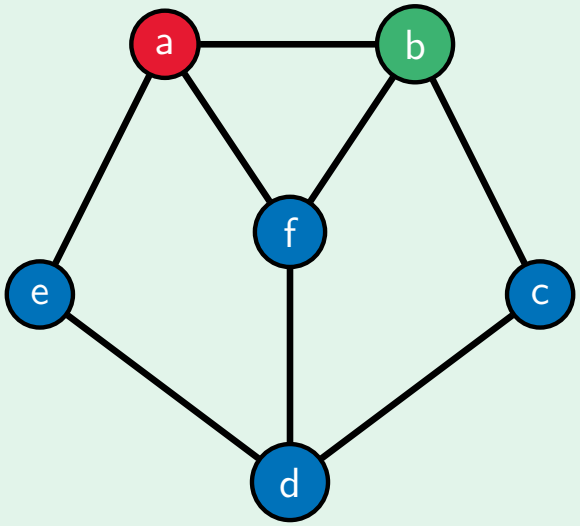
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a] \quad V' = [\not{b}, c, d, e, f]$
$T = []$
$A' = []$

a tiene adyacentes en V'

Se elige b como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

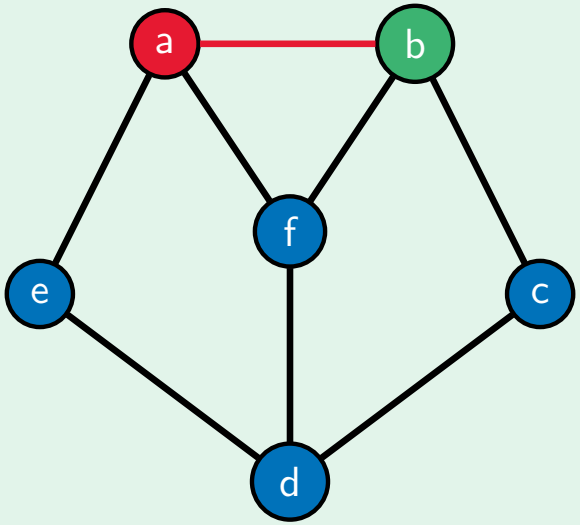
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Añadir arista nodo activo - nodo visitado



$P = [a] \quad V' = [c, d, e, f]$

$T = []$

$A' = [ab]$

Añadimos a A' la arista nodo activo - nodo visitado: ab

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

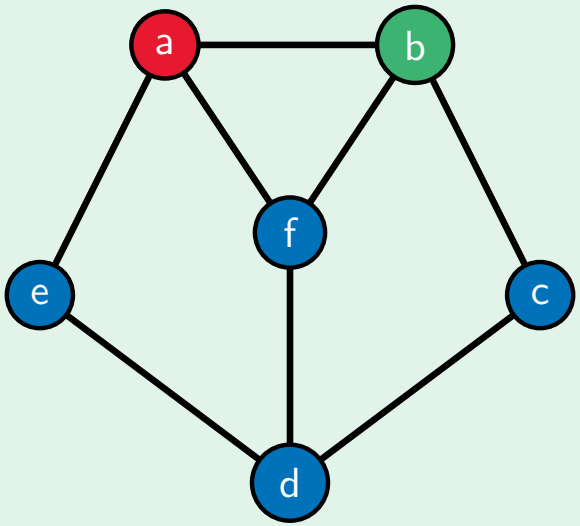
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Primera iteración)

Añadir nodo visitado al final de la pila



$P = [a, b] \quad V' = [c, d, e, f]$

$T = []$

$A' = [ab]$

Añadimos b al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

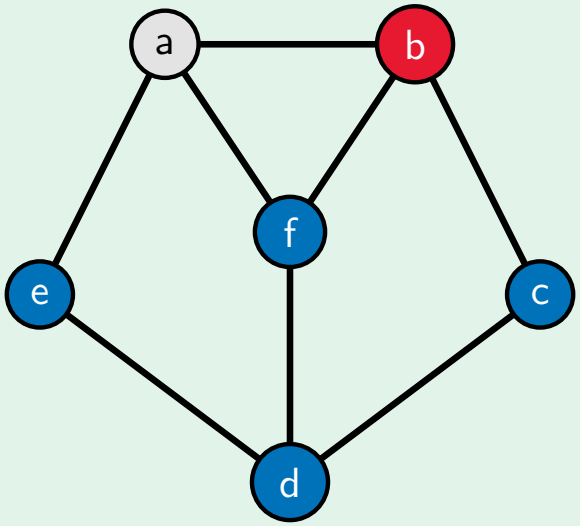
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b] \quad V' = [c, d, e, f]$
$T = []$
$A' = [ab]$

P no es vacía

Se elige como nodo activo al último nodo de P: b

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

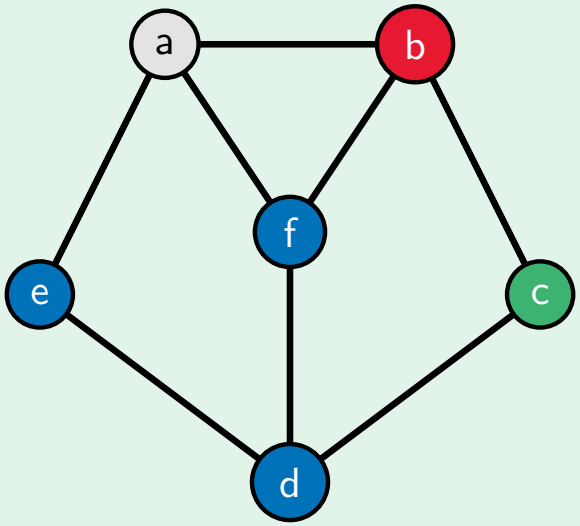
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a, b] \quad V' = [\not{c}, d, e, f]$
$T = []$
$A' = [ab]$

b tiene adyacentes en V'

Se elige c como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

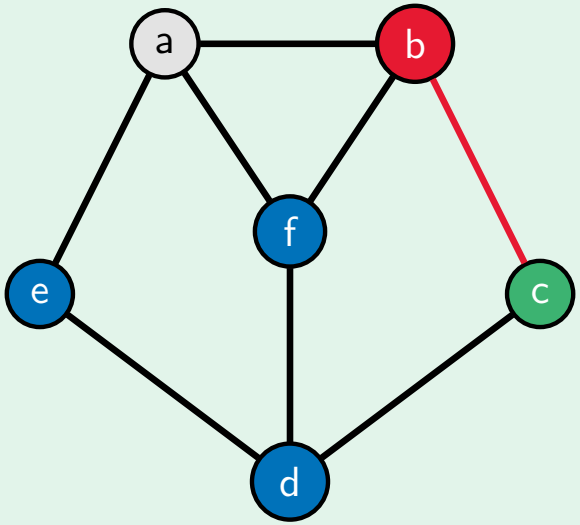
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Añadir arista nodo activo – nodo visitado



$P = [a, b] \quad V' = [d, e, f]$

$T = []$

$A' = [ab, bc]$

Añadimos a A' la arista nodo activo - nodo visitado: bc

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

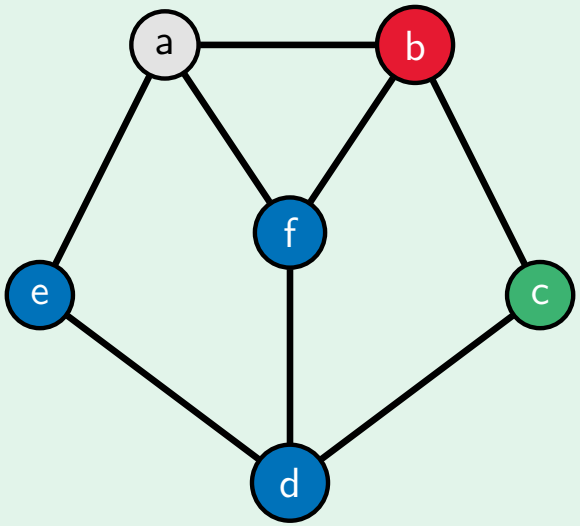
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Segunda iteración)

Añadir nodo visitado al final de la pila



$P = [a, b, c] \quad V' = [d, e, f]$
$T = []$
$A' = [ab, bc]$

Añadimos c al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

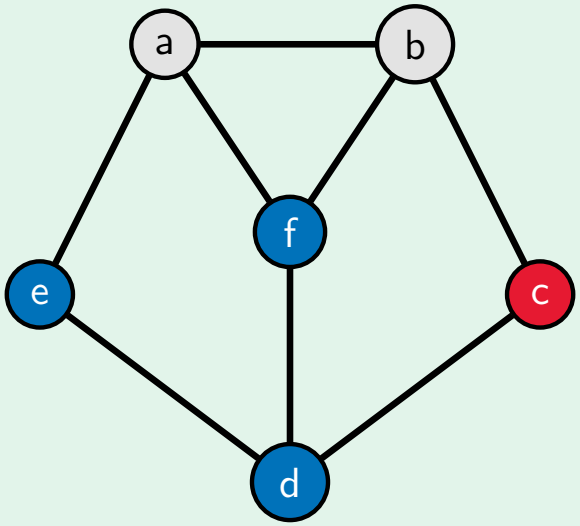
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b, c] \quad V' = [d, e, f]$
$T = []$
$A' = [ab, bc]$

P no es vacía

Se elige como nodo activo al último nodo de P: c

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

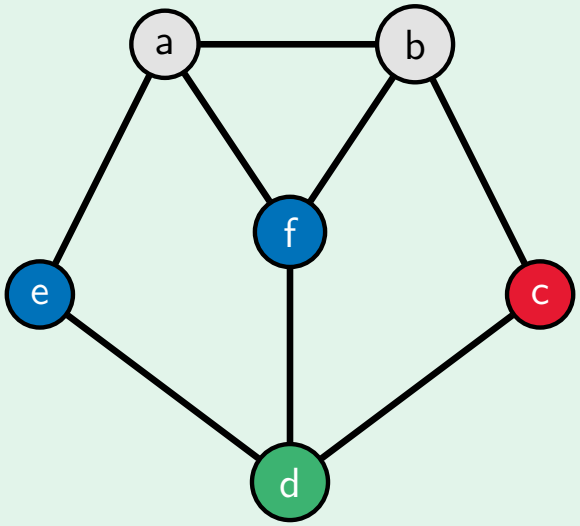
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a, b, c] \quad V' = [\not{d}, e, f]$
$T = []$
$A' = [ab, bc]$

c tiene adyacentes en V'

Se elige d como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

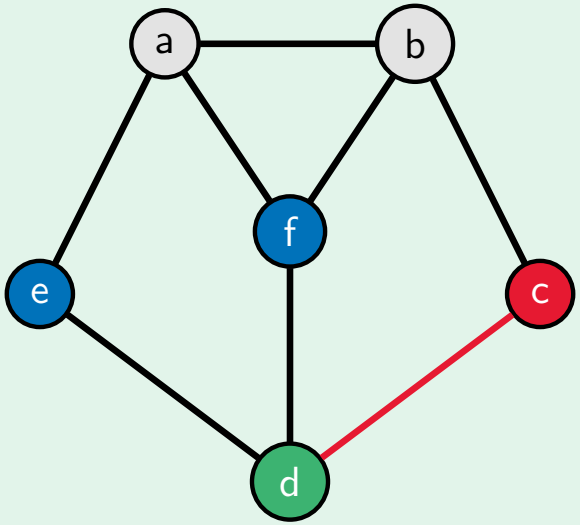
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Añadir arista nodo activo – nodo visitado



$P = [a, b, c] \quad V' = [e, f]$
$T = []$
$A' = [ab, bc, cd]$

Añadimos a A' la arista nodo activo - nodo visitado: cd

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

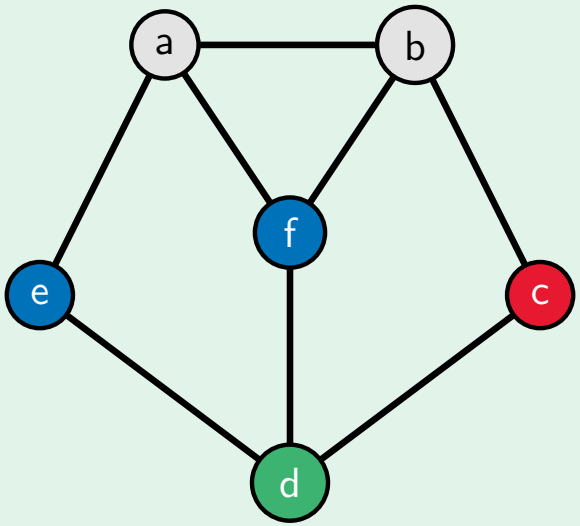
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Tercera iteración)

Añadir nodo visitado al final de la pila



$P = [a, b, c, d] \quad V' = [e, f]$
$T = []$
$A' = [ab, bc, cd]$

Añadimos d al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

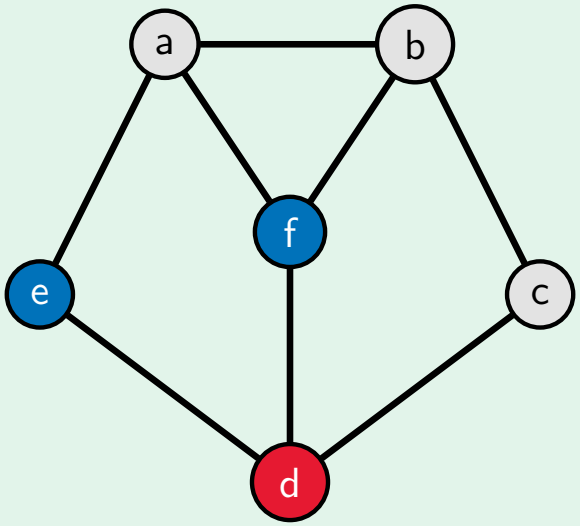
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Cuarta iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b, c, d] \quad V' = [e, f]$

$T = []$

$A' = [ab, bc, cd]$

P no es vacía

Se elige como nodo activo al último nodo de P: d

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

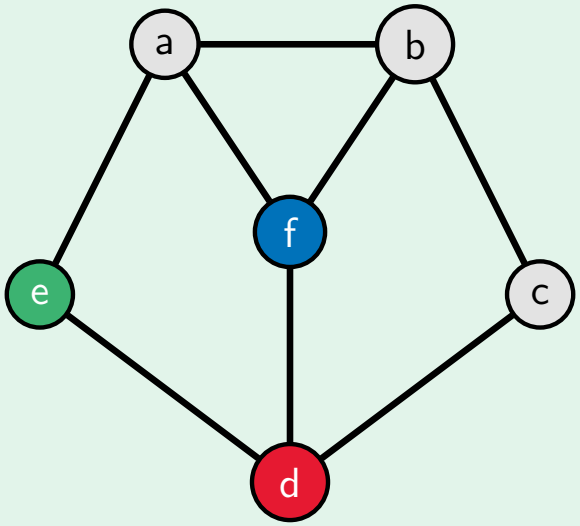
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Cuarta iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a, b, c, d] \quad V' = [\not{e}, f]$
$T = []$
$A' = [ab, bc, cd]$

c tiene adyacentes en V'

Se elige e como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

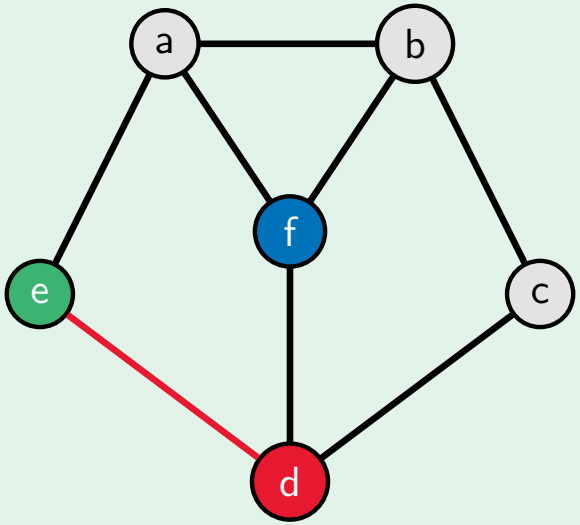
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Cuarta iteración)

Añadir arista nodo activo – nodo visitado



$P = [a, b, c, d] \quad V' = [f]$

$T = []$

$A' = [ab, bc, cd, de]$

Añadimos a A' la arista nodo activo - nodo visitado: cd

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

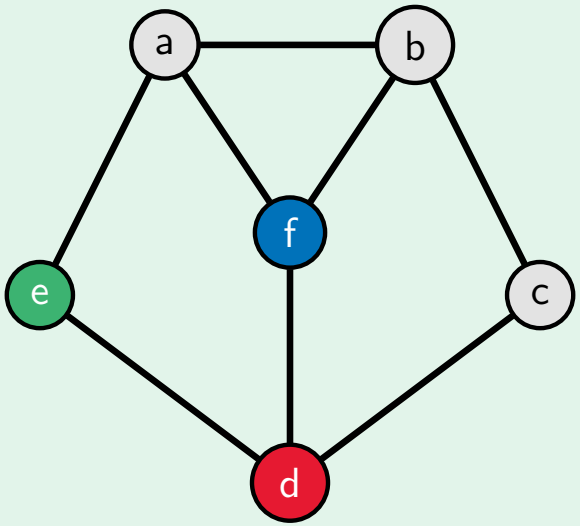
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Cuarta iteración)

Añadir nodo visitado al final de la pila



$P = [a, b, c, d, e] \quad V' = [f]$
$T = []$
$A' = [ab, bc, cd, de]$

Añadimos e al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

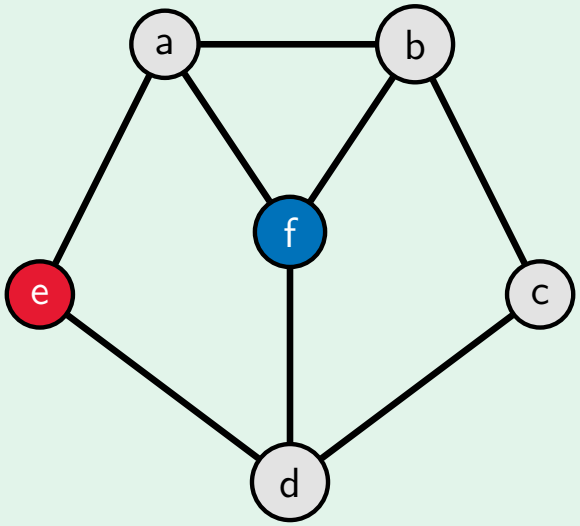
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Quinta iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b, c, d, e] \quad V' = [f]$

$T = []$

$A' = [ab, bc, cd, de]$

P no es vacía

Se elige como nodo activo al último nodo de P: e

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

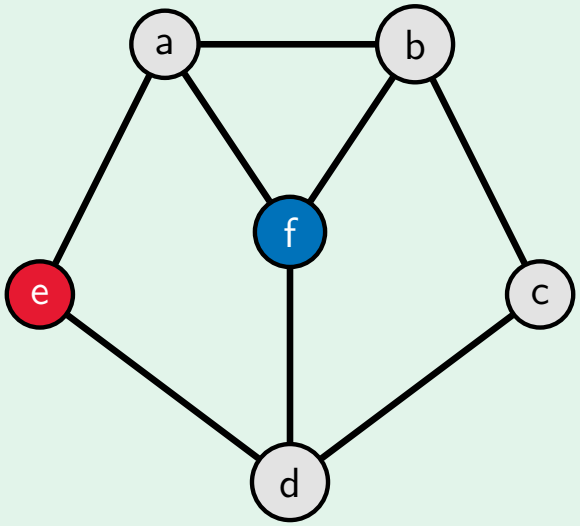
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Quinta iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [a, b, c, d, \cancel{e}]$ $\quad V' = [f]$

$T = [e]$

$A' = [ab, bc, cd, de]$

e no tiene adyacentes en V'

Se elimina e de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

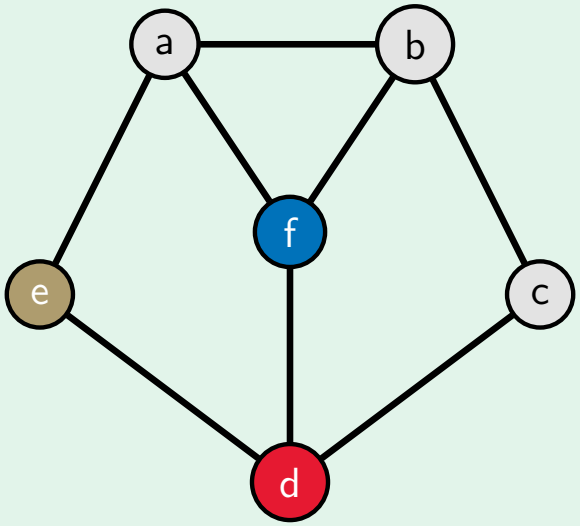
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Sexta iteración)

Comprobación de la pila y selección de nodo activo



$P = [a, b, c, d] \quad V' = [f]$
$T = [e]$
$A' = [ab, bc, cd, de]$

P no es vacía

Se elige como nodo activo al último nodo de P: d

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

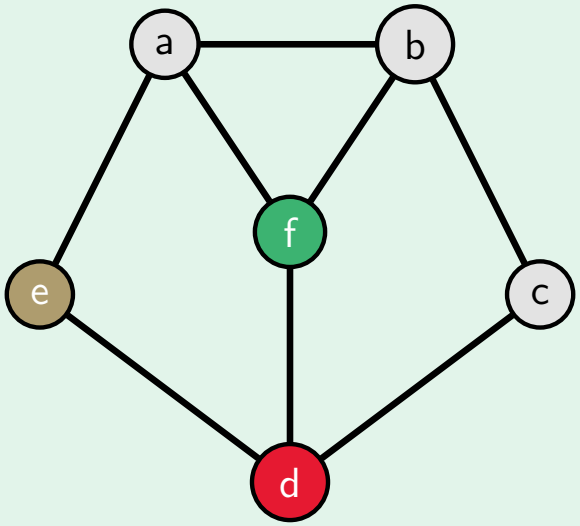
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Sexta iteración)

Comprobación de adyacentes y selección de nodo visitado



$P = [a, b, c, d] \quad V' = [f]$

$T = [e]$

$A' = [ab, bc, cd, de]$

d tiene adyacentes en V'

Se elige f como nodo visitado y se elimina de V'

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;

$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

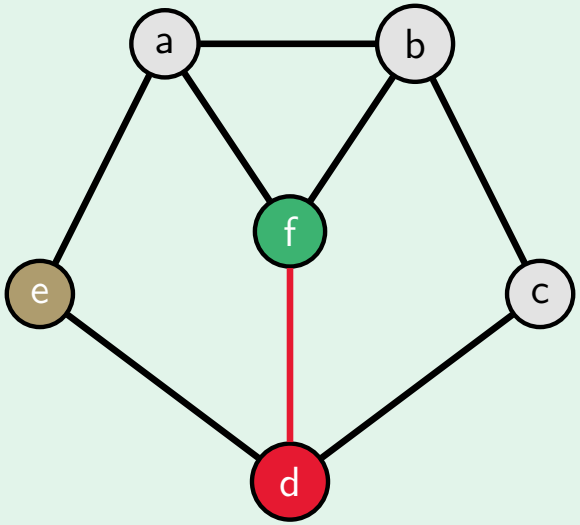
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Sexta iteración)

Añadir arista nodo activo - nodo visitado



$P = [a, b, c, d] \quad V' = []$

$T = [e]$

$A' = [ab, bc, cd, de, df]$

Añadimos a A' la arista nodo activo - nodo visitado: df

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$

$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$

$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

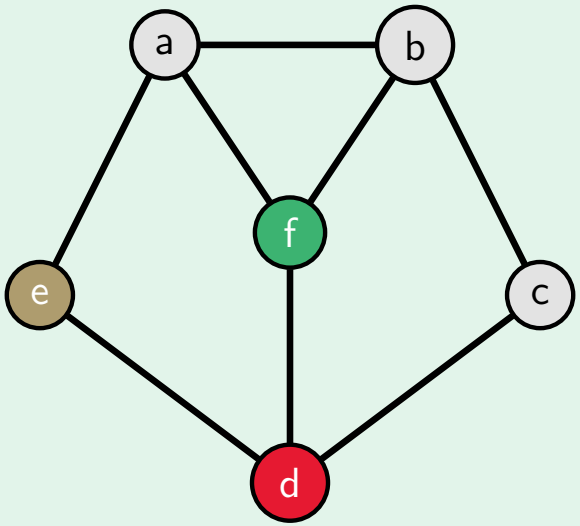
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Sexta iteración)

Añadir nodo visitado al final de la pila



$P = [a, b, c, d, f] \quad V' = []$
$T = [e]$
$A' = [ab, bc, cd, de, df]$

Añadimos f al final de la pila P

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

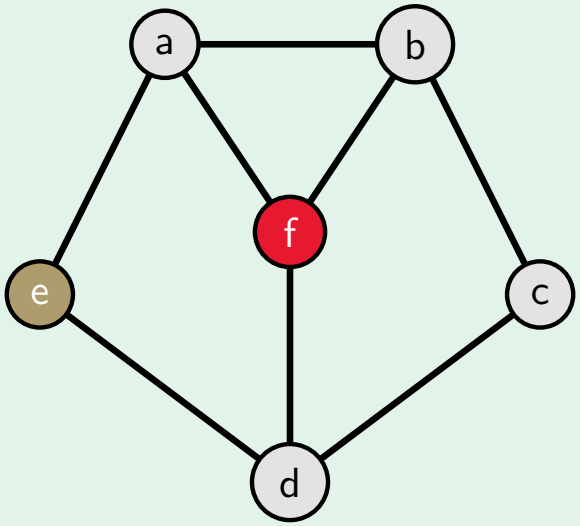
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Séptima iteración)

Comprobación de la pila y selección del nodo activo



$P = [a, b, c, d, f] \quad V' = []$

$T = [e]$

$A' = [ab, bc, cd, de, df]$

P no es vacía

Se elige como nodo activo al último nodo de P: f

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

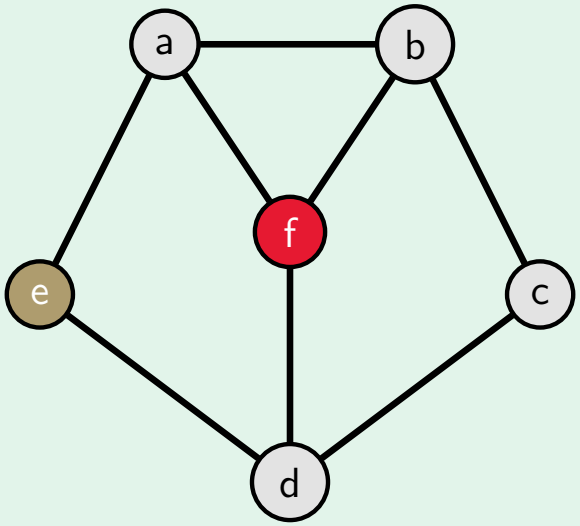
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Séptima iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [a, b, c, d, f]$ $\quad V' = []$

$T = [e, f]$

$A' = [ab, bc, cd, de, df]$

f no tiene adyacentes en V'

Se elimina f de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

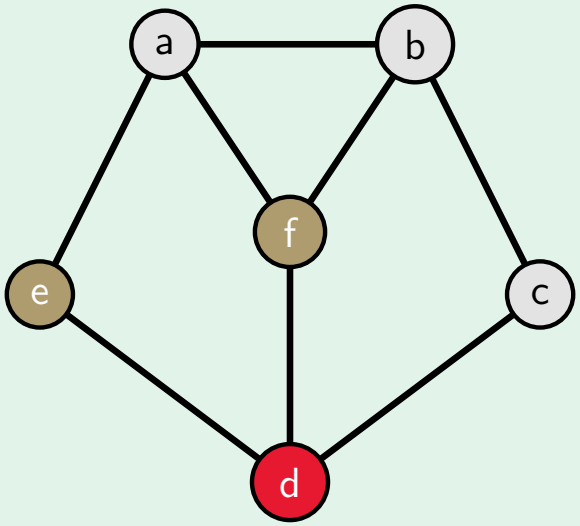
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Octava iteración)

Comprobación de la pila y selección del nodo activo



$P = [a, b, c, d] \quad V' = []$

$T = [e, f]$

$A' = [ab, bc, cd, de, df]$

P no es vacía

Se elige como nodo activo al último nodo de P: d

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

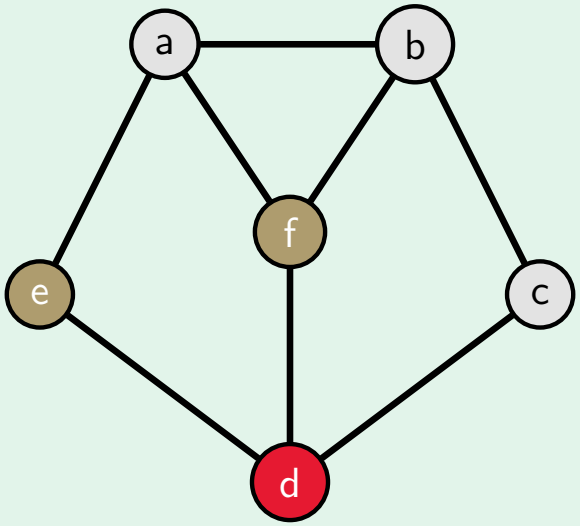
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Octava iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [a, b, c, \cancel{d}] \quad V' = []$
$T = [d, e, f]$
$A' = [ab, bc, cd, de, df]$

d no tiene adyacentes en V'

Se elimina d de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

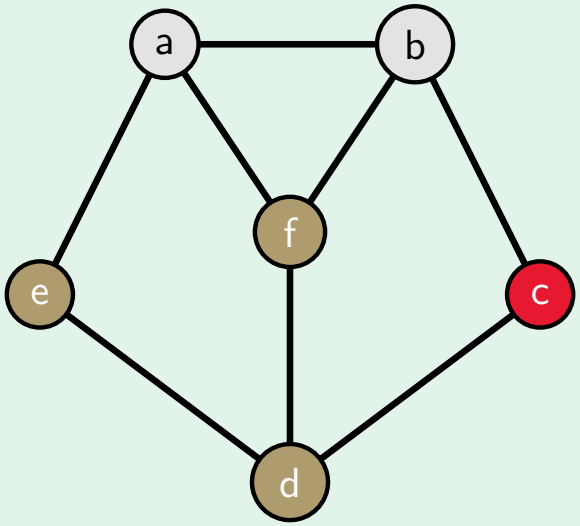
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Novena iteración)

Comprobación de la pila y selección del nodo activo



$P = [a, b, c] \quad V' = []$
$T = [d, e, f]$
$A' = [ab, bc, cd, de, df]$

P no es vacía

Se elige como nodo activo al último nodo de P: c

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

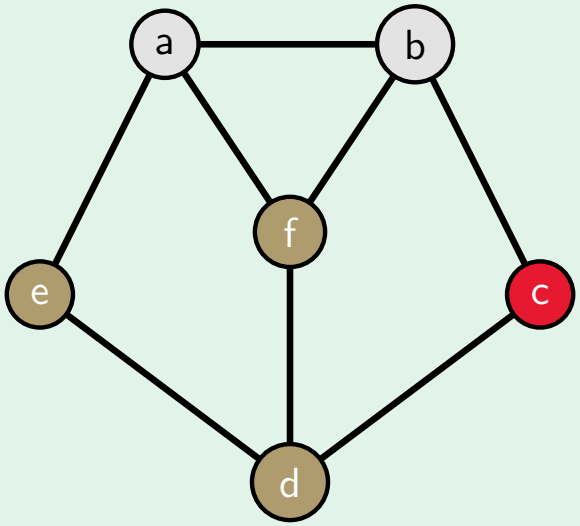
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Novena iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [a, b, \not{c}] \quad V' = []$

$T = [c, d, e, f]$

$A' = [ab, bc, cd, de, df]$

c no tiene adyacentes en V'

Se elimina c de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

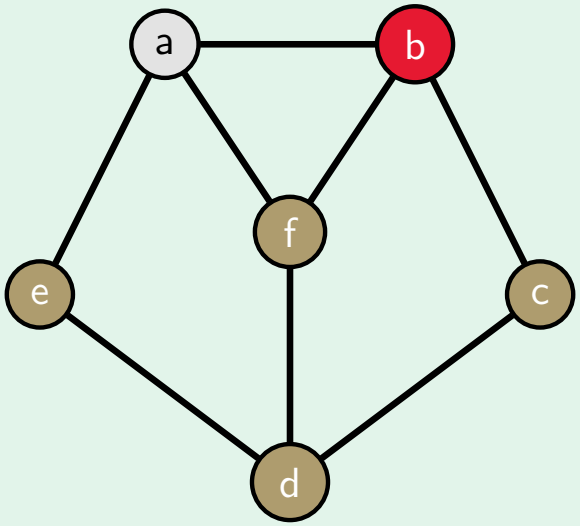
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Décima iteración)

Comprobación de la pila y selección del nodo activo



$P = [a, b] \quad V' = []$
$T = [c, d, e, f]$
$A' = [ab, bc, cd, de, df]$

P no es vacía

Se elige como nodo activo al último nodo de P: b

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

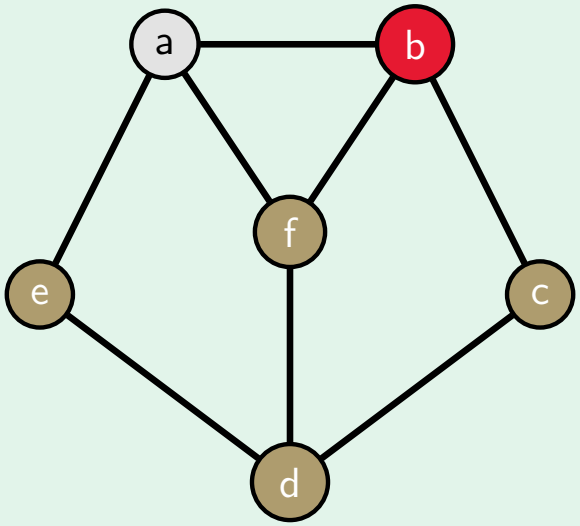
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Décima iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [a, \not{b}] \quad V' = []$

$T = [b, c, d, e, f]$

$A' = [ab, bc, cd, de, df]$

b no tiene adyacentes en V'

Se elimina b de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

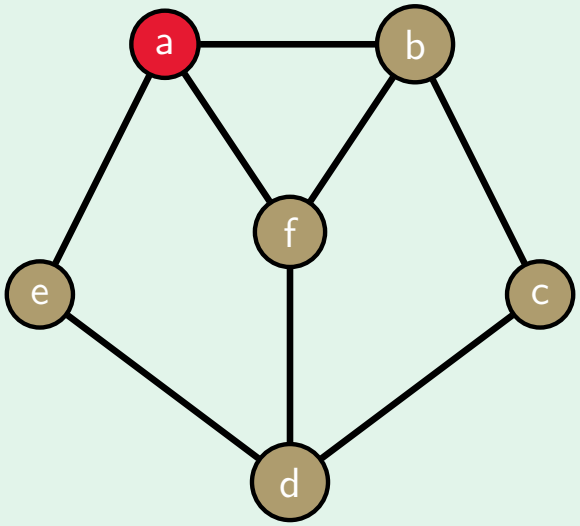
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Undécima iteración)

Comprobación de la pila y selección del nodo activo



$P = [a] \quad V' = []$

$T = [b, c, d, e, f]$

$A' = [ab, bc, cd, de, df]$

P no es vacía

Se elige como nodo activo al último nodo de P: a

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P

/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

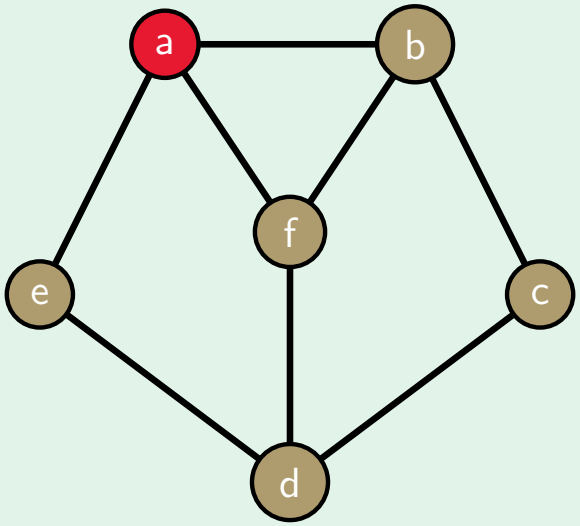
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Undécima iteración)

Comprobación de adyacentes y eliminación de la pila



$P = [\not{a}] \quad V' = []$

$T = [a, b, c, d, e, f]$

$A' = [ab, bc, cd, de, df]$

a no tiene adyacentes en V'

Se elimina a de la pila P y se añade a T

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

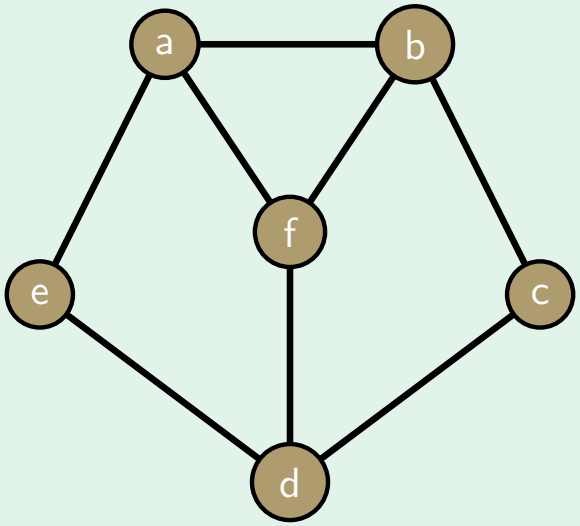
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Duodécima iteración)

Comprobación de la pila y finalización del argumento



$$P = [] \quad V' = []$$
$$T = [a, b, c, d, e, f]$$
$$A' = [ab, bc, cd, de, df]$$

P es vacía

Fin del algoritmo

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

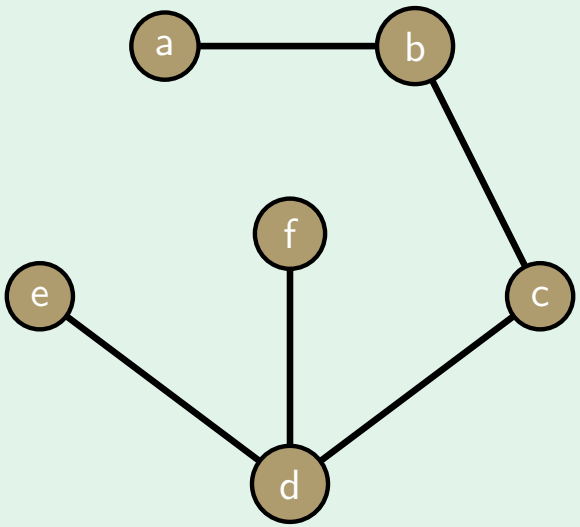
End if

End while

Return (T, A') árbol de búsqueda

Ejemplo (Resultado)

Comprobación de la pila y finalización del argumento



$P = []$ $\quad V' = []$

$T = [a, b, c, d, e, f]$

$A' = [ab, bc, cd, de, df]$

(T, A') es el árbol resultante

Pseudocódigo

$V' \leftarrow [v_2, v_3, \ldots, v_n]$; $T \leftarrow \emptyset$; $A' \leftarrow \emptyset$; $P \leftarrow [v_1]$
/* Inicialización de datos /*

while P no vacía

NodoActivo $\leftarrow$ **último** nodo de P
/* Activación en pila (LIFO) /*

if existe $v \in V'$ adyacente a NodoActivo:

NodoVisitado $\leftarrow v$;
$V' \leftarrow V' - \{\text{NodoVisitado}\}$
$A' \leftarrow A' \cup \{\text{NodoActivo-NodoVisitado}\}$
$P \leftarrow [\text{vert}(P), \text{NodoVisitado}]$
/* Se añade v al final de la pila P /*

Else

$P \leftarrow P - [\text{NodoActivo}]$
/* Se elimina el vértice activo de la pila /*

$T \leftarrow T \cup [\text{NodoActivo}]$
/* Se añade el vértice activo al árbol /*

End if

End while

Return (T, A') árbol de búsqueda

Comparación y aplicaciones de los algoritmos de búsqueda

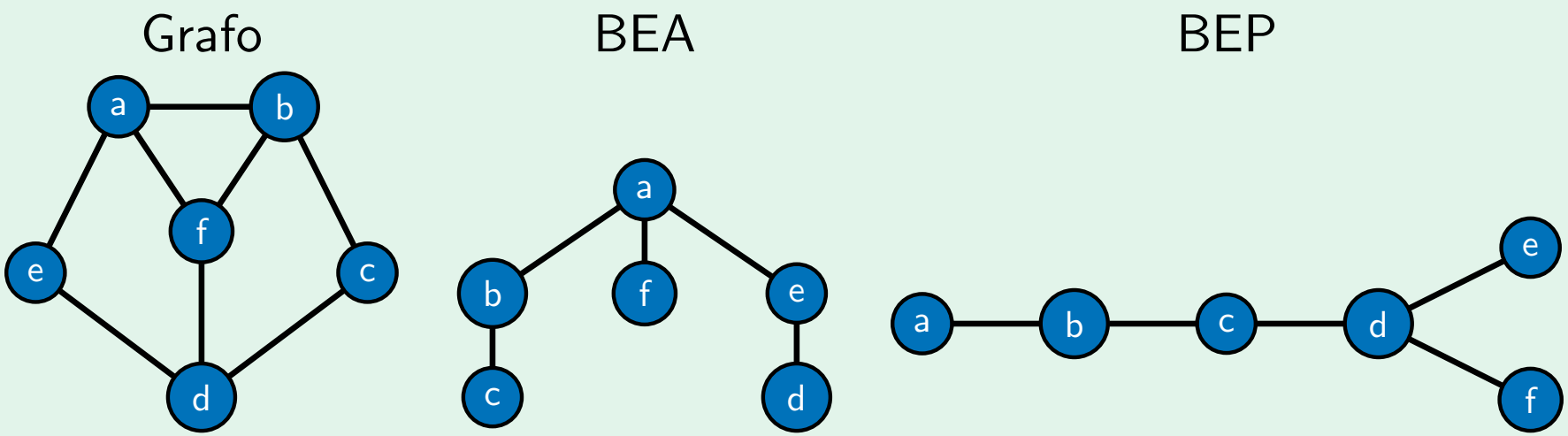
Comparación de algoritmos de búsqueda

Búsqueda en profundidad vs Búsqueda en anchura

Proposición

Sea G un grafo simple y sea O uno de sus vértices.
El algoritmo de búsqueda en anchura aplicado a G, con origen O, produce un árbol de altura inferior o igual a cualquier árbol obtenido por el algoritmo de búsqueda en profundidad aplicado a G, con origen O.

Ejemplo (Comparación de árboles resultantes)



Aplicaciones de los algoritmos de búsqueda

Búsqueda en anchura y búsqueda en profundidad

- Búsqueda de un vértice específico.
- Estudio de conectividad: ambos algoritmos permiten comprobar si el grafo es conexo si al terminar el árbol resultante T contiene todos los vértices del grafo.
- Árboles generadores: si G es conexo, ambos algoritmos proporcionan un árbol generador.
- Camino entre dos vértices: BEA con uno de los vértices como primer elemento proporciona un camino que los une, si ambos están en la misma componente conexa.
- Búsqueda de ciclos: Si el grafo es conexo, ambos algoritmos detectan si posee ciclos y calculan uno en su caso.
- Búsqueda de vértices-corte.
- Biconexión en grafos (conexión sin vértices corte).
- Obtención de caminos hamiltonianos.

Detección de vértices-corte usando BEP

Motivación

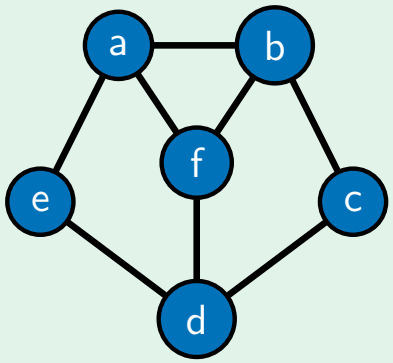
Los árboles de búsqueda en profundidad permiten detectar vértices-corte

Proposición

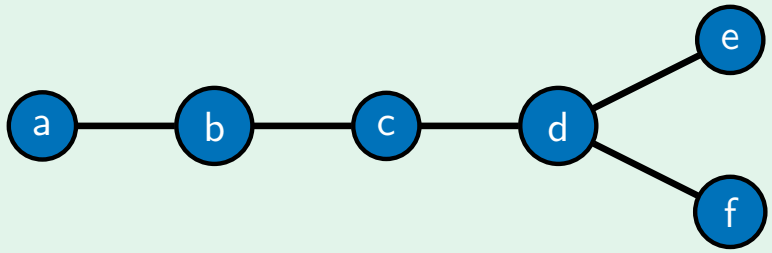
Sea G un grafo y T el árbol de búsqueda en profundidad de G con raíz v_0. La raíz es un vértice-corte si y sólo si tiene más de un hijo.

Ejemplo (Detección de vértices-corte)

Grafo



BEP con raíz en a



a solo tiene un hijo $\Longrightarrow$ a no es vértice-corte

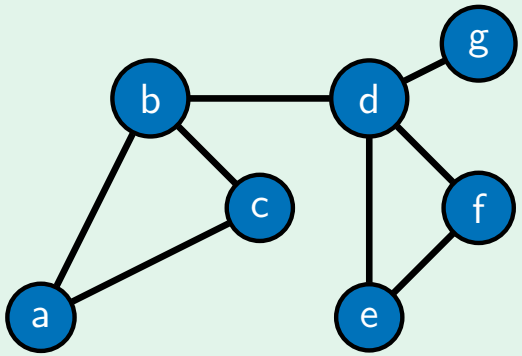
Detección de vértices-corte usando BEP

Proposición

Sea G un grafo y T el árbol de búsqueda en profundidad de G con raíz v_0. Un vértice $v \neq v_0$ es un vértice-corte si y sólo si tiene algún hijo w tal que w ni sus descendientes están conectados a algún ancestro de v por una arista no perteneciente a T, que llamamos **arista de retroceso**.

Ejemplo (Detección de vértices-corte)

Grafo

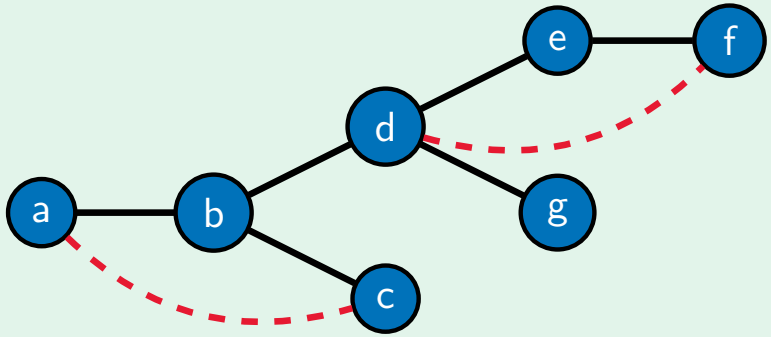


c, f, g no son vértice-corte

e no es vértice-corte

b, d son vértice-corte

BEP con raíz en a



c, f y g no tienen hijos

e tiene hijo f adyacente a ancestro d por arista de retroceso

b, d tienen hijo y descendientes sin aristas retroceso

Algoritmo de doble etiquetado

Detección de vértices-corte usando BEP

Idea: Modificación de algoritmo de BEP

Añadir un etiquetado adicional, *low*, al algoritmo de profundidad para aplicar la proposición anterior y detectar los vértices-corte.
Para cada vértice w, definimos la etiqueta $low(w)$ como el menor valor:

- $df(w)$;
- $df(v)$ con v adyacente a w por arista de retroceso;
- $low(u)$ con u un hijo de w.

Proposición: Vértices-corte y aristas puente

Sea G un grafo y T árbol de búsqueda en profundidad de G con raíz v_0.

- Un vértice $v \neq v_0$ es un vértice-corte si y sólo si $low(w) \geq df(v)$ para algún hijo w de v.
- Una arista $e = \{u, v\}$ es una arista puente si y sólo si e está en T y $low(v) > df(u)$ para u padre de v.

Algoritmo de doble etiquetado

Detección de vértices-corte usando BEP

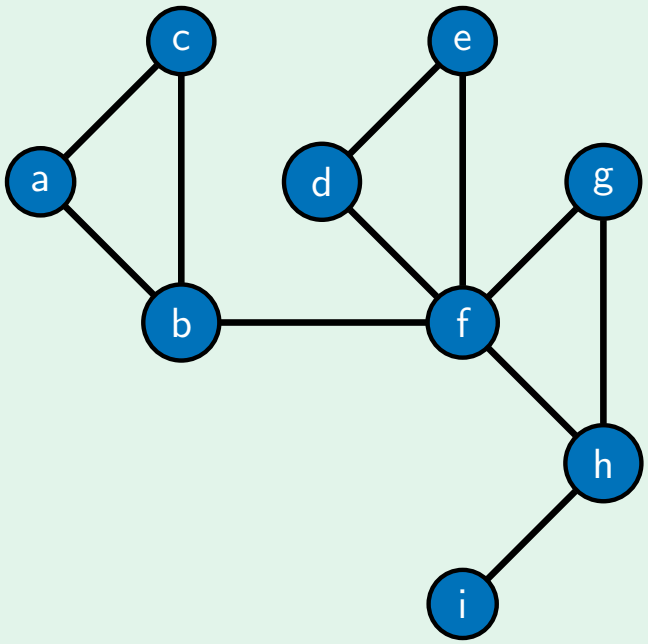
Algoritmo para generar el etiquetado *low*

Se recorren todos los vértices del árbol BEP por orden de alcance.

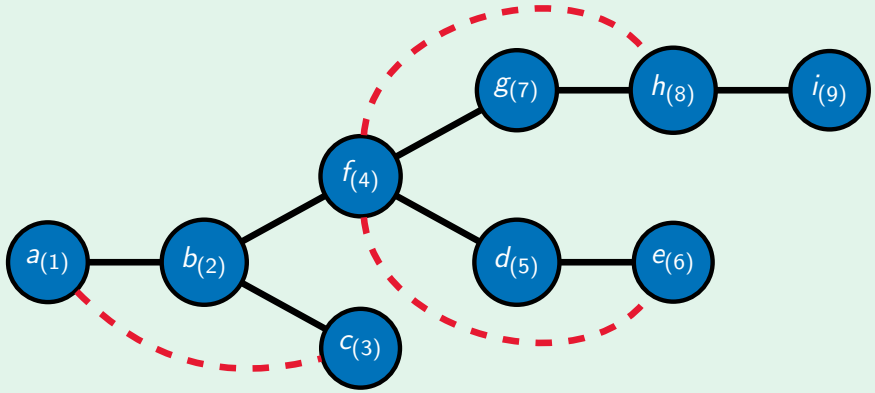
1. Si el vértice v no tiene etiqueta: $low(v) = df(v)$
2. Para cada arista en retroceso, de v a un descendiente suyo w:
   1. Si w no tiene etiqueta: $low(w) = df(v)$
   2. Para cada ancestro u de w que no tiene etiqueta: $low(u) = low(w)$

Ejemplo (Construcción del árbol generador)

Grafo inicial



Construcción por BEP de árbol T con raíz a

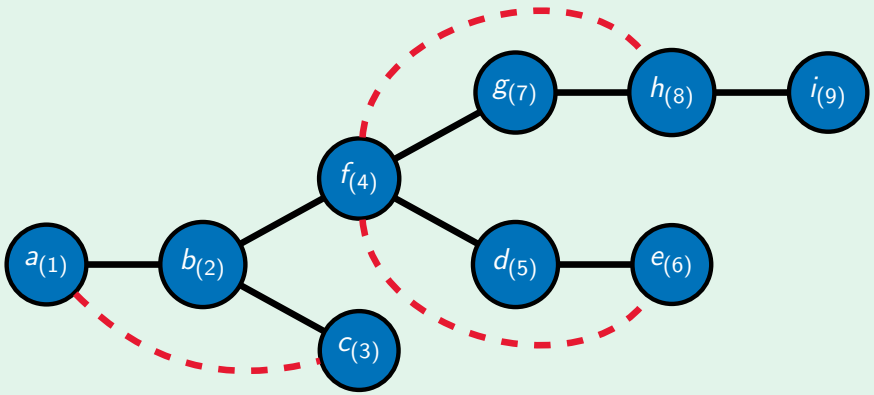


Aristas de retroceso

Cada vértice lleva su etiqueta df

Ejemplo (Primera iteración del doble etiquetado)

Árbol T con raíz a obtenido por BEP



$a \leftrightarrow (1,1)$, $c \leftrightarrow (3,1)$, $b \leftrightarrow (2,1)$

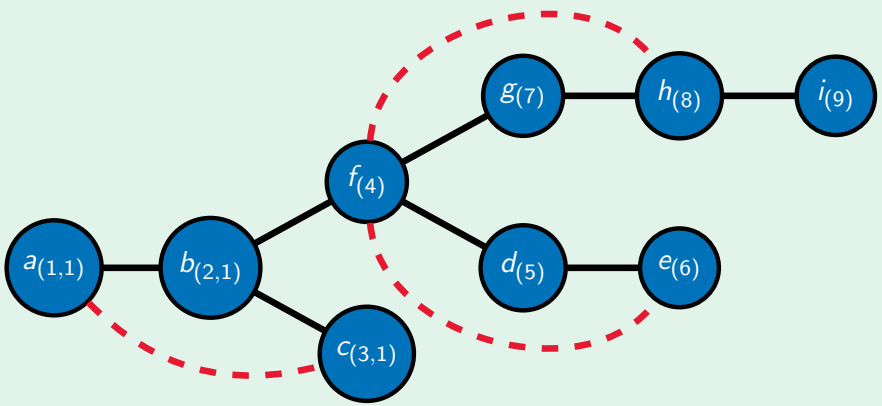
Hallamos el segundo etiquetado, *low*

El vértice a no tiene etiqueta:

1. $low(a) := df(a) = 1$
2. c es descendiente con arista de retroceso
   1. c no tiene etiqueta: $low(c) := df(a) = 1$
   2. b es ancestro de c y no tiene etiqueta: $low(b) := low(c) = 1$
   3. No hay más ancestros de c sin etiqueta
3. No hay más aristas de retroceso de a

Ejemplo (Segunda a cuarta iteración del doble etiquetado)

Árbol T con raíz a obtenido por BEP



$f \leftrightarrow (4,4)$, $e \leftrightarrow (6,4)$, $d \leftrightarrow (5,4)$
$h \leftrightarrow (8,4)$, $g \leftrightarrow (7,4)$

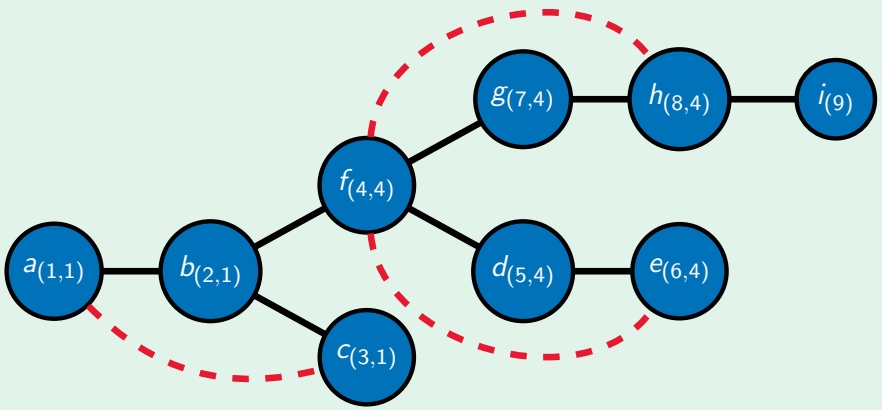
Hallamos el segundo etiquetado, *low*

El vértice b tiene etiqueta: $(2,1)$
El vértice c tiene etiqueta: $(3,1)$
El vértice f no tiene etiqueta: $low(f) := df(f) = 4$

1. e es descendiente con arista de retroceso
   1. e no tiene etiqueta: $low(e) := df(f) = 4$
   2. d es ancestro de e y no tiene etiqueta: $low(d) := low(e) = 4$
   3. No hay ancestros de e sin etiqueta
2. h es descendiente con arista de retroceso
   1. h no tiene etiqueta: $low(h) := df(f) = 4$
   2. g es ancestro de h y no tiene etiqueta: $low(g) := low(h) = 4$
   3. No hay más ancestros de h sin etiqueta
3. No hay más aristas de retroceso de f

Ejemplo (Quinta a novena iteración del doble etiquetado)

Árbol T con raíz a obtenido por BEP



$i \leftrightarrow (9, 9)$

Hallamos el segundo etiquetado, *low*

El vértice d tiene etiqueta: $(5, 4)$

El vértice e tiene etiqueta: $(6, 4)$

El vértice g tiene etiqueta: $(7, 4)$

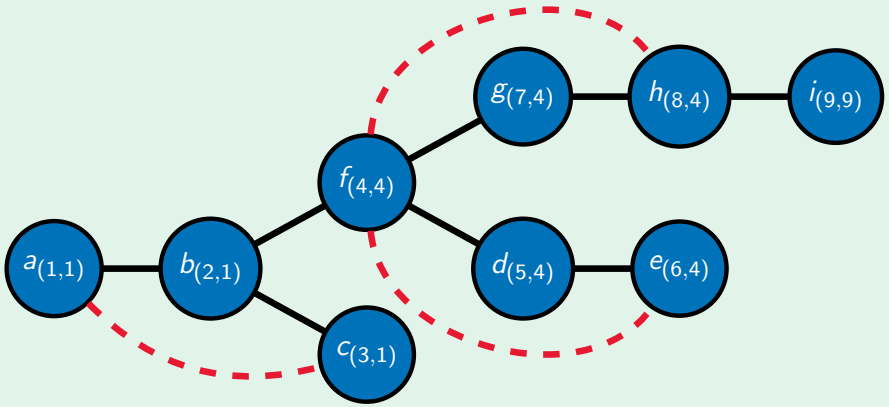
El vértice h tiene etiqueta: $(8, 4)$

El vértice i no tiene etiqueta: $low(i) := df(i) = 9$

1. i no tiene aristas de retroceso

Ejemplo (Fin del doble etiquetado)

Árbol T con raíz a obtenido por BEP



Fin del segundo etiquetado, *low*

Cada vértice tiene su doble etiqueta (df, low)

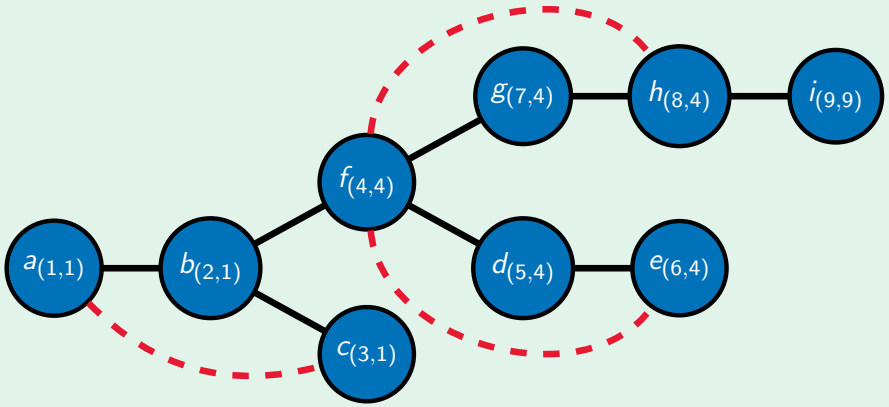
$a \leftrightarrow (1,1)$, $b \leftrightarrow (2,1)$, $c \leftrightarrow (3,1)$

$d \leftrightarrow (5,4)$, $e \leftrightarrow (6,4)$, $f \leftrightarrow (4,4)$

$g \leftrightarrow (7,4)$, $h \leftrightarrow (8,4)$, $i \leftrightarrow (9,9)$

Ejemplo (Búsqueda de vértices-corte)

Árbol T con raíz a obtenido por BEP



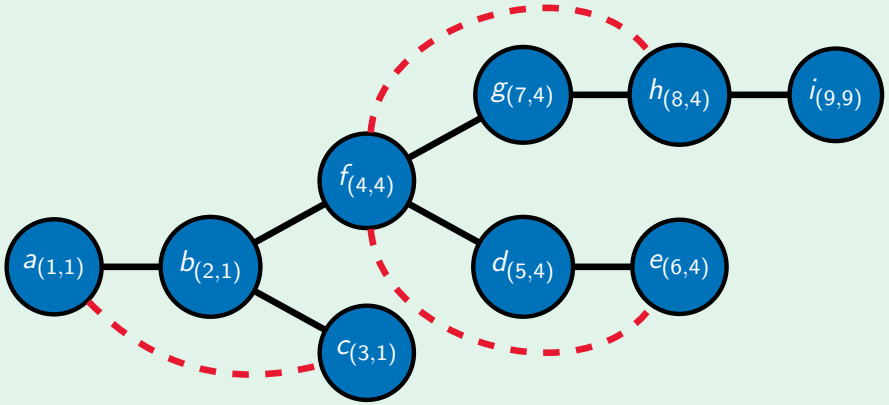
- b, f, h son vértices-corte
- c, d, e, g, i no son vértices-corte

Hallamos los vértices de corte

- f hijo de b: $2 = df(b) \leq low(f) = 4$
- c no tiene hijos
- e único hijo de d: $5 = df(d) > low(e) = 4$
- e no tiene hijos
- d hijo de f: $4 = df(f) \leq low(d) = 4$
- h único hijo de g: $7 = df(g) > low(h) = 4$
- i hijo de h: $8 = df(h) \leq low(i) = 9$
- i no tiene hijos

Ejemplo (Búsqueda de aristas puente)

Árbol T con raíz a obtenido por BEP



- bf, hi son aristas puente
- ab, ac, bc, de, fd, fe, fh, fg, gh no son aristas puente

Hallamos las aristas puente

- ab: $1 = low(b) \leq df(a) = 1$
- bc: $1 = low(c) \leq df(b) = 2$
- bf: $4 = low(f) > df(b) = 2$
- fd: $4 = low(d) \leq df(f) = 4$
- de: $4 = low(e) \leq df(d) = 5$
- fg: $4 = low(g) \leq df(f) = 4$
- gh: $4 = low(h) \leq df(g) = 7$
- hi: $9 = low(i) > df(h) = 8$
- ac, fe, fh no son aristas de T

Grafos ponderados

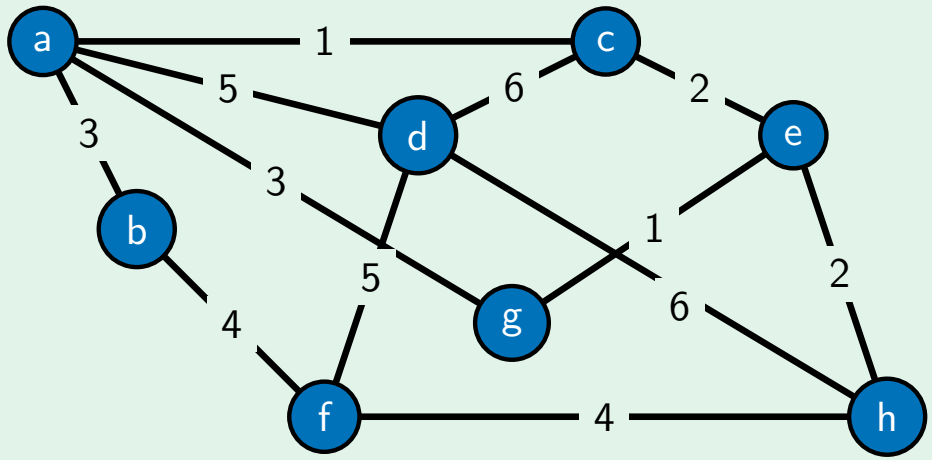
Definición

Un **grafo ponderado** (o grafo con pesos) es un grafo $G = (V, E, \varphi)$ equipado con una función de pesos $w\colon E \to \mathbb{R}$, que asigna a cada arista $e \in E$ un **peso** $w(e)$.

Ejemplo (Grafo ponderado)

Grafo ponderado $G = (V, E, \varphi)$ con función de pesos:

$$
\begin{aligned}
w(ac) &= w(eg) = 1 \\
w(ce) &= w(eh) = 2 \\
w(ab) &= w(ag) = 3 \\
w(bf) &= w(fh) = 4 \\
w(ad) &= w(df) = 5 \\
w(cd) &= w(dh) = 6
\end{aligned}
$$



Peso de un subgrafo

Definición y longitud de un camino

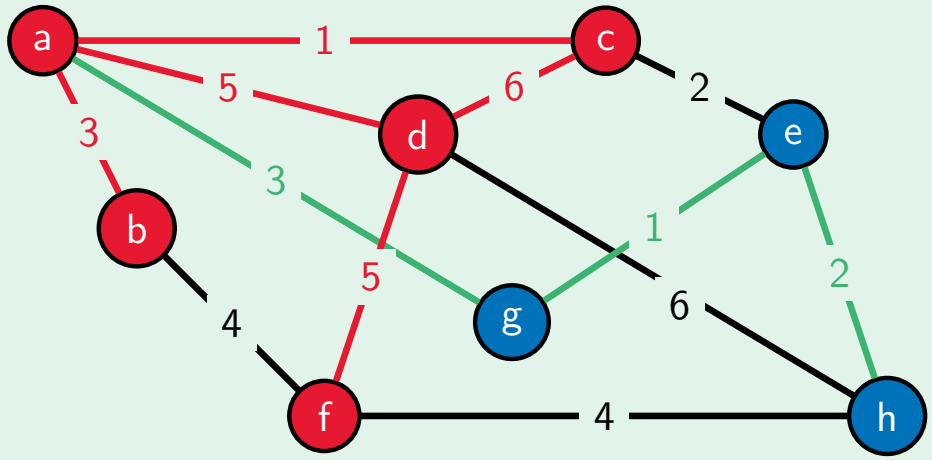
Definición

Sea (G, w) un grafo ponderado.

- El **peso del subgrafo** $H = (V', E', \varphi')$ de G es $w(H) := \sum_{e \in E'} w(e)$.
- La **longitud** o **peso** de un camino que une dos vértices es la suma de los pesos de las aristas que lo componen.

Ejemplo (Pesos de subgrafos)

Grafo ponderado $G = (V, E, \varphi)$
Subgrafo H
Camino desde a hasta h
Peso del subgrafo: $w(H) = 20$
Peso del camino: $w(a \to h) = 6$



Árboles generadores de peso mínimo

Ideas generales

Dado un grafo conexo, el árbol generador de peso mínimo:

- Es un árbol que contiene todos los vértices del grafo conectados por aristas, tal que la suma de los pesos es mínima.
- También se conoce como Minimum Spanning Tree (MST)
- No tiene porqué ser único.
- Resuelve problemas de optimización de costes.
- No incluye a la arista de mayor peso de un ciclo.
- No necesariamente contiene a la arista de menor peso de un ciclo.

Algoritmos para construir MST

- **Algoritmo de Kruskal**: Añade aristas de peso mínimo comprobando que no forman un ciclo.
- **Algoritmo de Prim**: Añade un vértice y su arista correspondiente de menor peso.

Algoritmo de Kruskal

Estructura del algoritmo

Input: Grafo ponderado conexo $G = (V, E, \varphi)$ con peso $w\colon E \to \mathbb{R}$.

1. Iniciamos $E_T = \emptyset$
2. Mientras $|E_T| < n - 1$:
   1. Escogemos una arista $e \in E$ con menor peso $w(e)$.
   2. Si $(V, E_T \cup \{e\})$ no tiene ciclos:
      1. Añadimos la arista e al árbol: $E_T = E_T \cup \{e\}$
      2. Eliminamos la arista e del conjunto auxiliar: $E = E - \{e\}$
3. Devuelve el árbol generador de peso mínimo $T = (V, E_T)$.

Output: Árbol generador $T = (V, E_T)$ de G que tiene peso mínimo.

Pseudocódigo

Input: Grafo ponderado conexo $G = (V, E, \varphi, w)$
$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$ /* Inicialización de datos /*
while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T, hacer:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$ /* Eliminamos la arista del conjunto auxiliar /*

End while
Return $T = (V, E_T)$ árbol de peso mínimo

Pseudocódigo

Input: Grafo ponderado conexo $G = (V, E, \varphi, w)$

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$ /* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T, hacer:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

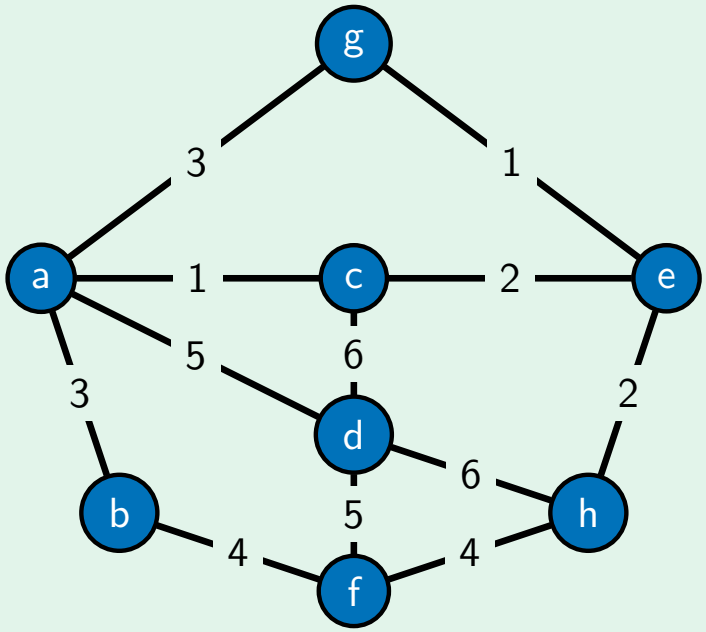
End if

$E \leftarrow E - \{a\}$ /* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Inicialización)



$E = [ac, eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \emptyset$

$i = 1, \quad n = 8$

Ordenamos las aristas de E e iniciamos los datos

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$

/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

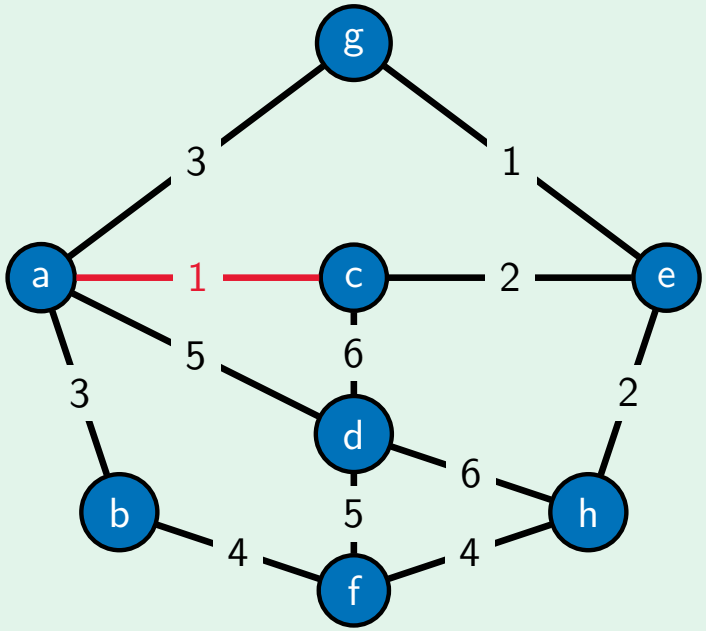
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Primera iteración)

Comprobación de paso y selección de arista de menor peso



$E = [ac, eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \emptyset$

$i = 1, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ac

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

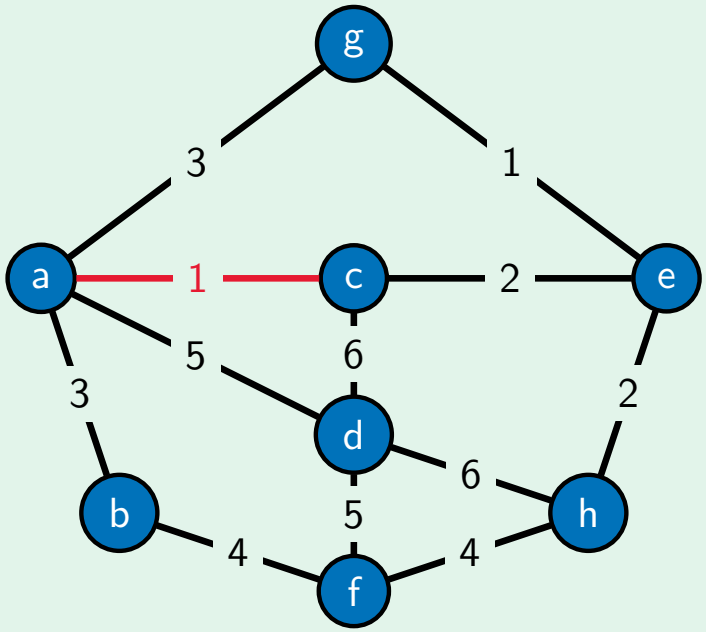
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Primera iteración)

Actualización de estructuras



$E = [\cancel{ac}, eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac\}$ ac no forma ciclo en E_T

$i = 1 + 1, \quad n = 8$

ac se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

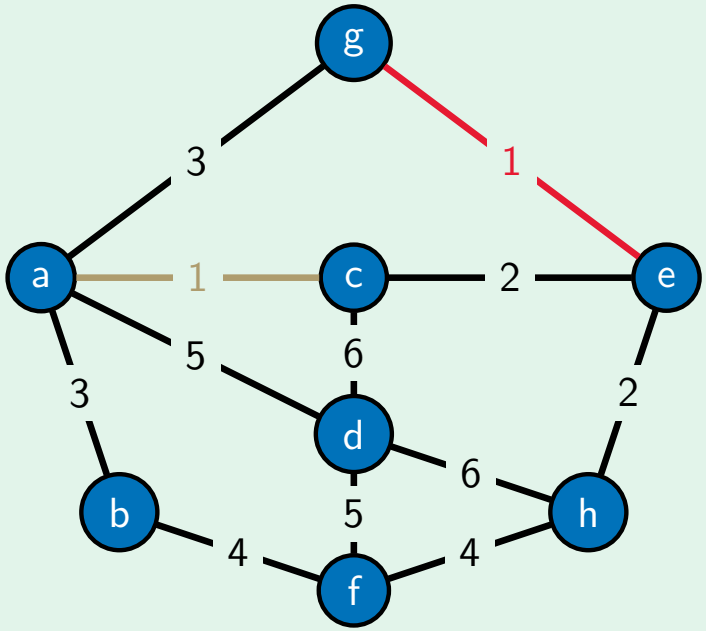
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Segunda iteración)

Comprobación de paso y selección de arista de menor peso



$E = [eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac\}$

$i = 2, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: eg

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

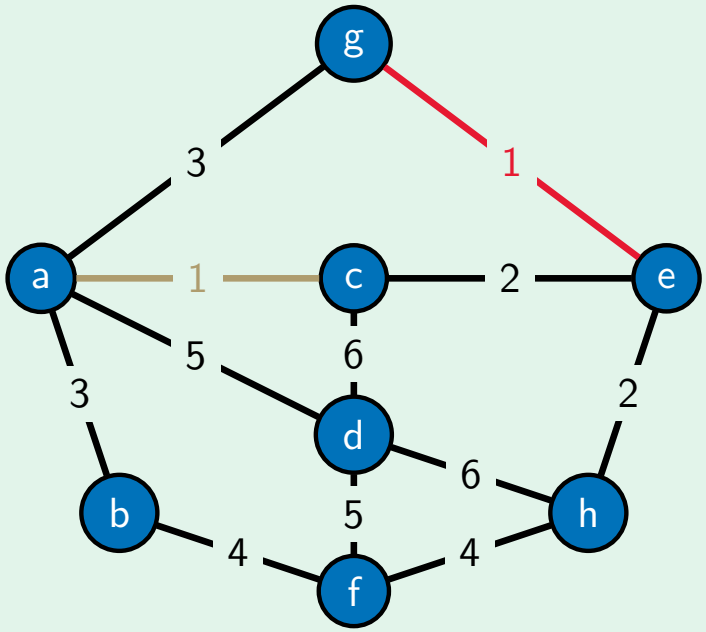
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Segunda iteración)

Actualización de estructuras



$E = [\cancel{eg}, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg\}$ eg no forma ciclo en E_T

$i = 2 + 1, \quad n = 8$

eg se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

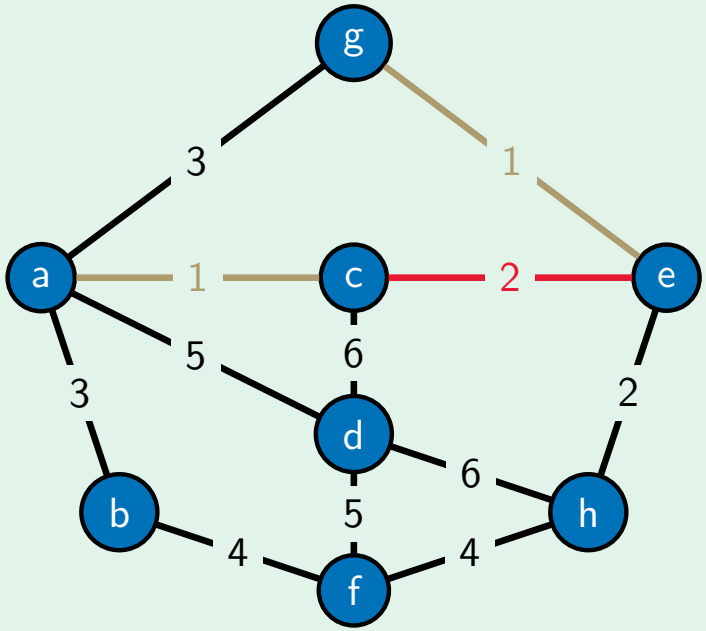
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Comprobación de paso y selección de arista de menor peso



$E = [ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg\}$

$i = 3, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ce

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

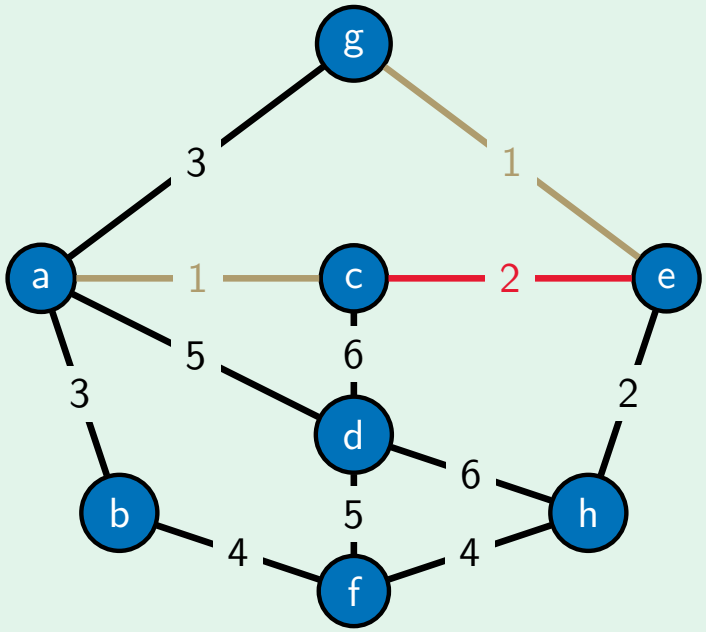
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Actualización de estructuras



$E = [\cancel{ce}, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce\}$ ce no forma ciclo en E_T

$i = 3 + 1, \quad n = 8$

ce se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

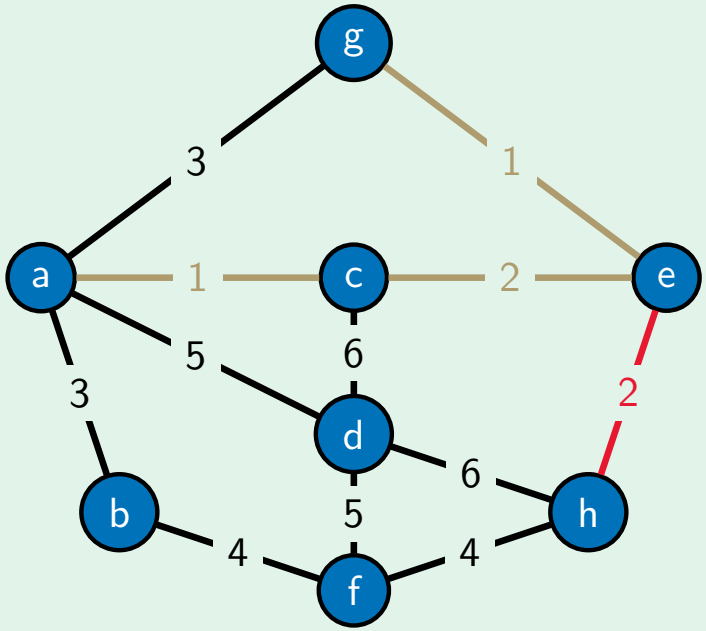
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Cuarta iteración)

Comprobación de paso y selección de arista de menor peso



$E = [eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce\}$

$i = 4, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: eh

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

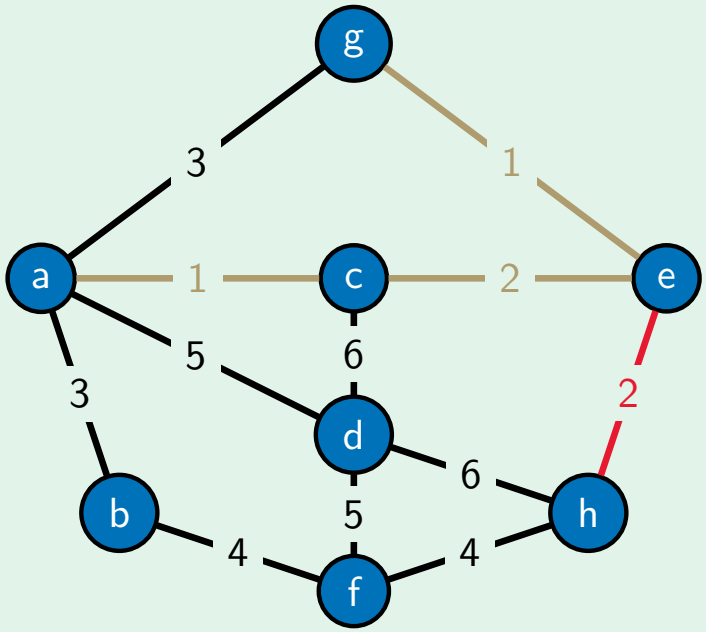
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Cuarta iteración)

Actualización de estructuras



$E = [\cancel{eh}, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh\}$ eh no forma ciclo en E_T

$i = 4 + 1, \quad n = 8$

eh se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

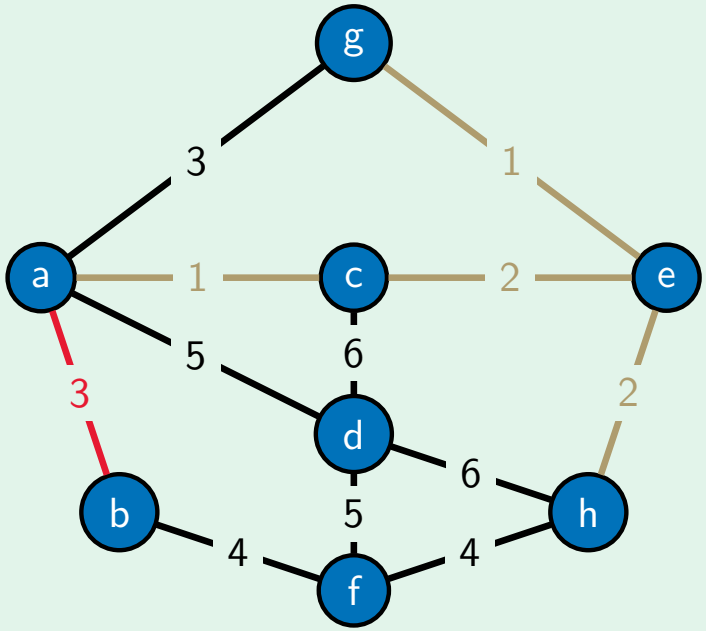
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Comprobación de paso y selección de arista de menor peso



$E = [ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh\}$

$i = 5, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ab

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

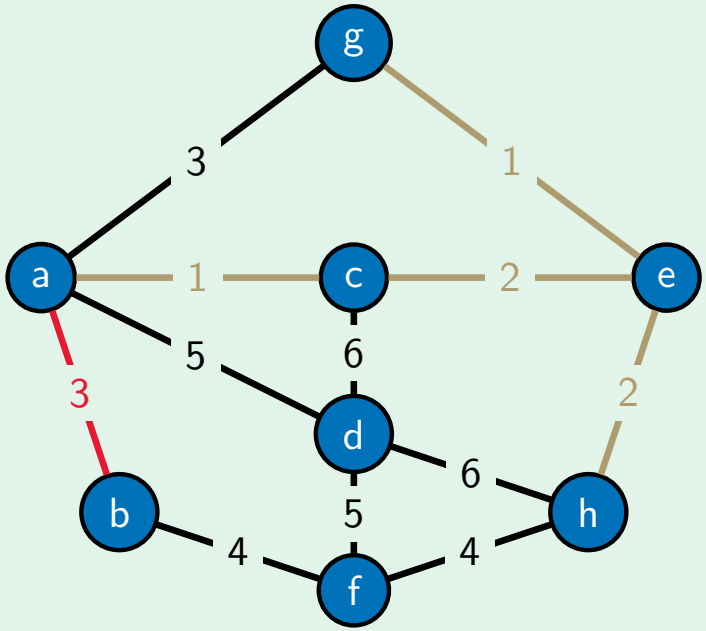
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Actualización de estructuras



$E = [\cancel{ab}, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

$i = 5 + 1, \quad n = 8$ ab no forma ciclo en E_T

ab se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$

/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

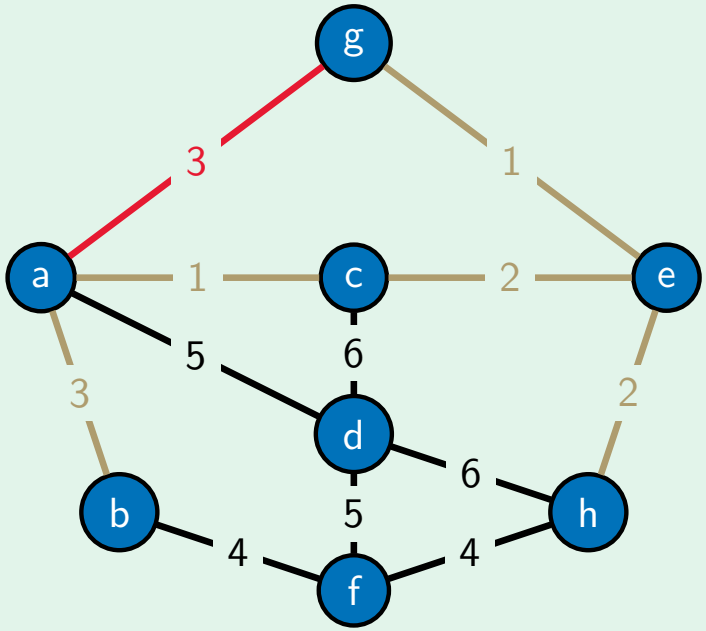
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Sexta iteración)

Comprobación de paso y selección de arista de menor peso



$E = [ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

$i = 6, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ag

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

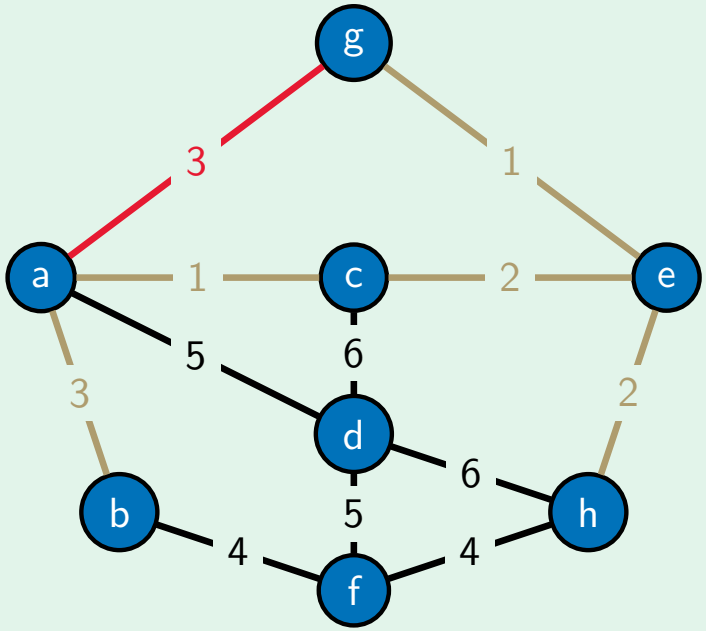
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Sexta iteración)

Actualización de estructuras



$E = [\cancel{ag}, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

$i = 6, \quad n = 8$ ag forma ciclo en E_T

ag se elimina de E

Se mantiene el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

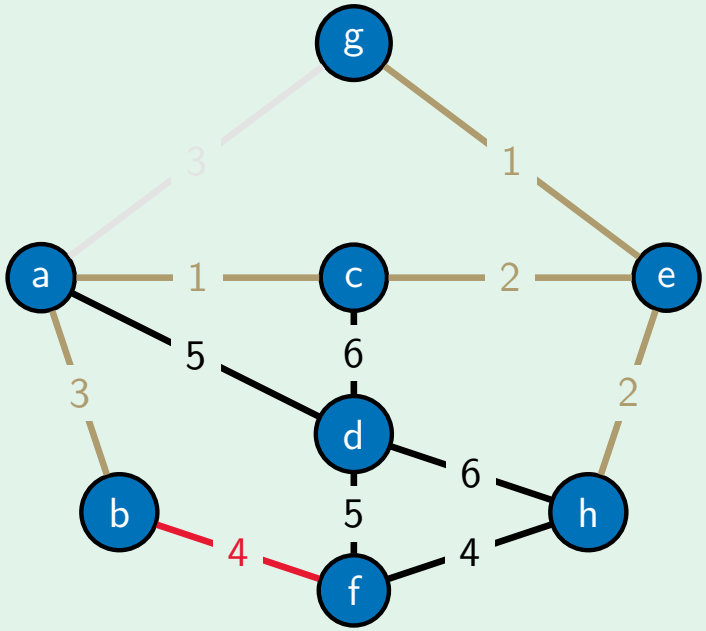
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Séptima iteración)

Comprobación de paso y selección de arista de menor peso



$E = [bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

$i = 6, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: bf

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

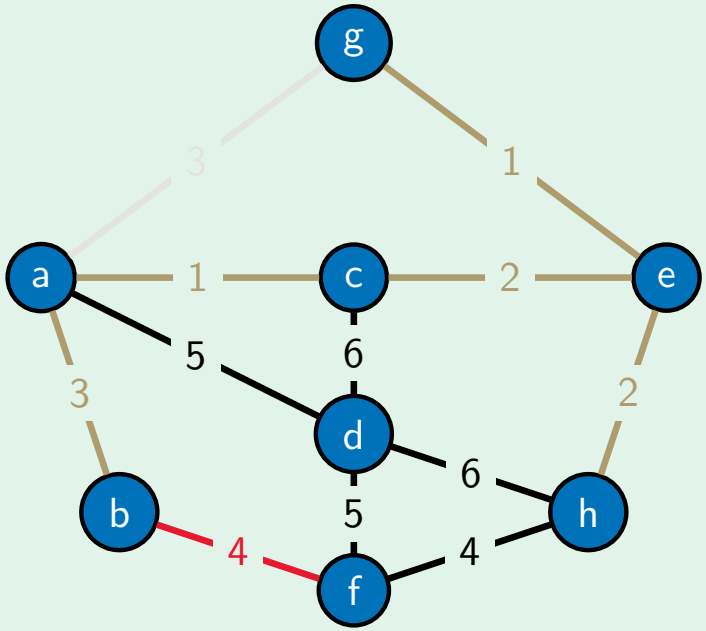
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Séptima iteración)

Actualización de estructuras



$E = [\cancel{bf}, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

$i = 6 + 1, \quad n = 8$ bf no forma ciclo en E_T

bf se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

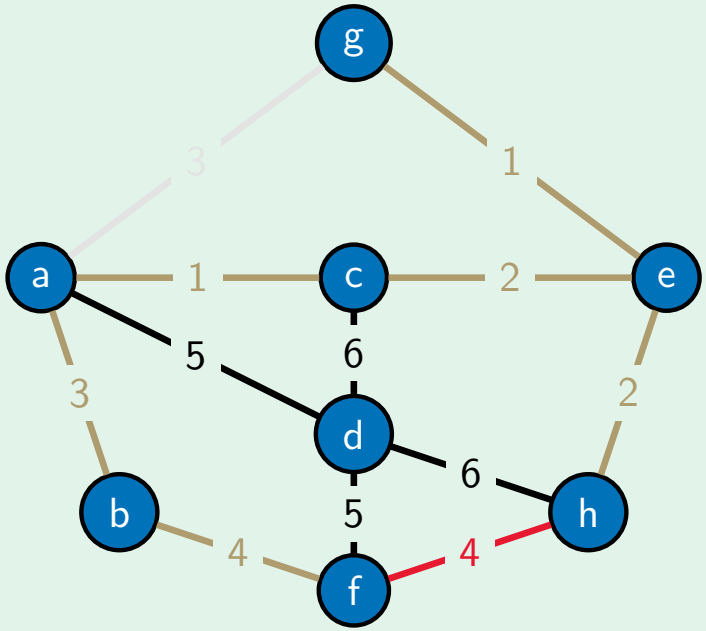
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Octava iteración)

Comprobación de paso y selección de arista de menor peso



$E = [fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

$i = 7, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: fh

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

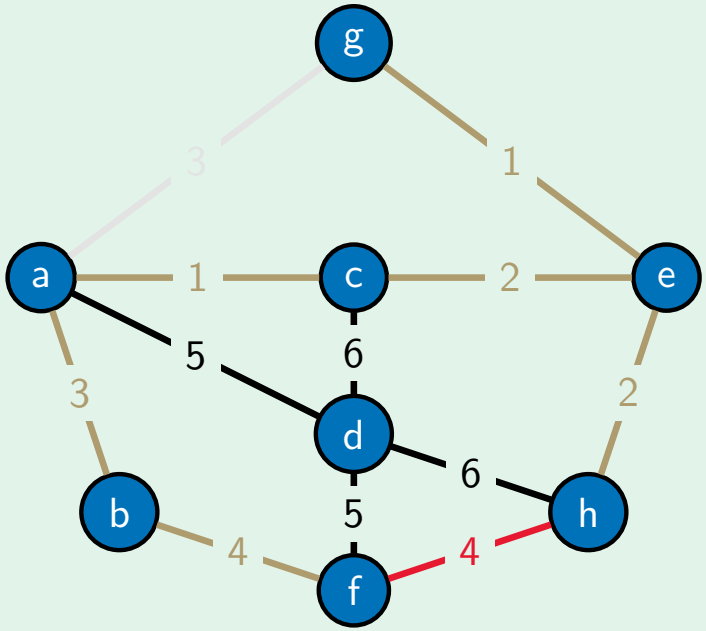
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Octava iteración)

Actualización de estructuras



$E = [\cancel{fh}, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

$i = 7, \quad n = 8$ *fh* forma ciclo en E_T

fh se elimina de E

Se mantiene el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

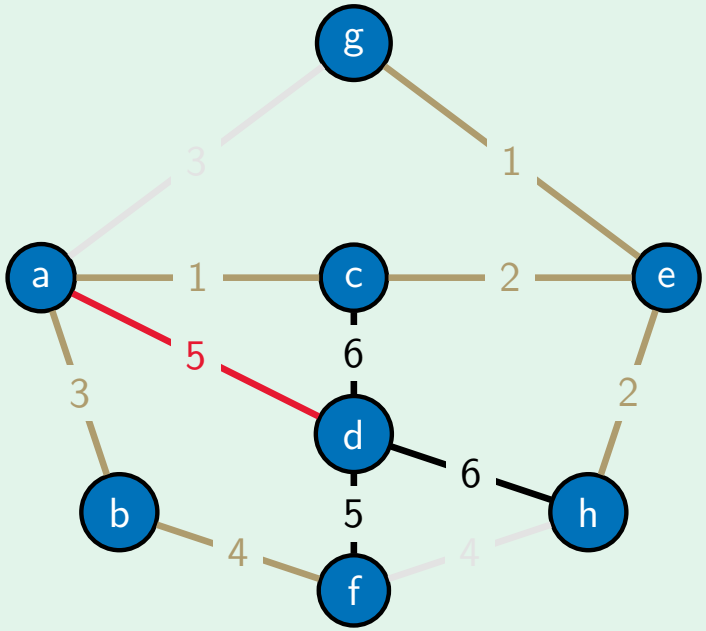
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Novena iteración)

Comprobación de paso y selección de arista de menor peso



$E = [ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

$i = 7, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ad

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

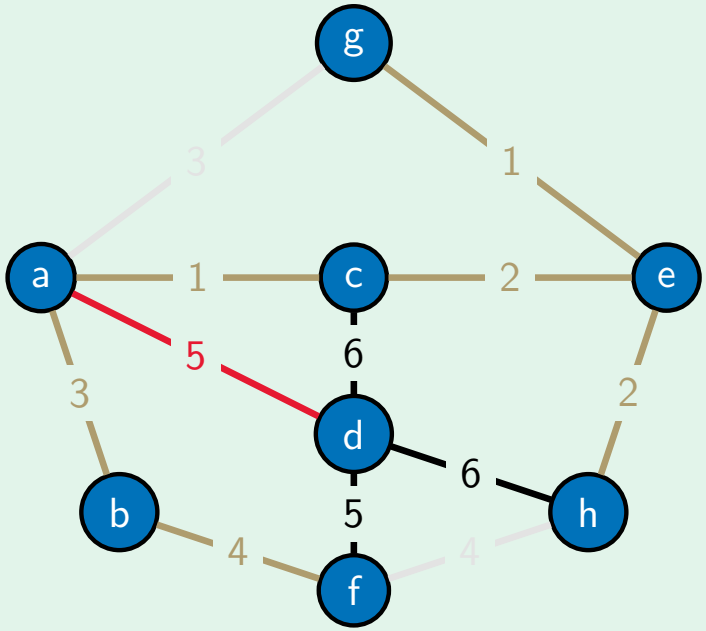
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Novena iteración)

Actualización de estructuras



$E = [\cancel{ad}, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf, ad\}$

$i = 7 + 1, \quad n = 8$ ad no forma ciclo en E_T

ad se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$

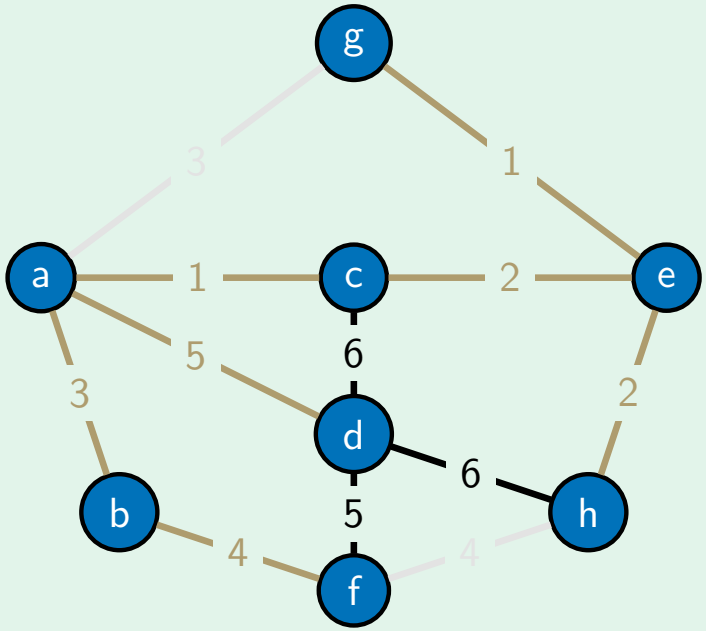
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Décima iteración)

Comprobación de paso y fin del algoritmo



$E = [df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf, ad\}$

$i = 8, \quad n = 8$ No se cumple $i < n$

Finalizamos el algoritmo

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

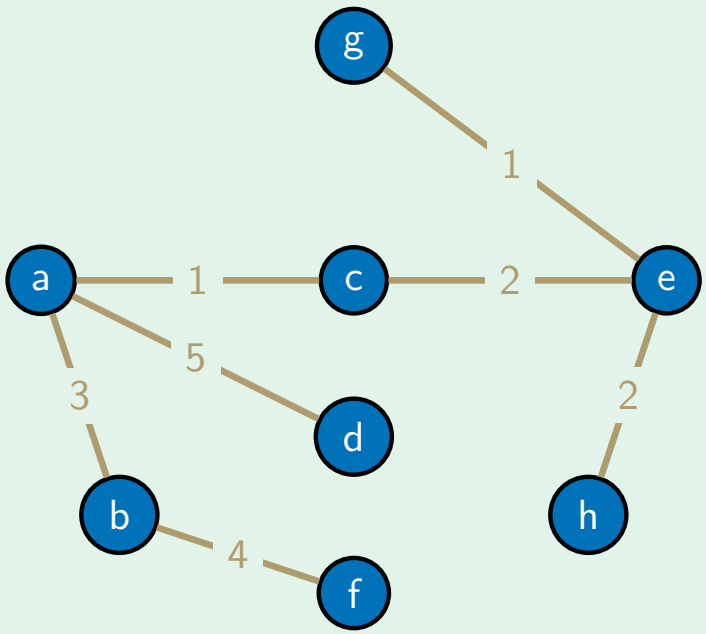
$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Resultado)

Árbol resultante



$V = \{a, b, c, d, e, f, g, h\}$

$E_T = \{ac, eg, ce, eh, ab, bf, ad\}$

$T = (V, E_T)$ es el árbol resultante

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Teorema: Validez del algoritmo de Kruskal

Dado un grafo conexo y ponderado, el algoritmo de Kruskal construye un árbol generador de peso mínimo para dicho grafo.

Demostración.

Sea G un grafo conexo, de n vértices, con función de peso w.
Sea T un grafo obtenido al aplicar el algoritmo de Kruskal sobre G.

- Se observa que T es un árbol generador:
  - Es un árbol porque no tiene ciclos, como, además tiene $n-1$ aristas para n vértices, es conexo.
  - Es generador porque contiene todos los vértices de G.
- Debemos comprobar que T tiene peso mínimo.

Continuación de la demostración.

- Probamos que T tiene peso mínimo por contradicción.
  - Supongamos que T no tiene peso mínimo, entonces existe otro árbol generador de G con peso estrictamente menor.
  - Entre esos árboles, tomamos S un árbol generador de peso mínimo que contiene el mayor número de aristas de T (en orden).
  - En orden, tomamos la primera arista e de T que no aparece en S.
  - El grafo $S \cup \{e\}$ contiene un ciclo C donde aparece la arista e.
  - Como T es un árbol, no tiene ciclos. Debe existir otra arista e^* del ciclo C, no contenida en T.
  - El algoritmo escoge la arista de menor peso posible que no forme un ciclo para añadir a T. Al añadir e al árbol, también estaba disponible e^*. Entonces $w(e) \leq w(e^*)$
  - Consideramos $S^* = S \cup \{e\} - \{e^*\}$. Es un árbol generador, conexo, con $n-1$ aristas, de peso mínimo pues $w(S^*) \leq w(S)$.
  - Sin embargo, S^* tiene una arista más en común con T que S. **¡Contradicción!**

Algoritmo de Kruskal

Análisis de complejidad

Pseudocódigo

$E \leftarrow$ ordenación de E según peso;
$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E

if a no forma ciclo con las aristas de E_T:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

La comprobación del ciclo
es el paso que añade
más complejidad al algoritmo

Los vértices no adyacentes u y v
están en la misma
componente conexa

⇕

Al añadir la arista uv,
se forma un ciclo con
las otras aristas de esa componente

Sustituimos la comprobación de ciclo
por la comprobación de componente

Pseudocódigo modificado

Input: Grafo ponderado conexo $G = (V, E, \varphi, w)$

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$;

$Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$ /* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E; $(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices asociados a u y v en $Comp$

if $j \neq k$, hacer:

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$i \leftarrow i + 1$; en $Comp$ sustituir k por j en todos los pares que aparece

End if

$E \leftarrow E - \{a\}$ /* Eliminamos la arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

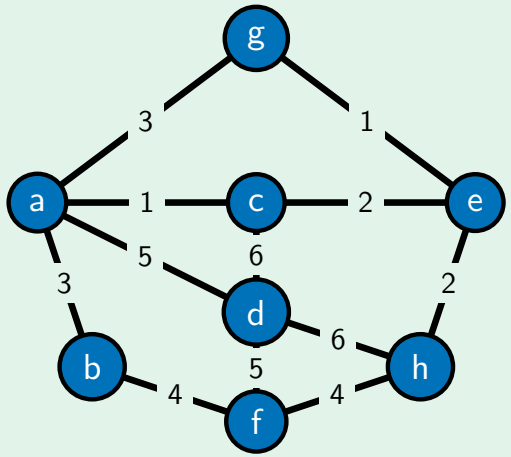
Algoritmo de Kruskal modificado

Ejemplo

Ejemplo (Inicialización)

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8



$E = [ac, eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \emptyset$

$i = 1, \quad n = 8$

Ordenamos las aristas de E e iniciamos los datos

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;

$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$

/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

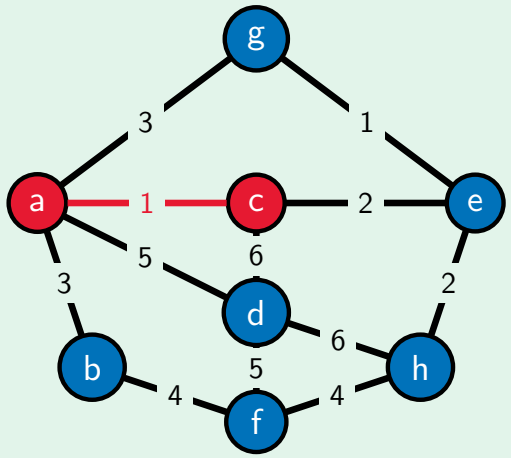
Ejemplo

Ejemplo (Primera iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8



$E = [ac, eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \emptyset$

$i = 1, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ac

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

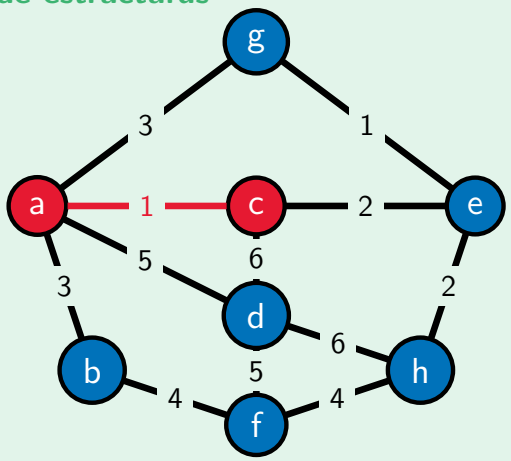
Ejemplo

Ejemplo (Primera iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	~~3~~	4	5	6	7	8
1	2	1	4	5	6	7	8



$E = [\cancel{ac}, eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac\}$ Extremos de ac en distintas componentes

$i = 1 + 1, \quad n = 8$

ac se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

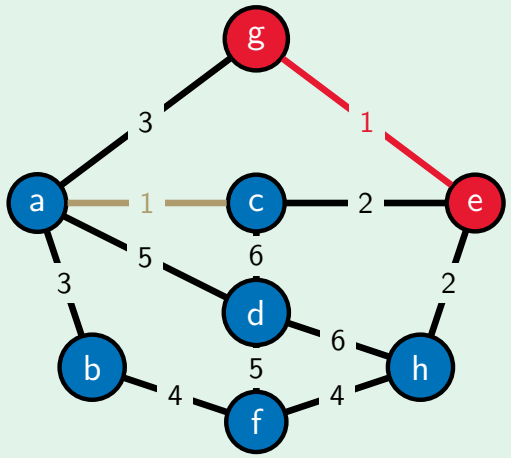
Ejemplo

Ejemplo (Segunda iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8



$E = [eg, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac\}$

$i = 2, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: eg

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

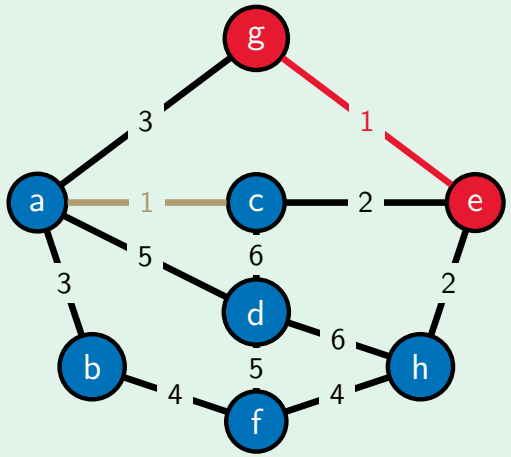
Ejemplo

Ejemplo (Segunda iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	~~7~~	8
1	2	1	4	5	6	5	8



$E = [\cancel{eg}, ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, \cancel{eg}\}$ Extremos de eg en distintas componentes

$i = 2 + 1, \quad n = 8$

eg se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

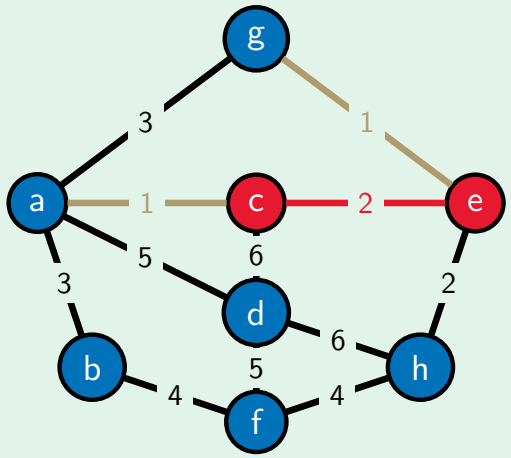
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8



$E = [ce, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg\}$

$i = 3, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ce

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

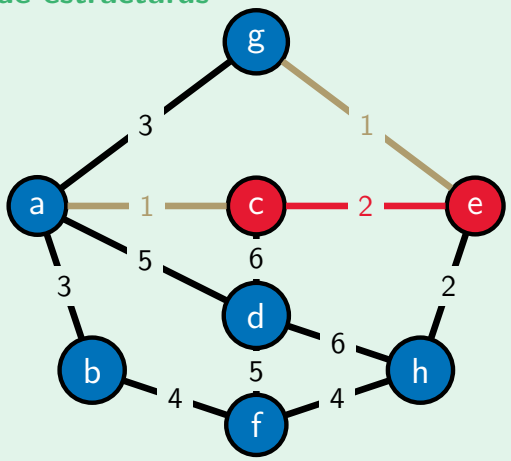
Ejemplo

Ejemplo (Tercera iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	~~5~~	6	~~5~~	8
1	2	1	4	1	6	1	8



$E = [\cancel{ce}, eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce\}$ Extremos de ce en distintas componentes

$i = 3 + 1, \quad n = 8$

ce se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

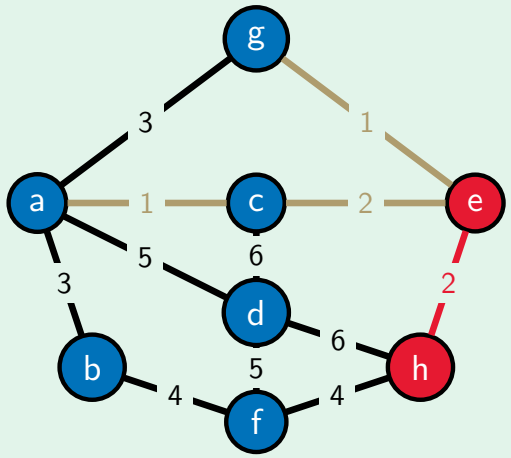
Ejemplo

Ejemplo (Cuarta iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8



$E = [eh, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce\}$

$i = 4, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: eh

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

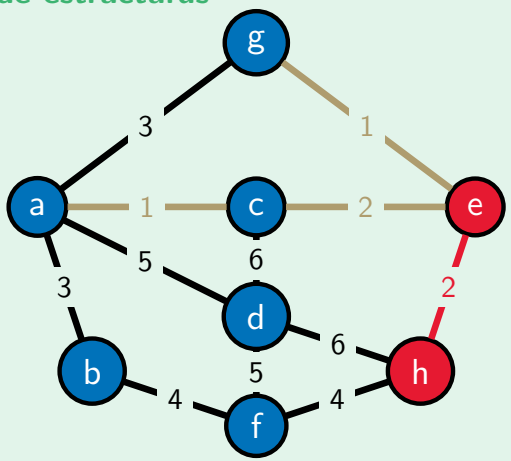
Ejemplo

Ejemplo (Cuarta iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	~~8~~
1	2	1	4	1	6	1	1



$E = [\cancel{eh}, ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh\}$ Extremos de eh en distintas componentes

$i = 4 + 1, \quad n = 8$

eh se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

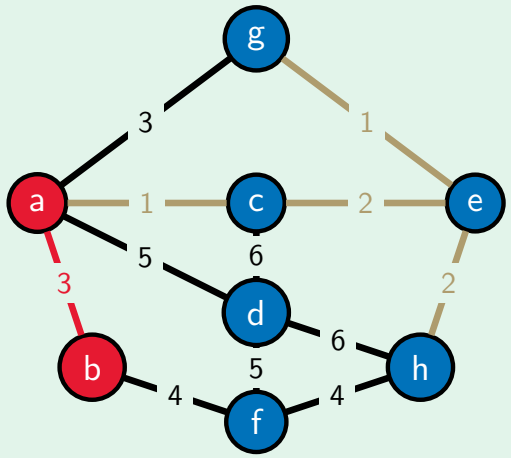
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1



$E = [ab, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh\}$

$i = 5, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ab

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

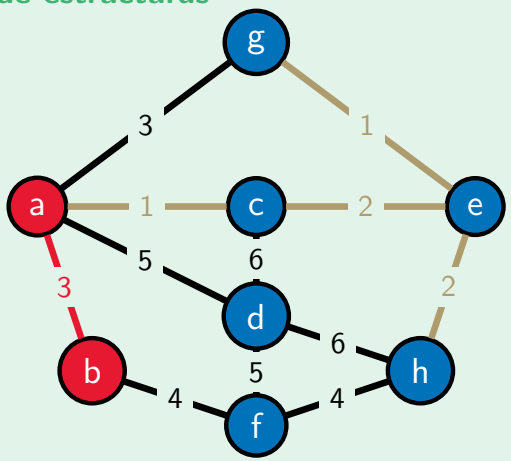
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	~~2~~	1	4	1	6	1	1
1	1	1	4	1	6	1	1



$E = [\cancel{ab}, ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

Extremos de ab en distinta componente

$i = 5 + 1, \quad n = 8$

ab se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

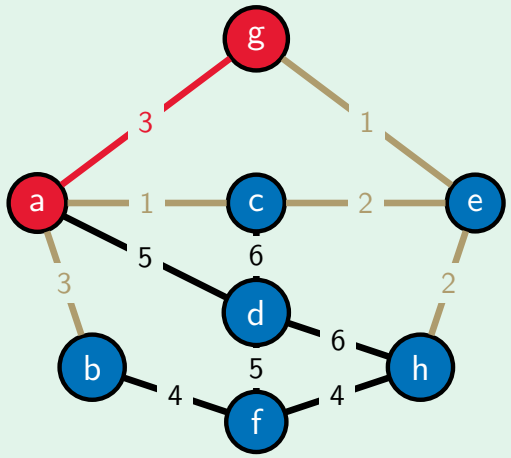
Ejemplo

Ejemplo (Sexta iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1



$E = [ag, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

$i = 6, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ag

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

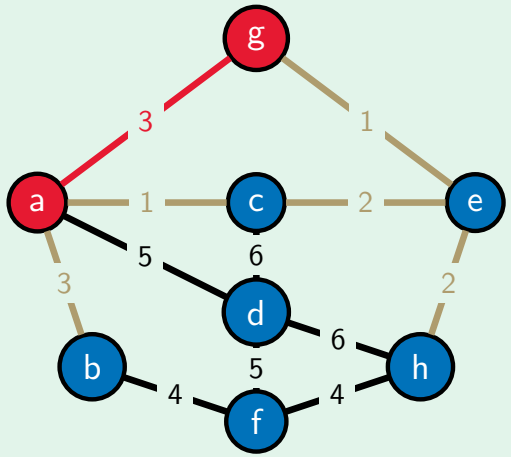
Ejemplo

Ejemplo (Sexta iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1



$E = [\cancel{ag}, bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$ Extremos de ag en misma componente

$i = 6, \quad n = 8$

ag se elimina de E

Se mantiene el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

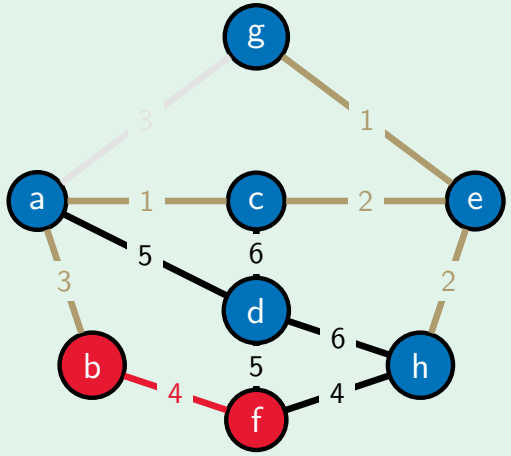
Ejemplo

Ejemplo (Séptima iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1



$E = [bf, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab\}$

$i = 6, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: bf

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

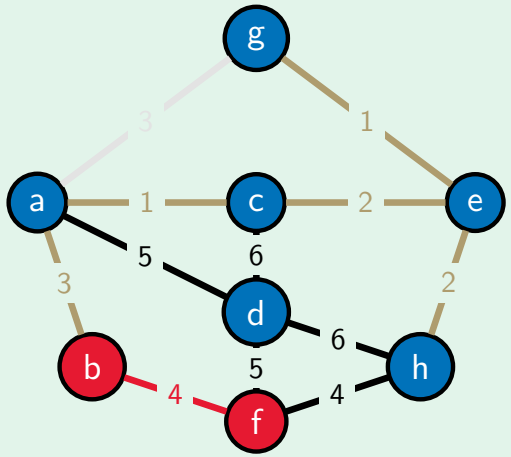
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Séptima iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	~~6~~	1	1
1	1	1	4	1	1	1	1



$E = [\cancel{bf}, fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

Extremos de bf en distintas componentes

$i = 6 + 1, \quad n = 8$

bf se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

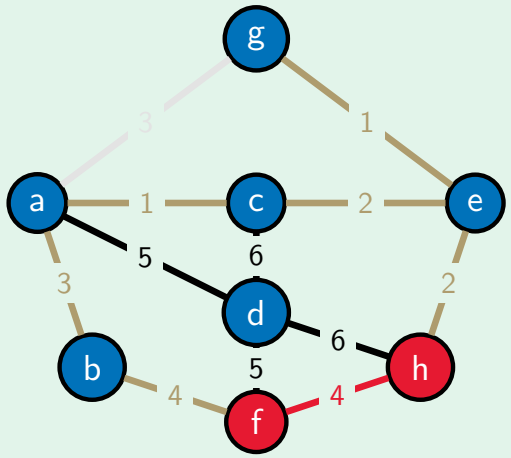
Ejemplo

Ejemplo (Octava iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1
1	1	1	4	1	1	1	1



$E = [fh, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

$i = 7, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: fh

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

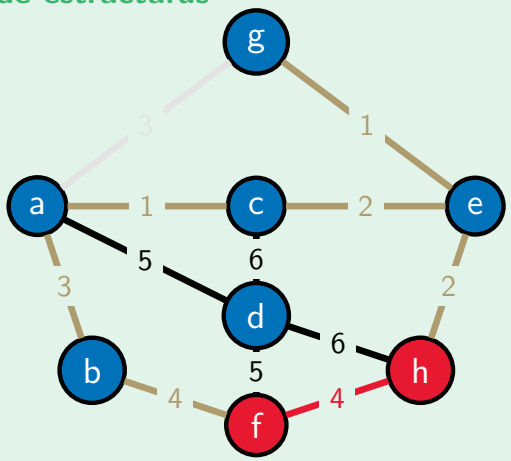
Ejemplo

Ejemplo (Octava iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1
1	1	1	4	1	1	1	1



$E = [\cancel{fh}, ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

Extremos de fh en misma componente

$i = 7, \quad n = 8$

fh se elimina de E

Se mantiene el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

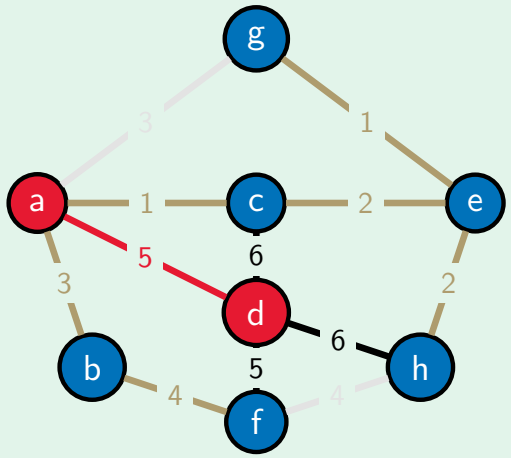
Ejemplo

Ejemplo (Novena iteración)

Comprobación de paso y selección de arista de peso mínimo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1
1	1	1	4	1	1	1	1



$E = [ad, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf\}$

$i = 7, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos una arista de peso mínimo: ad

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a ;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

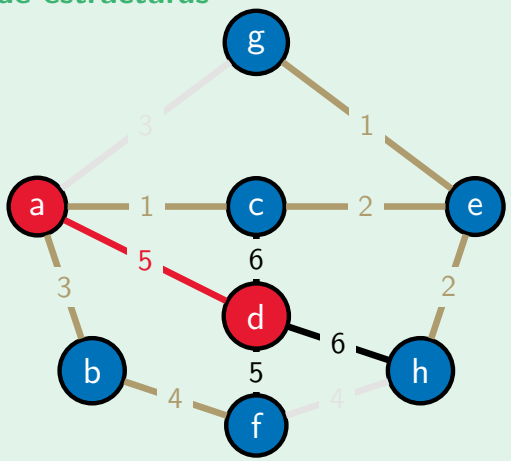
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Novena iteración)

Actualización de estructuras

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1
1	1	1	~~4~~	1	1	1	1
1	1	1	1	1	1	1	1



$E = [\cancel{ad}, df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf, ad\}$

Extremos de ad en misma componente

$i = 7 + 1, \quad n = 8$

ad se añade a E_T y se elimina de E

Se incrementa el contador

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$

if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*

$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

$E \leftarrow E - \{a\}$

/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Kruskal modificado

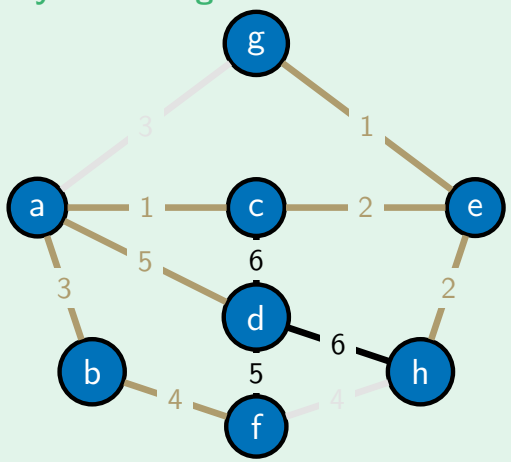
Ejemplo

Ejemplo (Décima iteración)

Comprobación de paso y fin del algoritmo

Comp

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8
1	2	1	4	5	6	7	8
1	2	1	4	5	6	5	8
1	2	1	4	1	6	1	8
1	2	1	4	1	6	1	1
1	1	1	4	1	6	1	1
1	1	1	4	1	1	1	1
1	1	1	1	1	1	1	1



$E = [df, cd, dh]$

$E_T = \{ac, eg, ce, eh, ab, bf, ad\}$

$i = 8, \quad n = 8$ No se cumple $i < n$

Finalizamos el algoritmo

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$
if $j \neq k$:

$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if

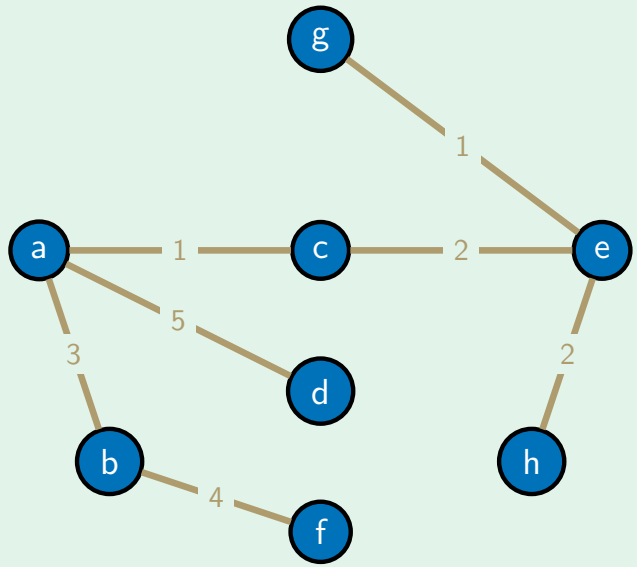
$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Resultado)

Árbol resultante



$V = \{a, b, c, d, e, f, g, h\}$

$E_T = \{ac, eg, ce, eh, ab, bf, ad\}$

$T = (V, E_T)$ es el árbol resultante

Pseudocódigo

$E \leftarrow$ ordenación de E según peso; $E_T \leftarrow \emptyset$;
$n \leftarrow |V|$; $i \leftarrow 1$; $Comp \leftarrow [(v_1, 1), \ldots, (v_n, n)]$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo de E;

$(u, v) \leftarrow$ extremos de a;

$(j, k) \leftarrow$ índices de u y v en $Comp$
if $j \neq k$:
$E_T \leftarrow E_T \cup \{a\}$

/* Añadimos la arista al árbol /*
$i \leftarrow i + 1$

en $Comp$ $k \leftarrow j$

End if
$E \leftarrow E - \{a\}$
/* Eliminamos arista del conjunto auxiliar /*

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim

Estructura del algoritmo

Input: Grafo ponderado conexo $G = (V, E, \varphi)$ con peso $w\colon E \to \mathbb{R}$ y un vértice inicial v_0.

1. Iniciamos $T = (\{v_0\}, \emptyset)$
2. Mientras haya aristas frontera:
   1. Escogemos una arista frontera $e \in E$ con menor peso $w(e)$.
   2. Añadimos al árbol T la arista e y su extremo que no está en T
3. Devuelve el árbol generador de peso mínimo $T = (V, E_T)$.

Output: Árbol generador $T = (V, E_T)$ de G con peso mínimo y raíz v_0.

Observación

El algoritmo de Prim tiene los mismos pasos que en los algoritmos de búsqueda en profundidad o anchura, sólo que en este caso se escogen las aristas frontera de menor peso.

Pseudocódigo

Input: Grafo ponderado conexo $G = (V, E, \varphi, w)$
$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$ /* Inicialización de datos /*
while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

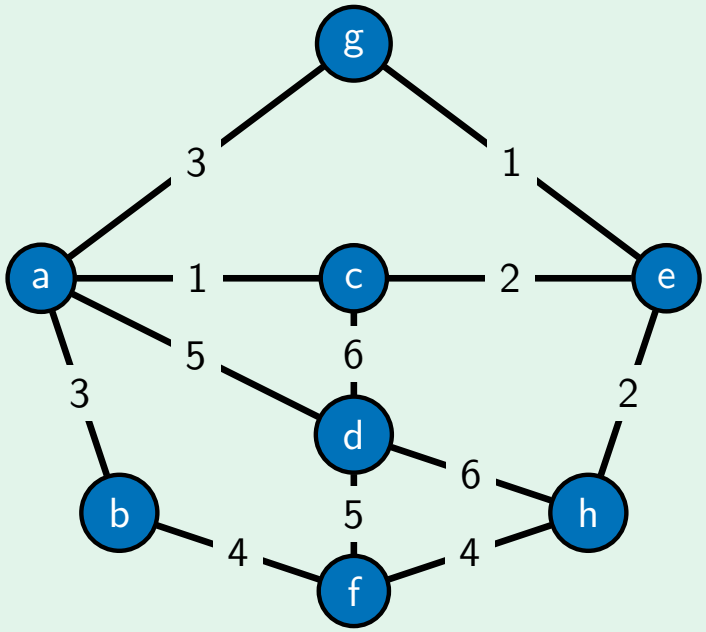
$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$ /* Trasladamos el vértice alcanzado /*

$i \leftarrow i + 1$

End while
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Inicialización)



$V_r = \{a\}, \quad V_n = \{b, c, d, e, f, g, h\}$

$E_T = \emptyset$

$i = 1, \quad n = 8$

Iniciamos los datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

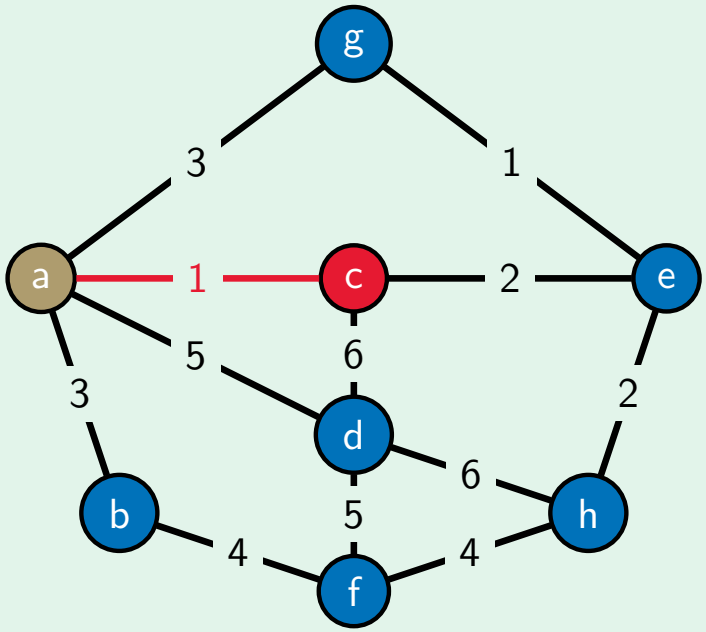
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Primera iteración)

Comprobación del paso y selección de arista



$V_r = \{a\}, \quad V_n = \{b, c, d, e, f, g, h\}$

$E_T = \emptyset$ Elegimos arista de menor peso: ac

$i = 1, \quad n = 8$

Señalamos el vértice alcanzado: c

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r
con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

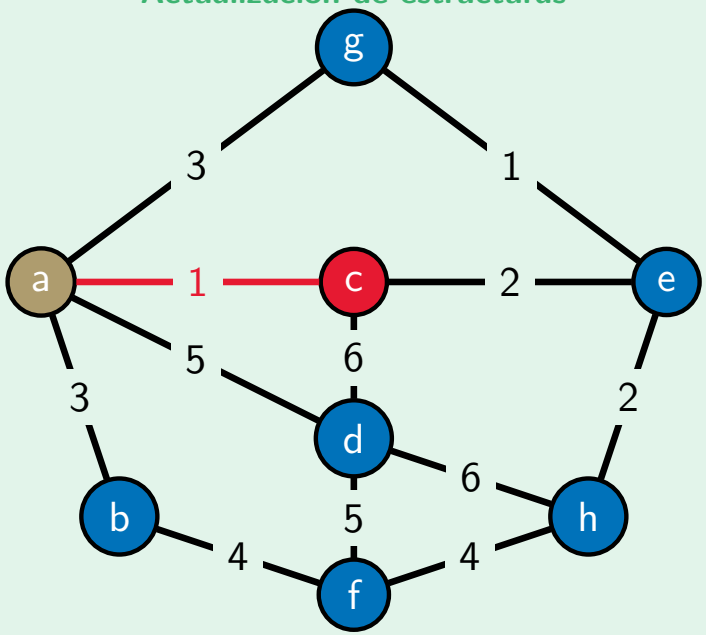
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Primera iteración)

Actualización de estructuras



$V_r = \{a, c\}$, $V_n = \{b, \not{c}, d, e, f, g, h\}$

$E_T = \{ac\}$ ac se añade a E_T

$i = 1 + 1$, $n = 8$ Se incrementa el contador

c se elimina de V_n y se añade a V_r

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

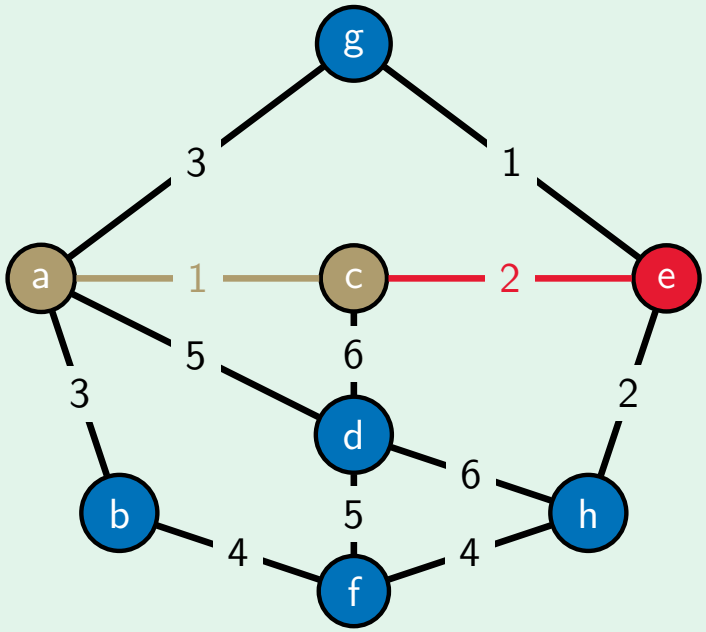
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Segunda iteración)

Comprobación del paso y selección de arista



$V_r = \{a, c\}, \quad V_n = \{b, d, e, f, g, h\}$

$E_T = \{ac\}$ Elegimos arista de menor peso: ce

$i = 2, \quad n = 8$

Señalamos el vértice alcanzado: e

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r
con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

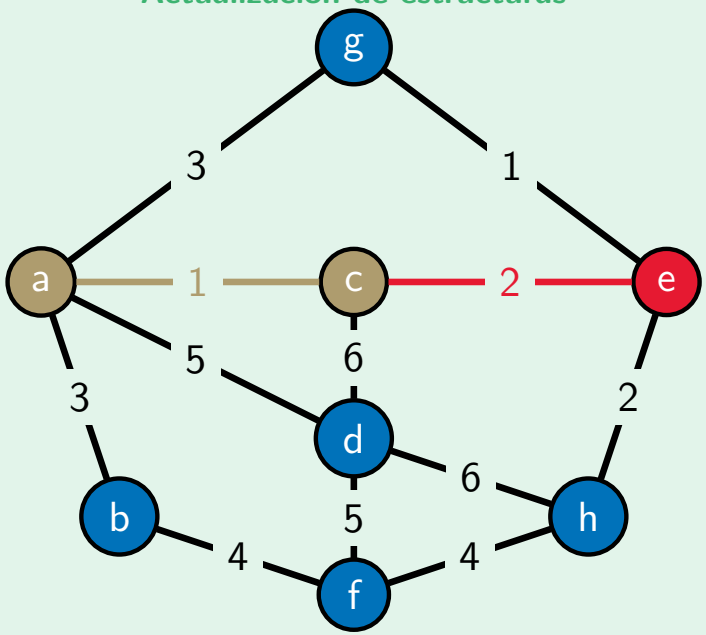
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Segunda iteración)

Actualización de estructuras



$V_r = \{a, c, e\}$, $V_n = \{b, d, \not{e}, f, g, h\}$

$E_T = \{ac, ce\}$ ce se añade a E_T

$i = 2 + 1$, $n = 8$ Se incrementa el contador

e se elimina de V_n y se añade a V_r

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

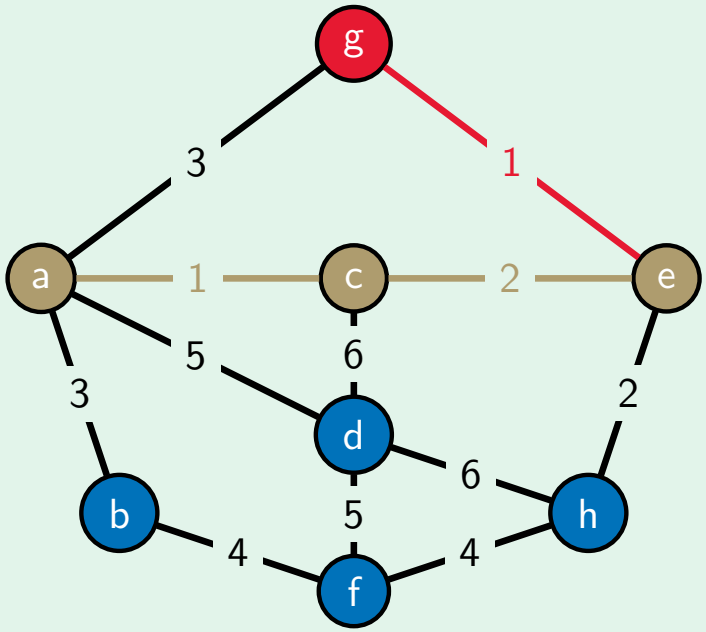
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Comprobación del paso y selección de arista



$V_r = \{a, c, e\}, \quad V_n = \{b, d, f, g, h\}$

$E_T = \{ac, ce\}$ Elegimos arista de menor peso: eg

$i = 3, \quad n = 8$

Señalamos el vértice alcanzado: g

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

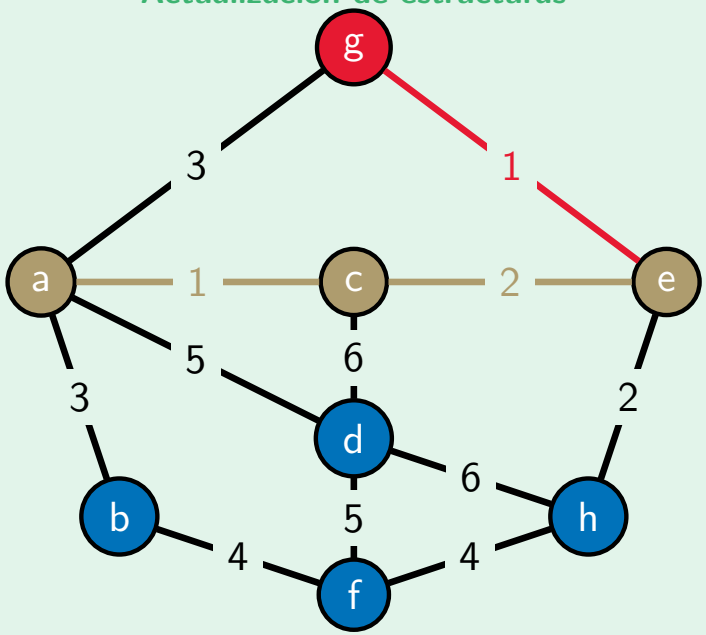
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Actualización de estructuras



$V_r = \{a, c, e, g\}$, $V_n = \{b, d, f, \cancel{g}, h\}$

$E_T = \{ac, ce, eg\}$ eg se añade a E_T

$i = 3 + 1$, $n = 8$ Se incrementa el contador

g se elimina de V_n y se añade a V_r

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

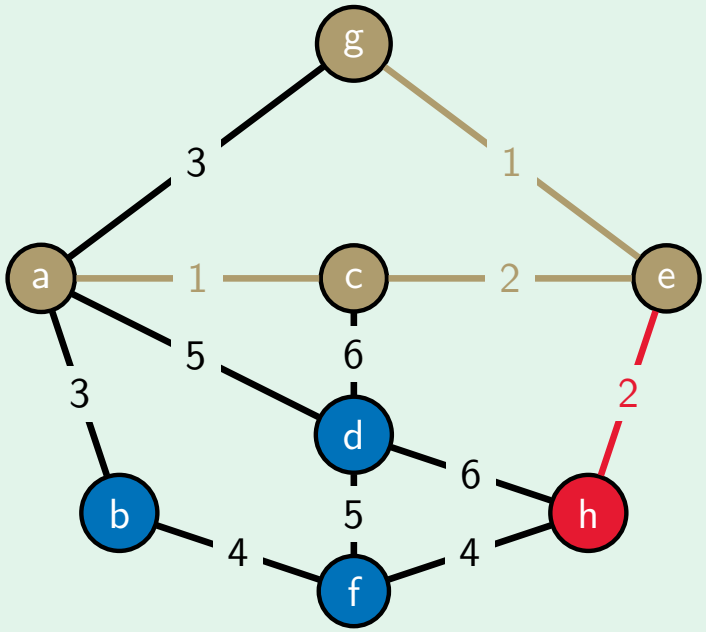
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Cuarta iteración)

Comprobación del paso y selección de arista



$V_r = \{a, c, e, g\}, \quad V_n = \{b, d, f, h\}$

$E_T = \{ac, ce, eg\}$

Elegimos arista de menor peso: eh

$i = 4, \quad n = 8$

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r
con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

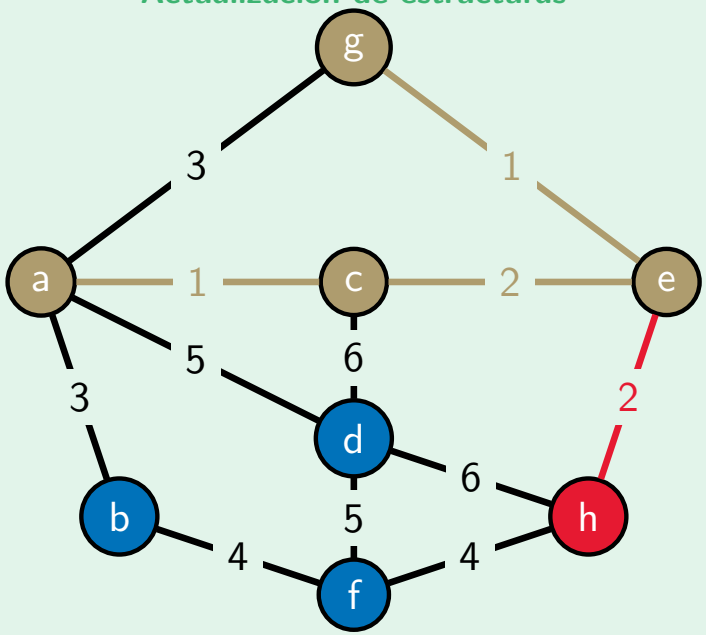
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Cuarta iteración)

Actualización de estructuras



$V_r = \{a, c, e, g, h\}$, $V_n = \{b, d, f, \not{h}\}$

$E_T = \{ac, ce, eg, eh\}$ eh se añade a E_T

$i = 4 + 1$, $n = 8$ Se incrementa el contador

g se elimina de V_n y se añade a V_r

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

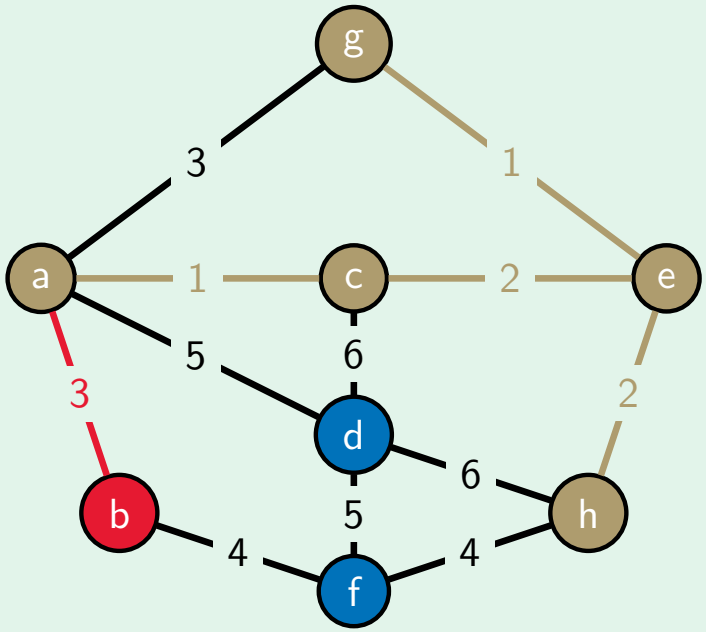
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Comprobación del paso y selección de arista



$V_r = \{a, c, e, g, h\}, \quad V_n = \{b, d, f\}$

$E_T = \{ac, ce, eg, eh\}$

Elegimos arista de menor peso: ab

$i = 5, \quad n = 8$

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r
con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

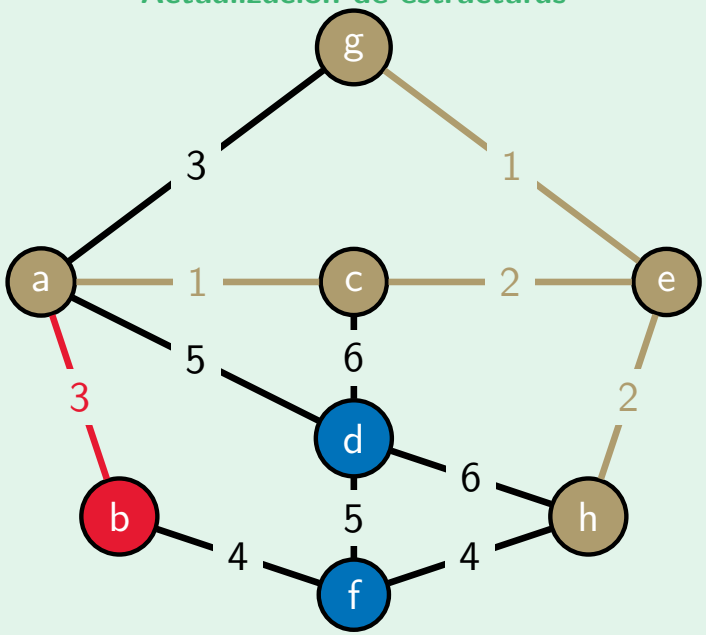
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Actualización de estructuras



$V_r = \{a, b, c, e, g, h\}$, $V_n = \{\not{b}, d, f\}$

$E_T = \{ac, ce, eg, eh, ab\}$ ab se añade a E_T

$i = 5 + 1$, $n = 8$ Se incrementa el contador

b se elimina de V_n y se añade a V_r

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

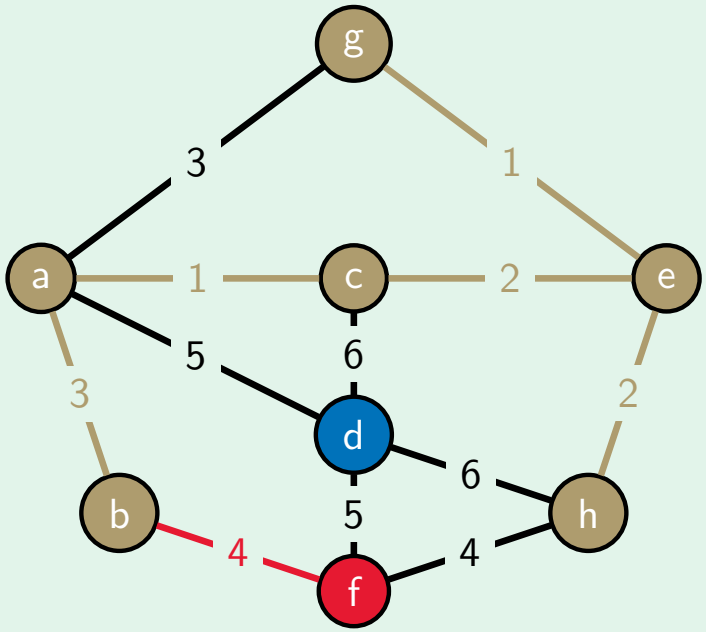
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Sexta iteración)

Comprobación del paso y selección de arista



$V_r = \{a, b, c, e, g, h\}, \quad V_n = \{d, f\}$

$E_T = \{ac, ce, eg, eh, ab\}$

Elegimos arista de menor peso: bf

$i = 6, \quad n = 8$

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r
con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

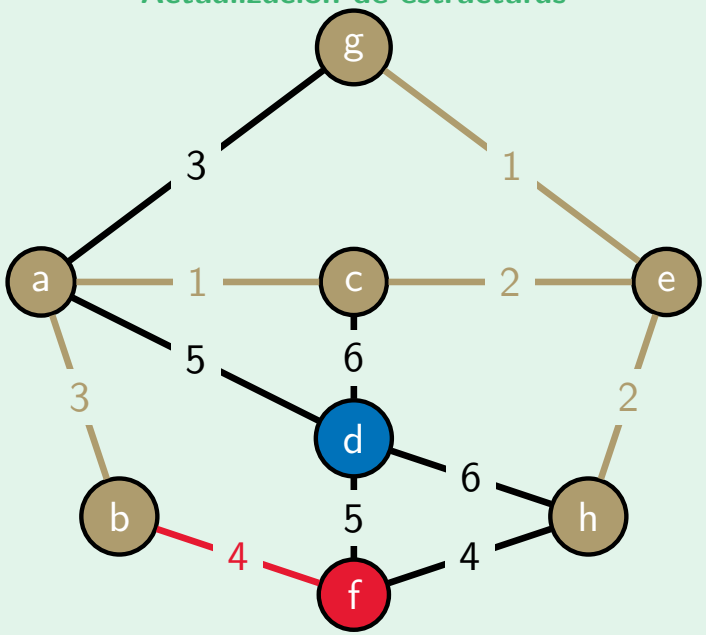
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Sexta iteración)

Actualización de estructuras



$V_r = \{a, b, c, e, f, g, h\}$, $V_n = \{d, \cancel{f}\}$

$E_T = \{ac, ce, eg, eh, ab, bf\}$ bf se añade a E_T

$i = 6 + 1$, $n = 8$ Se incrementa el contador

f se elimina de V_n y se añade a V_r

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

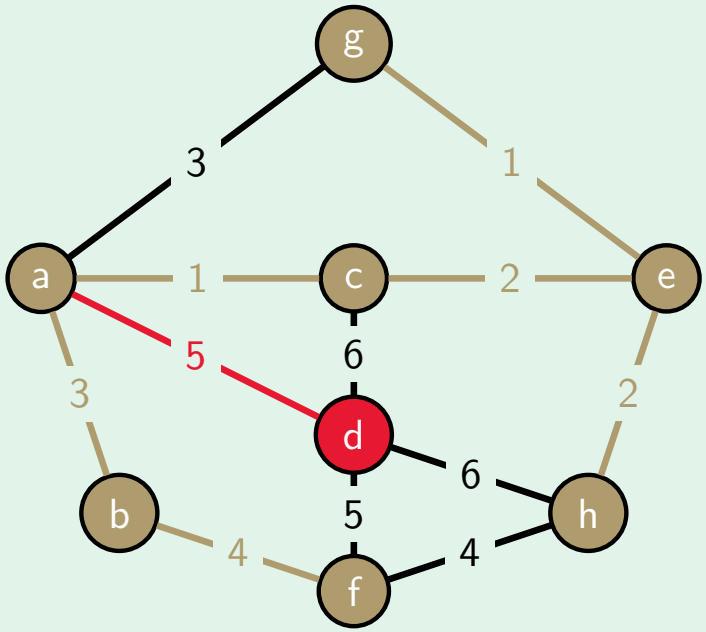
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Séptima iteración)

Comprobación del paso y selección de arista



$V_r = \{a, b, c, e, f, g, h\}, \quad V_n = \{d\}$

$E_T = \{ac, ce, eg, eh, ab, bf\}$

Elegimos arista de menor peso: ad

$i = 7, \quad n = 8$

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r
con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

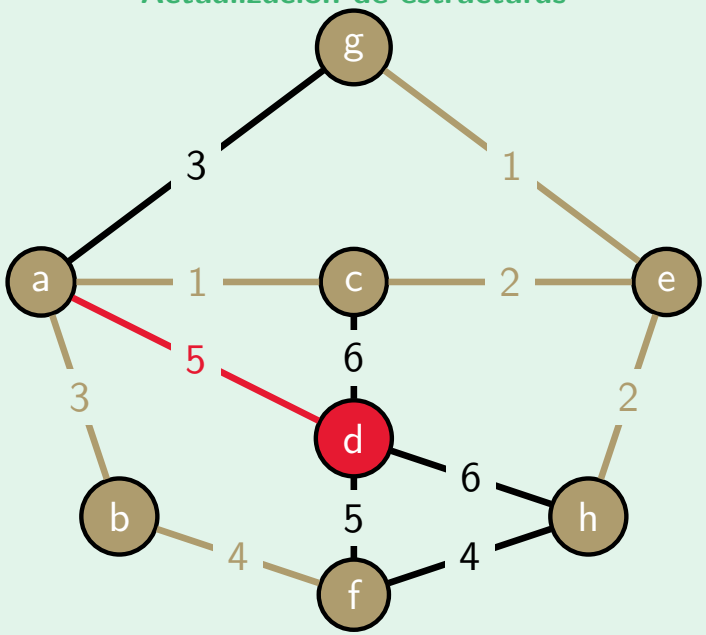
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Séptima iteración)

Actualización de estructuras



$V_r = \{a, b, c, d, e, f, g, h\}$, $V_n = \{\not{\phi}\}$

$E_T = \{ac, ce, eg, eh, ab, bf, ad\}$

ad se añade a E_T

$i = 7 + 1$, $n = 8$ Se incrementa el contador

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

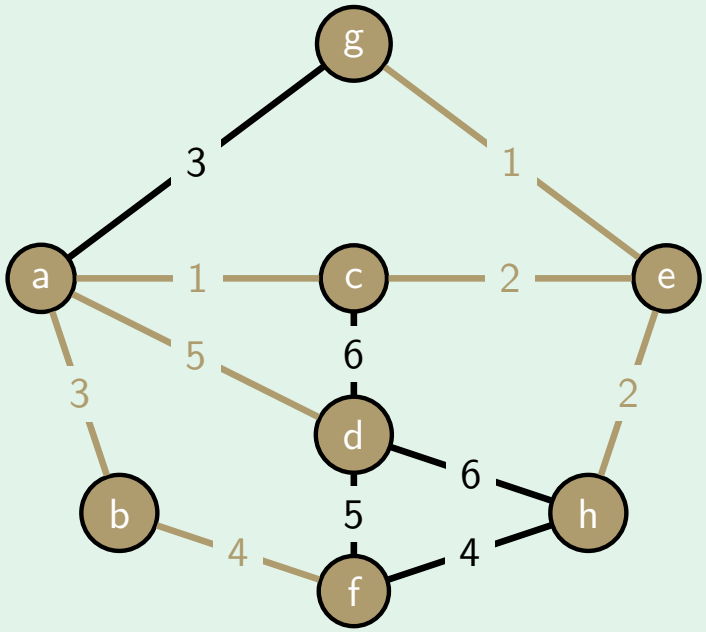
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Octava iteración)

Comprobación del paso y fin del algoritmo



$V_r = \{a, b, c, d, e, f, g, h\}, \quad V_n = \emptyset$

$E_T = \{ac, ce, eg, eh, ab, bf, ad\}$

$i = 8, \quad n = 8$ No se cumple $i < n$

Fin del algoritmo

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

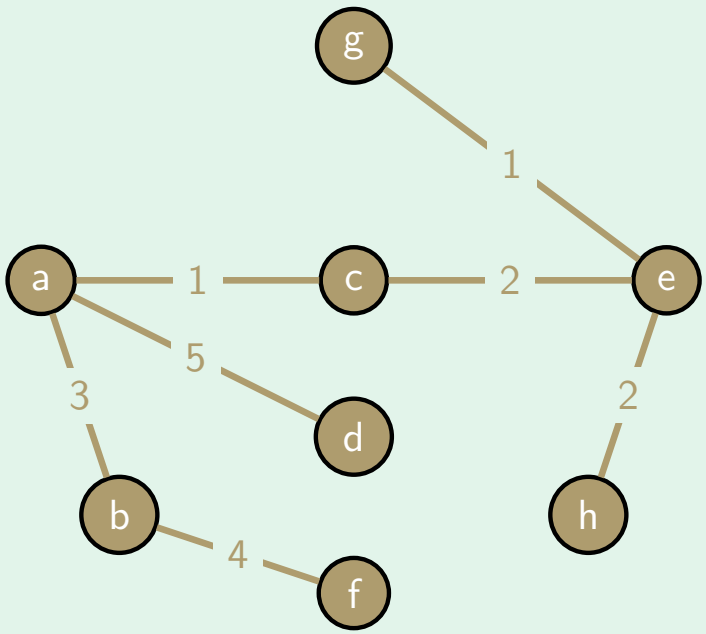
$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Resultado)

Árbol resultante



$V = \{a, b, c, d, e, f, g, h\}$

$E_T = \{ac, ce, eg, eh, ab, bf, ad\}$

$T = (V, E_T)$ es el árbol resultante

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim

Teorema de validez

Teorema: Validez del algoritmo de Prim

Dado un grafo conexo y ponderado, el algoritmo de Prim construye un árbol generador de peso mínimo para dicho grafo.

Idea de la demostración.

Sea G un grafo conexo, de n vértices, con función de peso w.
Sea T un grafo obtenido al aplicar el algoritmo de Prim sobre G.

1. Observa que T es un árbol generador:
2. Comprueba que T tiene peso mínimo, por contradicción.
    1. Supón que T no tiene peso mínimo.
    2. Toma S un árbol generador de peso mínimo que contiene el mayor número de aristas de T (en orden).
    3. Construye otro árbol S^* de peso mínimo que contenga más aristas de T
    4. **¡Contradicción!**

Algoritmo de Prim

Análisis de complejidad

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$;
$V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
/* Inicialización de datos /*

while $i < n$

$a \leftarrow$ arista de peso mínimo que une
vértice de V_r con vértice de V_n

$v \leftarrow$ vértice extremo de a en V_n

$E_T \leftarrow E_T \cup \{a\}$
/* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$i \leftarrow i + 1$

End while

Return $T = (V, E_T)$ árbol de peso mínimo

La selección de aristas
con estas condiciones
es el paso que añade
más complejidad al algoritmo

La información de aristas
con peso mínimo
que unen vértices de V_r
con otros vértices de V_n
se puede codificar
en una estructura de datos

Sustituimos la comprobación
de aristas de peso mínimo
por la comprobación de
la estructura de datos

Pseudocódigo modificado

Input: Grafo ponderado conexo $G = (V, E, \varphi, w)$
$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$; si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima; $a \leftarrow$ arista $t_1(v)v$

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$ /* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$;

for z en V_n

if $t_2(z) > w(vz)$, **then** $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

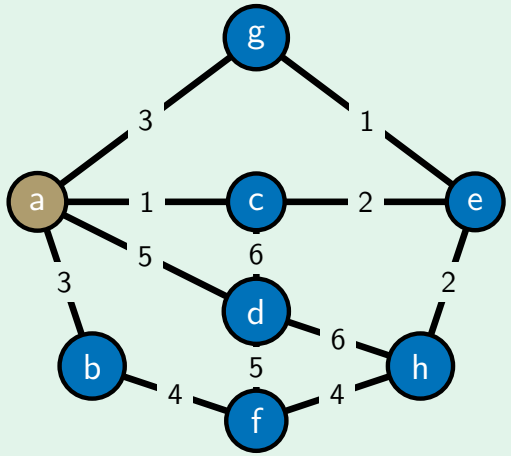
Algoritmo de Prim modificado

Ejemplo

Ejemplo (Inicialización)

Estructura de datos

V_r	a						
V_n	b	c	d	e	f	g	h
t_1	a	a	a			a	
t_2	3	1	5	∞	∞	3	∞



$V_r = \{a\}, \quad V_n = \{b, c, d, e, f, g, h\}$

$E_T = \emptyset$

$i = 1, \quad n = 8$

Iniciamos los datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

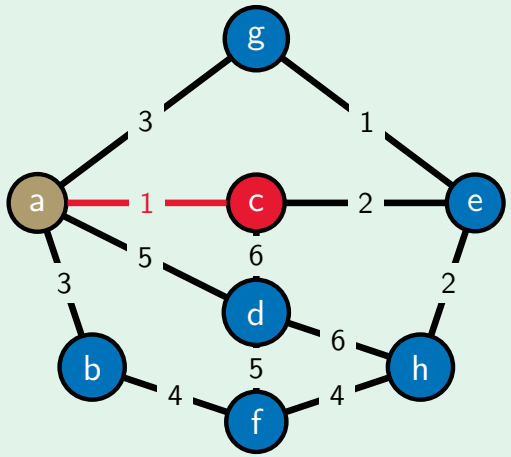
Ejemplo

Ejemplo (Primera iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a						
V_n	b	c	d	e	f	g	h
t_1	a	a	a			a	
t_2	3	1	5	∞	∞	3	∞



$V_r = \{a\}, \quad V_n = \{b, c, d, e, f, g, h\}$

$E_T = \emptyset$

$i = 1, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, c, y arista ac

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

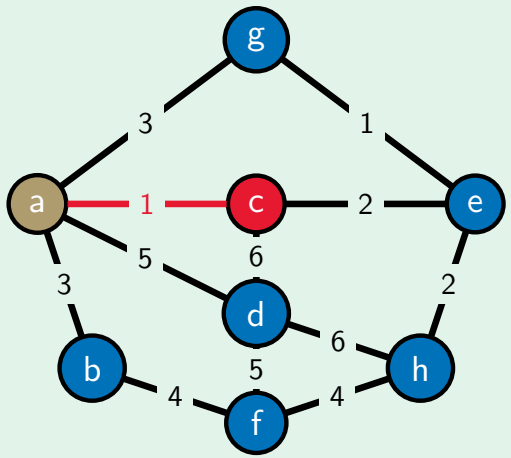
Ejemplo

Ejemplo (Primera iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c					
V_n	b	~~c~~	d	e	f	g	h
t_1	a		a	c		a	
t_2	3		5	2	∞	3	∞



$V_r = \{a, c\}, \quad V_n = \{b, \not{c}, d, e, f, g, h\}$

$E_T = \{ac\}$ ac se añade a E_T

$i = 1 + 1, \quad n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

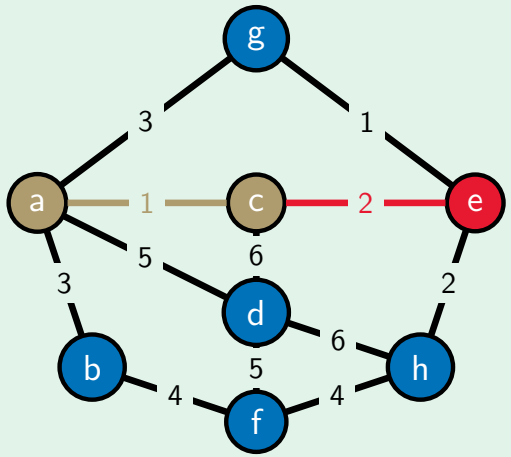
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Segunda iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a	c					
V_n	b		d	e	f	g	h
t_1	a		a	c		a	
t_2	3		5	2	∞	3	∞



$V_r = \{a, c\}, \quad V_n = \{b, d, e, f, g, h\}$

$E_T = \{ac\}$

$i = 2, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, e, y arista ce

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

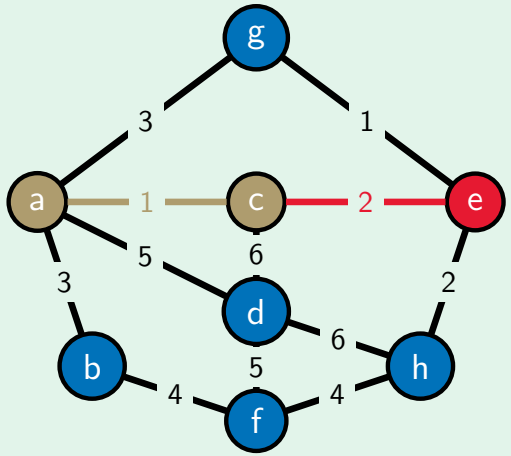
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Segunda iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c	e				
V_n	b		d	~~e~~	f	g	h
t_1	a		a			e	e
t_2	3		5		∞	1	2



$V_r = \{a, c, e\}$, $V_n = \{b, d, ~~e~~, f, g, h\}$

$E_T = \{ac, ce\}$ ce se añade a E_T

$i = 2 + 1$, $n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

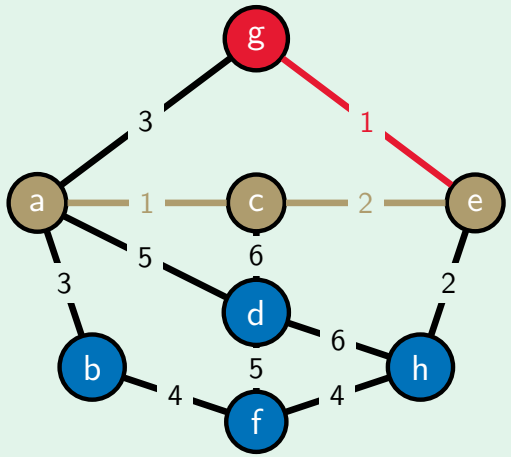
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a	c	e				
V_n	b		d		f	g	h
t_1	a		a			e	e
t_2	3		5		∞	1	2



$V_r = \{a, c, e\}, \quad V_n = \{b, d, f, g, h\}$

$E_T = \{ac, ce\}$

$i = 3, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, g, y arista eg

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for

End while

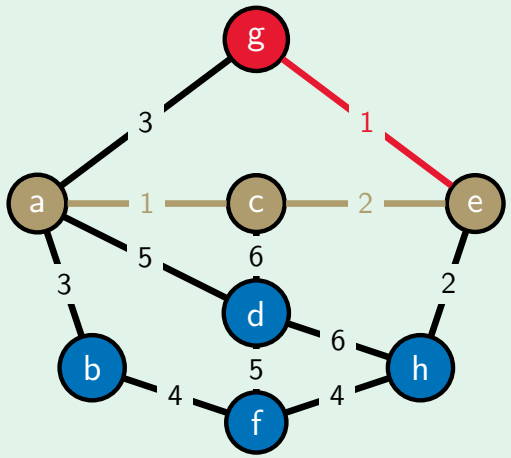
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Tercera iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c	e	g			
V_n	b		d		f	~~$\not{g}$~~	h
t_1	a		a				e
t_2	3		5		∞		2



$V_r = \{a, c, e, g\}$, $V_n = \{b, d, f, \not{g}, h\}$

$E_T = \{ac, ce, eg\}$ eg se añade a E_T

$i = 3 + 1$, $n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

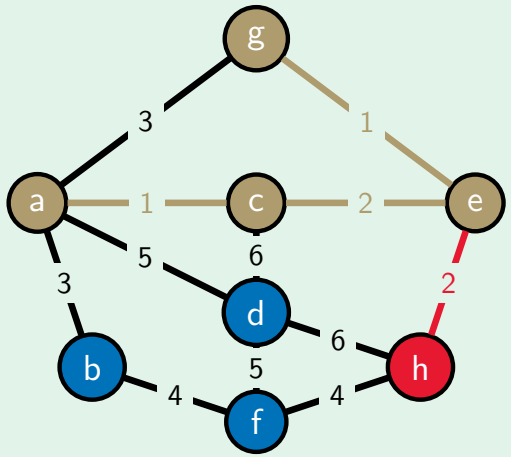
Ejemplo

Ejemplo (Cuarta iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a	c	e	g			
V_n	b		d		f		h
t_1	a		a				e
t_2	3		5		∞		2



$V_r = \{a, c, e, g\}, \quad V_n = \{b, d, f, h\}$

$E_T = \{ac, ce, eg\}$

$i = 4, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, h, y arista eh

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for

End while

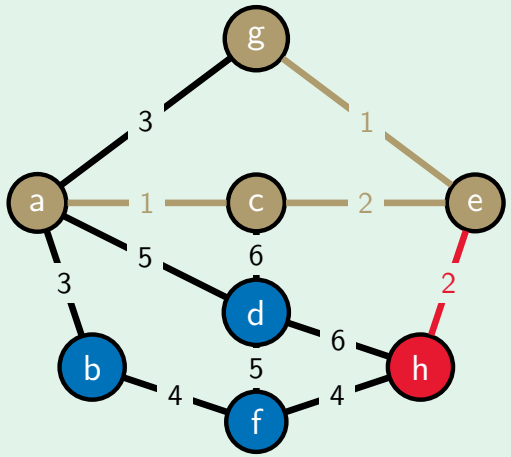
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Cuarta iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c	e	g	h		
V_n	b		d		f		~~h~~
t_1	a		a		h		
t_2	3		5		4		



$V_r = \{a, c, e, g, h\}$, $V_n = \{b, d, f, ~~h~~\}$

$E_T = \{ac, ce, eg, eh\}$ *eh* se añade a E_T

$i = 4 + 1$, $n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

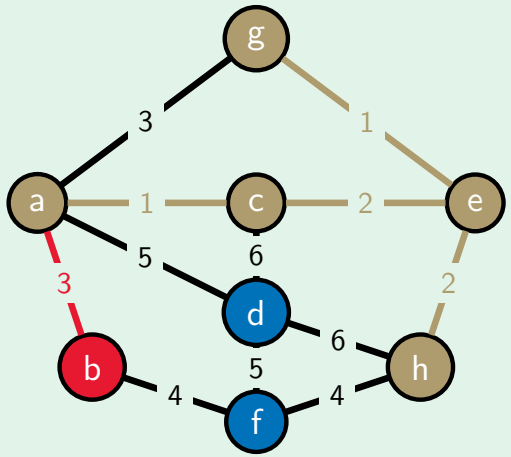
Ejemplo

Ejemplo (Quinta iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a	c	e	g	h		
V_n	b		d		f		
t_1	a		a		h		
t_2	3		5		4		



$V_r = \{a, c, e, g, h\}, \quad V_n = \{b, d, f\}$

$E_T = \{ac, ce, eg, eh\}$

$i = 5, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, b, y arista ab

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for

End while

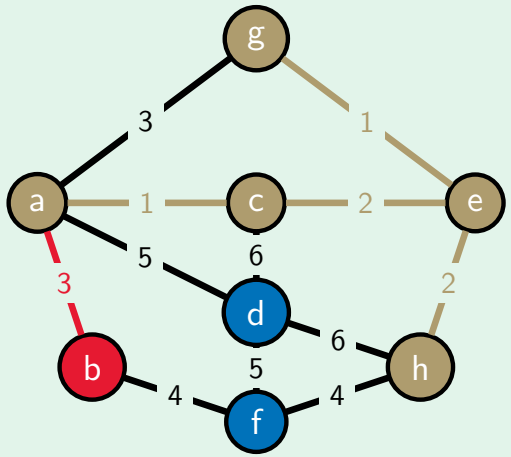
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Quinta iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c	e	g	h	b	
V_n	~~b~~		d		f		
t_1			a		h		
t_2			5		4		



$V_r = \{a, b, c, e, g, h, \}, \quad V_n = \{\not{b}, d, f\}$

$E_T = \{ac, ce, eg, eh, ab\}$ ab se añade a E_T

$i = 5 + 1, \quad n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

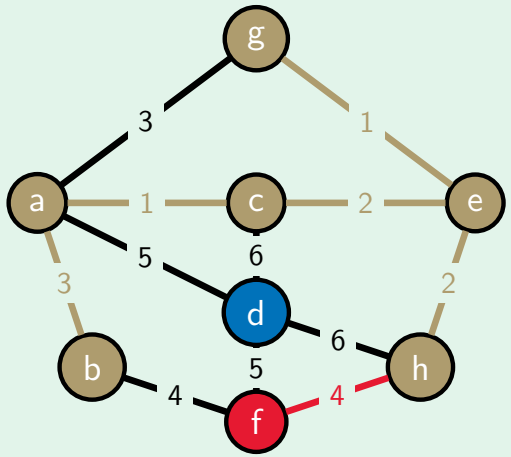
Ejemplo

Ejemplo (Sexta iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a	c	e	g	h	b	
V_n			d		f		
t_1			a		h		
t_2			5		4		



$V_r = \{a, b, c, e, g, h\}, \quad V_n = \{d, f\}$

$E_T = \{ac, ce, eg, eh, ab\}$

$i = 6, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, f, y arista fh

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for

End while

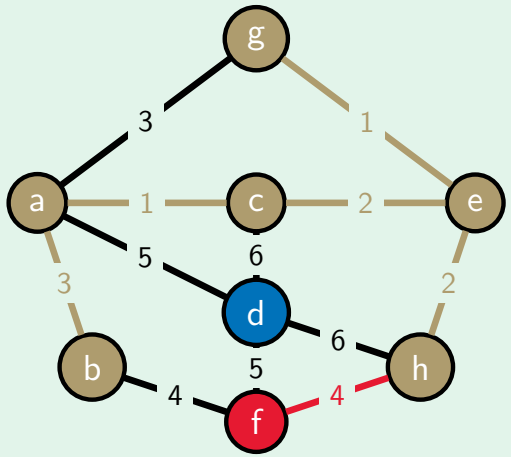
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Sexta iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c	e	g	h	b	f
V_n			d		~~f~~		
t_1			a				
t_2			5				



$V_r = \{a, b, c, e, f, g, h, \}, \quad V_n = \{d, f\}$

$E_T = \{ac, ce, eg, eh, ab, fh\}$ fh se añade a E_T

$i = 6 + 1, \quad n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

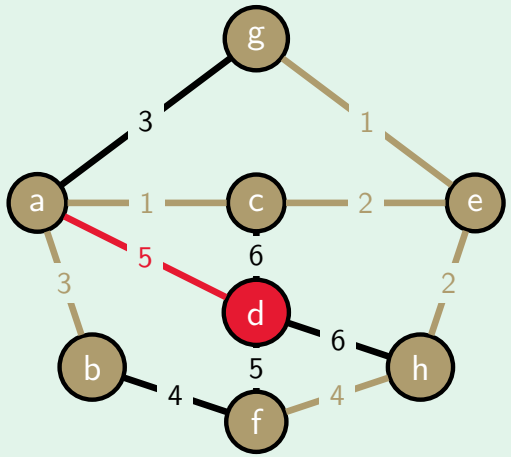
Ejemplo

Ejemplo (Séptima iteración)

Comprobación del paso y selección de vértice

Estructura de datos

V_r	a	c	e	g	h	b	f
V_n			d				
t_1			a				
t_2			5				



$V_r = \{a, b, c, e, f, g, h\}, \quad V_n = \{d\}$

$E_T = \{ac, ce, eg, eh, ab, fh\}$

$i = 7, \quad n = 8$ Se comprueba que $i < n$

Seleccionamos vértice con menor etiqueta t_2, d, y arista ad

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for

End while

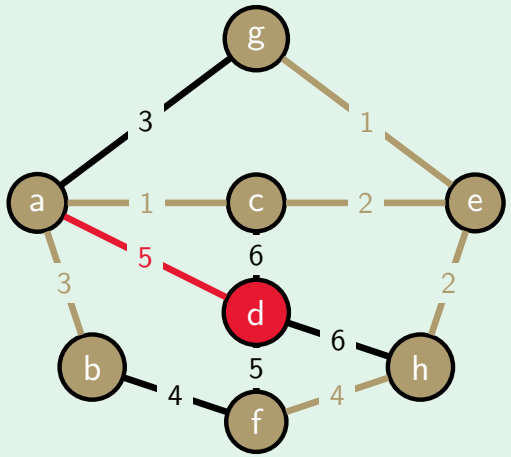
Return $T = (V, E_T)$ árbol de peso mínimo

Ejemplo (Séptima iteración)

Actualización de estructuras

Estructura de datos

V_r	a	c	e	g	h	b	f
V_n			$\not{e}$				
t_1							
t_2							



$V_r = \{a, b, c, d, e, f, g, h\}$, $\quad V_n = \{\not{e}\}$

$E_T = \{ac, ce, eg, eh, ab, fh, ad\}$ ad se añade a E_T

$i = 7 + 1$, $\quad n = 8$ Se incrementa el contador

Actualizamos la estructura de datos

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$

for z en V_n

si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;

si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$

end for /* Inicialización de datos /*

while $i < n$

$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;

$a \leftarrow$ arista $t_1(v)v$;

$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*

$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$

/* Trasladamos el vértice alcanzado /*

$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$

for z en V_n

if $t_2(z) > w(vz)$,

then $t(z) \leftarrow [v, w(vz)]$ **end if**

end for

End while

Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

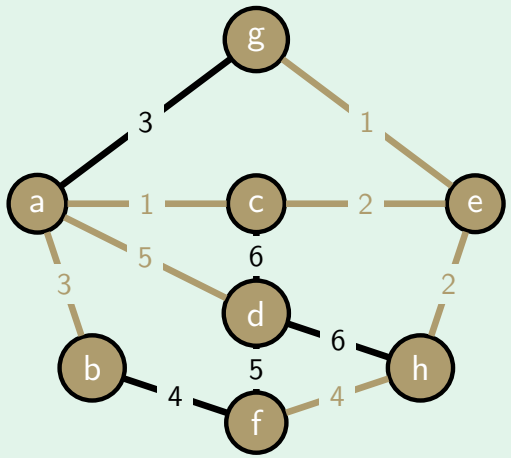
Ejemplo

Ejemplo (Octava iteración)

Comprobación del paso y fin del algoritmo

Estructura de datos

V_r	a	c	e	g	h	b	f	d
V_n								
t_1								
t_2								



$V_r = \{a, b, c, d, e, f, g, h\}, \quad V_n = \emptyset$

$E_T = \{ac, ce, eg, eh, ab, fh, ad\}$

$i = 8, \quad n = 8$ No se cumple $i < n$

Fin del algoritmo

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$
$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;
$a \leftarrow$ arista $t_1(v)v$;
$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$
for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for
End while

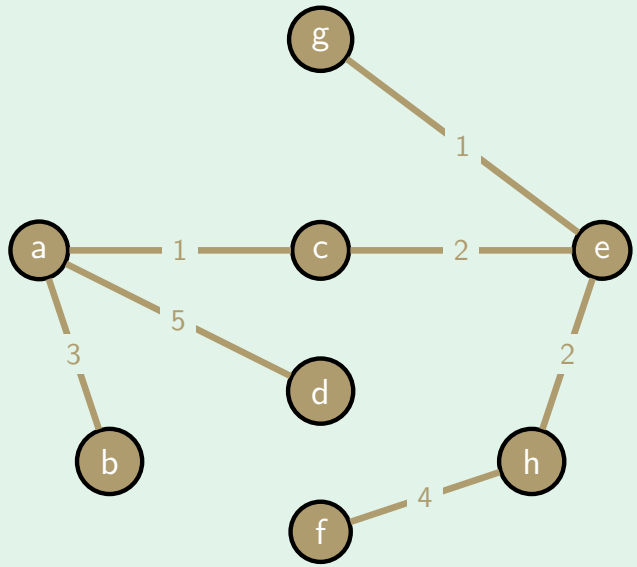
Return $T = (V, E_T)$ árbol de peso mínimo

Algoritmo de Prim modificado

Ejemplo

Ejemplo (Resultado)

Árbol resultante



$$V = \{a, b, c, d, e, f, g, h\}$$

$$E_T = \{ac, ce, eg, eh, ab, fh, ad\}$$

$T = (V, E_T)$ es el árbol resultante

Pseudocódigo

$E_T \leftarrow \emptyset$; $n \leftarrow |V|$; $i \leftarrow 1$; $V_r \leftarrow \{v_0\}$; $V_n \leftarrow V - \{v_0\}$
for z en V_n
si existe v_0z: $t(z) \leftarrow [v_0, w(v_0z)]$;
si no existe v_0z: $t(z) \leftarrow [\cdot, \infty]$
end for /* Inicialización de datos /*

while $i < n$
$v \leftarrow$ vértice en V_n con etiqueta $t_2(z)$ mínima;
$a \leftarrow$ arista $t_1(v)v$;
$E_T \leftarrow E_T \cup \{a\}$ /* Añadimos la arista al árbol /*
$V_r \leftarrow V_r \cup \{v\}$; $V_n \leftarrow V_n - \{v\}$
/* Trasladamos el vértice alcanzado /*
$t(v) \leftarrow [\cdot, \cdot]$; $i \leftarrow i + 1$
for z en V_n
if $t_2(z) > w(vz)$,
then $t(z) \leftarrow [v, w(vz)]$ **end if**
end for
End while

Return $T = (V, E_T)$ árbol de peso mínimo